

OpenCPU

Development Guide

GSM/GPRS Module Series

Rev. OPEN_CPU_DGD_V3.1

Date: 2013-01-05



Our aim is to provide customers with timely and comprehensive service. For any assistance, please contact our company headquarter:

Quectel Wireless Solutions Co., Ltd.

Room 501, Building 13, No.99, Tianzhou Road, Shanghai, China, 200233

Tel: +86 21 5108 6236

Mail: info@quectel.com

Or our local office, for more information, please visit:

http://www.quectel.com/quectel_sales_office.html

For technical support, to report documentation errors, please visit:

<http://www.quectel.com/tecsupport.aspx>

GENERAL NOTES

QUECTEL OFFERS THIS INFORMATION AS A SERVICE TO ITS CUSTOMERS. THE INFORMATION PROVIDED IS BASED UPON CUSTOMERS' REQUIREMENTS. QUECTEL MAKES EVERY EFFORT TO ENSURE THE QUALITY OF THE INFORMATION IT MAKES AVAILABLE. QUECTEL DOES NOT MAKE ANY WARRANTY AS TO THE INFORMATION CONTAINED HEREIN, AND DOES NOT ACCEPT ANY LIABILITY FOR ANY INJURY, LOSS OR DAMAGE OF ANY KIND INCURRED BY USE OF OR RELIANCE UPON THE INFORMATION. ALL INFORMATION SUPPLIED HEREIN ARE SUBJECT TO CHANGE WITHOUT PRIOR NOTICE.

COPYRIGHT

THIS INFORMATION CONTAINED HERE IS PROPRIETARY TECHNICAL INFORMATION OF QUECTEL CO., LTD. TRANSMITTABLE, REPRODUCTION, DISSEMINATION AND EDITING OF THIS DOCUMENT AS WELL AS UTILIZATION OF THIS CONTENTS ARE FORBIDDEN WITHOUT PERMISSION. OFFENDERS WILL BE HELD LIABLE FOR PAYMENT OF DAMAGES. ALL RIGHTS ARE RESERVED IN THE EVENT OF A PATENT GRANT OR REGISTRATION OF A UTILITY MODEL OR DESIGN.

Copyright © Quectel Wireless Solutions Co., Ltd. 2012. All rights reserved.

About the document

History

Revision	Date	Author	Description
3.0	2012-01-05	Stanley YONG	Updated TCP/IP, Call related APIs
3.1	2012-08-10	Ramos ZHANG	- Added APIs: I2C related - Modified SMS APIs - Added QI_Fs_Format API - Added QI_FileGetFreeSizeEx - Added APIs: Audio record related - Added APIs:Tone related - Added APIs: LowLevelTimer related - Added APIs:Location related - Added APIs : CFUN related - Added API set sleepMode - Added API QI_Alarm_StartUP - Added API QI_switchHeartBeat - Added API AI_PwrKey_Init.
	2012-09-28	Stanley YONG	Added QI_TTS_PlayInCall()
	2012-12-18	Stanley YONG	- Added a group of API functions for decoding DTMF. QI_EnableDTMF(),QI_DisableDTMF(),QI_GetDTMFConfig(),QI_SetDTMFConfig() - Added QI_ChangeUsbMode() to switch the USB working mode. - Rename QI_StarLowLevelTimer to QI_StartGPTimer, QI_StopLowLevelTimer to QI_StopGPTimer, and QI_UartDirectnessReadData to QI_UartRcvData. But the old names are still permitted for compatibility. - Added QI_Call_GetCurrCall() to retrieve the current call connection information. - Added QI_SetRingToneEnable() to switch on/off the ring of the coming call. - Added QI_GenericURC_Init() to register

	URC-related callback, which will be triggered when SIM card is lost during running.
--	---

Quectel
Confidential

Contents

About the document	2
Contents	4
Table Index	12
1 Introduction	13
2 OpenCPU SDK	14
3 OpenCPU Software Architecture	15
3.1. Software architecture	15
3.2. Application code frame	16
3.2.1. Least Application Code	16
3.2.2. Application entry	16
3.2.3. Application termination	16
3.3. Memory resources	17
4 Basic data types	18
5 Event	19
5.1. Event type	19
5.1.1. EVENT_INTR	20
5.1.2. EVENT_KEY	20
5.1.3. EVENT_UARTDATA	20
5.1.4. EVENT_UARTREADY	20
5.1.5. EVENT_MODMEDATA	20
5.1.6. EVENT_TIMER	20
5.1.7. EVENT_SERIALSTATUS	21
5.1.8. EVENT_MSG	21
5.1.9. EVENT_POWERKEY	21
5.1.10. EVENT_HEADSET	21
5.1.11. EVENT_UARTESCAPE	21
5.1.12. EVENT_UARTFE	22
5.1.13. EVENT_MEDIA_FINISH	22
5.2. Event Data Structure	22
5.2.1. Event Data Union	22
5.2.2. Timer_Event	23
5.2.3. Key_Event	23
5.2.4. PortData_Event	23
5.2.5. Intr_Event	24
5.2.6. PortStatus_Event	25
5.2.7. Msg_Event	25
5.2.8. Powerkey_Event	25
5.2.9. Headset_Event	26
5.2.10. UartEscape_Event	26

5.2.11. UartFE_Event	27
5.3. Example for Event Handling	27
6. API Functions	29
6.1. SYSTEM API	29
6.1.1. QI_GetEvent	29
6.1.2. QI_Reset	30
6.1.3. QI_Sleep	30
6.1.4. QI_PowerDown	30
6.1.5. QI_PwrKey_Init	31
6.1.6. QI_VoltageInd_Init	31
6.1.7. QI_GetPowerSupply	32
6.1.8. QI_StartWatchdog	32
6.1.9. QI_FeedWatchdog	33
6.1.10. QI_StopWatchdog	33
6.1.11. QI_ReadModuleUID	33
6.1.12. QI_GetCoreVer	34
6.1.13. QI_GetSDKVer	34
6.1.14. QI_GetPowerOnReason	34
6.1.15. QI_GetDeviceCurrentRunState	35
6.1.16. QI_SetSleepMode	36
6.1.17. QI_SetCFUN	36
6.1.18. QI_GetCFUN	37
6.1.19. QI_GetCHINFO	37
6.1.20. QI_StartJammingDet	38
6.1.21. QI_StopJammingDet	38
6.1.22. QI_SetLocInfoLang	39
6.1.23. QI_GetLocation	39
6.1.24. QI_GetLocationByCell	40
6.1.25. QI_GetAddrByCoordinate	41
6.1.26. OCPU_CB_LocInfo	41
6.1.27. QI_IsSIMInserted	42
6.1.28. QI_GSM_GetIMEI	42
6.1.29. QI_SIM_GetIMSI	43
6.1.30. QI_GetOperator	43
6.1.31. QI_RetrieveNodeBInfo	44
6.1.32. QI_ChangeUsbMode	44
6.1.33. QI_GenericURC_Init	44
6.2. Memory API	45
6.2.1. Define memory size	45
6.2.2. QI_GetMemory	45
6.2.3. QI_FreeMemory	46
6.2.4. QI_memMaxCanAllocSize	46
6.2.5. QI_memTotalLeftSize	46
6.3. File system API	47

6.3.1.	QI_FileGetFreeSize	47
6.3.2.	QI_FileGetFreeSizeEx	47
6.3.3.	QI_FileOpenEx.....	48
6.3.4.	QI_FileRead	49
6.3.5.	QI_FileWrite	49
6.3.6.	QI_FileSeek	50
6.3.7.	QI_FileGetFilePosition	50
6.3.8.	QI_FileTruncate	51
6.3.9.	QI_FileFlush.....	51
6.3.10.	QI_FileClose	51
6.3.11.	QI_FileGetSize.....	52
6.3.12.	QI_FileDelete	52
6.3.13.	QI_FileCheck	53
6.3.14.	QI_FileRename	53
6.3.15.	QI_FileCreateDir	54
6.3.16.	QI_FileRemoveDir	54
6.3.17.	QI_FileCheckDir.....	54
6.3.18.	QI_FileFindFirst	55
6.3.19.	QI_FileFindNext	55
6.3.20.	QI_FileFindClose	56
6.3.21.	QI_FileXDelete.....	56
6.3.22.	QI_FileXMove	57
6.3.23.	QI_FileSys_GetSpaceInfo.....	57
6.3.24.	QI_Fs_Format.....	58
6.4.	Periphery API	59
6.4.1.	Multifunctional Pins.....	59
6.4.2.	Pin Names	59
6.4.3.	Working Modes.....	60
6.4.4.	Configure multifunctional pins	62
6.4.5.	Pin functions	63
6.4.6.	QI_pinSubscribe	63
6.4.7.	QI_pinUnSubscribe.....	66
6.4.8.	QI_pinQueryMode.....	67
6.4.9.	QI_pinRead	67
6.4.10.	QI_pinWrite	68
6.4.11.	QI_pinControl	68
6.4.12.	QI_eintRead	69
6.4.13.	QI_eintMask	69
6.4.14.	QI_eintSetPolarity	69
6.4.15.	QI_ReadADC	70
6.4.16.	QI_GetGpioByName	70
6.4.17.	BUS functions	71
6.4.18.	QI_busSubscribe	71
6.4.19.	QI_busUnSubscribe	72

6.4.20.	QI_busWrite	73
6.4.21.	QI_busRead	74
6.4.22.	QI_busQuery	74
6.4.23.	I2C functions	75
6.4.24.	QI_I2C_Init	75
6.4.25.	QI_I2C_Config	75
6.4.26.	QI_I2C_Read	77
6.4.27.	QI_I2C_Write	77
6.5.	AUDIO API	78
6.5.1.	QI_PlayAudio	78
6.5.2.	QI_StopAudio	79
6.5.3.	QI_StartPlayAudioFile	79
6.5.4.	QI_PlayAudioFile_8k	80
6.5.5.	QI_StopAudioFile_8k	81
6.5.6.	QI_StopPlayAudioFile	81
6.5.7.	QI_StartPlayAudioStream	82
6.5.8.	QI_StopPlayAudioStream	83
6.5.9.	QI_VoiceCallChangePath	83
6.5.10.	QI_VoiceCallGetCurrentPath	83
6.5.11.	QI_SetVolume	84
6.5.12.	QI_SetVolume_Ex	84
6.5.13.	QI_GetVolume	85
6.5.14.	QI_SetMicGain	86
6.5.15.	QI_GetMicGain	86
6.5.16.	QI_SetSideToneGain	87
6.5.17.	QI_GetSideToneGain	87
6.5.18.	QI_CtrlEchoCancel	87
6.5.19.	QI_VTS	88
6.5.20.	QI_StartRecord	89
6.5.21.	QI_StartRecord_FullFunc	89
6.5.22.	QI_StopRecord	90
6.5.23.	QI_PlayTone	90
6.5.24.	QI_StopTone	91
6.5.25.	QI_SelectRingTone	91
6.5.26.	QI_WDTMF	91
6.5.27.	QI_EnableDTMF	92
6.5.28.	QI_DisableDTMF	92
6.5.29.	QI_GetDTMFConfig	93
6.5.30.	QI_SetDTMFConfig	93
6.5.31.	QI_SetRingToneEnable	94
6.6.	TIMER API	94
6.6.1.	TIMER STRUCTURE	94
6.6.2.	QI_StartTimer	95
6.6.3.	QI_StopTimer	95

6.6.4.	QI_SecondToTicks	96
6.6.5.	QI_MillisecondToTicks	96
6.6.6.	QI_GetRelativeTime.....	96
6.6.7.	QI_GetRelativeTime_Counter.....	96
6.6.8.	QI_GetLocalTime	97
6.6.9.	QI_SetLocalTime	97
6.6.10.	QI_Mktime	98
6.6.11.	QI_LocalTime2CalendarTime	98
6.6.12.	QI_CalendarTime2LocalTime	99
6.6.13.	QI_StartGPTimer.....	99
6.6.14.	QI_StopGPTimer.....	100
6.6.15.	QI_Alarm_StartUp.....	100
6.7.	FCM API.....	101
6.7.1.	Virtual modem port	101
6.7.2.	QI_OpenModemPort.....	102
6.7.3.	QI_SendToModem	102
6.7.4.	UART Port.....	103
6.7.5.	QI_SendToUart	103
6.7.6.	QI_SetPortRts	104
6.7.7.	QI_SetUartBaudRate	104
6.7.8.	QI_SetPortOwner.....	105
6.7.9.	QI_SetUartDCBConfig.....	105
6.7.10.	QI_SetUartFlowCtrl.....	106
6.7.11.	QI_UartGetBytesAvail.....	107
6.7.12.	QI_UartGetTxRoomLeft	107
6.7.13.	QI_UartGetTxRestBytes	108
6.7.14.	QI_UartConfigEscape	108
6.7.15.	QI_UartClrTxBuffer	109
6.7.16.	QI_UartClrRxBuffer.....	109
6.7.17.	QI_UartSetGenericThreshold	109
6.7.18.	QI_UartGenericClearFEFlag	110
6.7.19.	QI_UartSetVfifoThreshold.....	110
6.7.20.	QI_UartGetVfifoThresholdInfo	111
6.7.21.	QI_UartMaxGetVfifoThresholdInfo	112
6.7.22.	QI_UartRcvData.....	112
6.7.23.	QI_UartQueryDataMode	113
6.7.24.	QI_UartForceSendEscape.....	113
6.8.	TCP/IP API.....	114
6.8.1.	Possible error codes.....	114
6.8.2.	QI_GprsAPNSet.....	115
6.8.3.	QI_GprsAPNGet	116
6.8.4.	QI_GprsNetworkInitialize	116
6.8.5.	QI_GprsNetworkUnInitialize	117
6.8.6.	QI_GprsNetworkActive	118

6.8.7.	QI_GprsNetworkDeactive	118
6.8.8.	QI_GprsNetworkGetState	119
6.8.9.	QI_SocketCreate	120
6.8.10.	QI_SocketClose	120
6.8.11.	QI_SocketConnect.....	121
6.8.12.	QI_SocketSend	121
6.8.13.	QI_SocketRecv	122
6.8.14.	QI_SocketTcpAckNumber.....	123
6.8.15.	QI_SocketSendTo	123
6.8.16.	QI_SocketRecvFrom.....	124
6.8.17.	QI_SocketBind	124
6.8.18.	QI_SocketListen.....	125
6.8.19.	QI_SocketAccept	125
6.8.20.	QI_GetHostIpbyName.....	126
6.8.21.	QI_GetLocalIpAddress.....	127
6.8.22.	QI_GetDnsServerAddress	128
6.8.23.	QI_SetDnsServerAddress	128
6.8.24.	QI_SocketCheckIp	129
6.8.25.	QI_SocketSetSendBufferSize.....	129
6.8.26.	QI_SocketSetRecvBufferSize.....	130
6.8.27.	QI_SocketHtonl.....	131
6.8.28.	QI_SocketHtons.....	131
6.8.29.	QI_SwitchHeartBeat	131
6.8.30.	QI_CallBack_GprsAPNSet	132
6.8.31.	QI_CallBack_GprsAPNGet.....	132
6.8.32.	QI_CallBack_network_activated	133
6.8.33.	QI_CallBack_network_deactivated.....	133
6.8.34.	QI_Callback_socket_connect	134
6.8.35.	QI_Callback_socket_close	134
6.8.36.	QI_Callback_socket_accept	135
6.8.37.	QI_Callback_socket_read.....	135
6.8.38.	QI_Callback_socket_write	136
6.9.	CALL API.....	136
6.9.1.	QI_Call_Initialize	136
6.9.2.	QI_GetCallCntByType.....	137
6.9.3.	QI_Call_Dial	137
6.9.4.	QI_Call_Answer	138
6.9.5.	QI_Call_Hangup	138
6.10.	SMS API.....	139
6.10.1.	QI_SMSInitialize	139
6.10.2.	QI_SetSMSStorage.....	139
6.10.3.	QI_GetSMSStorageInfo	140
6.10.4.	QI_SetInfoCentreNum	140
6.10.5.	QI_GetInfoCentreNum	141

6.10.6.	QI_SetNewSMSDirectToTE	141
6.10.7.	QI_SetSMSFormat	142
6.10.8.	QI_ReadSMS	142
6.10.9.	QI_DeleteSMS	143
6.10.10.	QI_ListSMS	143
6.10.11.	QI_GetUnReadSMS	144
6.10.12.	QI_SendPDUSMS	144
6.10.13.	QI_SetTextSMSPara	145
6.10.14.	QI_SendTextSMS	146
6.10.15.	QI_SMS_DCS_ToCSCS	146
6.10.16.	CB_NewSMS	147
6.10.17.	CB_SendSMS	147
6.10.18.	CB_ReadPDUSMS	148
6.10.19.	CB_DeleteSMS	148
6.10.20.	CB_NewFlashPDUSMS	149
6.10.21.	CB_PDUStatusReport	149
6.10.22.	CB_ReadTextSMS	150
6.10.23.	CB_NewFlashTextSMS	150
6.10.24.	CB_TextStatusReport	151
6.11.	TTS API	152
6.11.1.	QI_TTS_Initialize	152
6.11.2.	QI_TTS_Play	152
6.11.3.	QI_TTS_PlayInCall	153
6.11.4.	QI_TTS_Stop	153
6.11.5.	QI_TTS_Query	154
6.12.	MULTITASK API	154
6.12.1.	Main Task	154
6.12.2.	Subtasks	155
6.12.3.	Possible Error Code	156
6.12.4.	QI_osSendEvent	156
6.12.5.	QI_osCreateMutex	157
6.12.6.	QI_osTakeMutex	157
6.12.7.	QI_osGiveMutex	158
6.12.8.	QI_osCreateSemaphore	158
6.12.9.	QI_osTakeSemaphore	158
6.12.10.	QI_osGiveSemaphore	159
6.12.11.	QI_SetLastErrorCode	159
6.12.12.	QI_GetLastErrorCode	160
6.12.13.	QI_osGetCurrentTaskPriority	160
6.12.14.	QI_osGetCurrentTaskRemainStackSize	160
6.12.15.	QI_osChangeTaskPriority	161
6.13.	OpenCPU FOTA API	161
6.13.1.	QI_Fota_App_Init	161
6.13.2.	QI_Fota_App_Write_Data	162

6.13.3. QI_Fota_App_Finish	163
6.13.4. QI_Fota_Update.....	163
6.14. DEBUG API.....	163
6.14.1. Debug Mode	163
6.14.2. QI_SetDebugMode	164
6.14.3. QI_DebugTrace.....	164
6.15. STANDARD LIBRARY API.....	165
7. System Configuration.....	167
7.1.1. Configure 'Customer_user_qlconfig'	167
7.1.2. Example for Headset	169
8. Appendix A - Error Codes Definition	172
9. Appendix B Reference.....	177

Table Index

TABLE 1: MULTIFUNCTION PINS AND WORKING MODES FOR M12 (R2.0).....	60
TABLE 2: RELATED DOCUMENTS	177
TABLE 3: GLOSSARY	177
TABLE 4: ABBREVIATIONS	177

Quectel
Confidential

1 Introduction

OpenCPU is a solution designed to facilitate the development of cost-effective wireless machine-to-machine applications for Quectel modules. Using standard C language, OpenCPU enables you to create innovative applications and embed them directly into Quectel GSM/GPRS modules. It is easy to migrate the application software to different MCU platform as well.

Quectel
Confidential

2 OpenCPU SDK

OpenCPU SDK provides some resources as follows for developers:

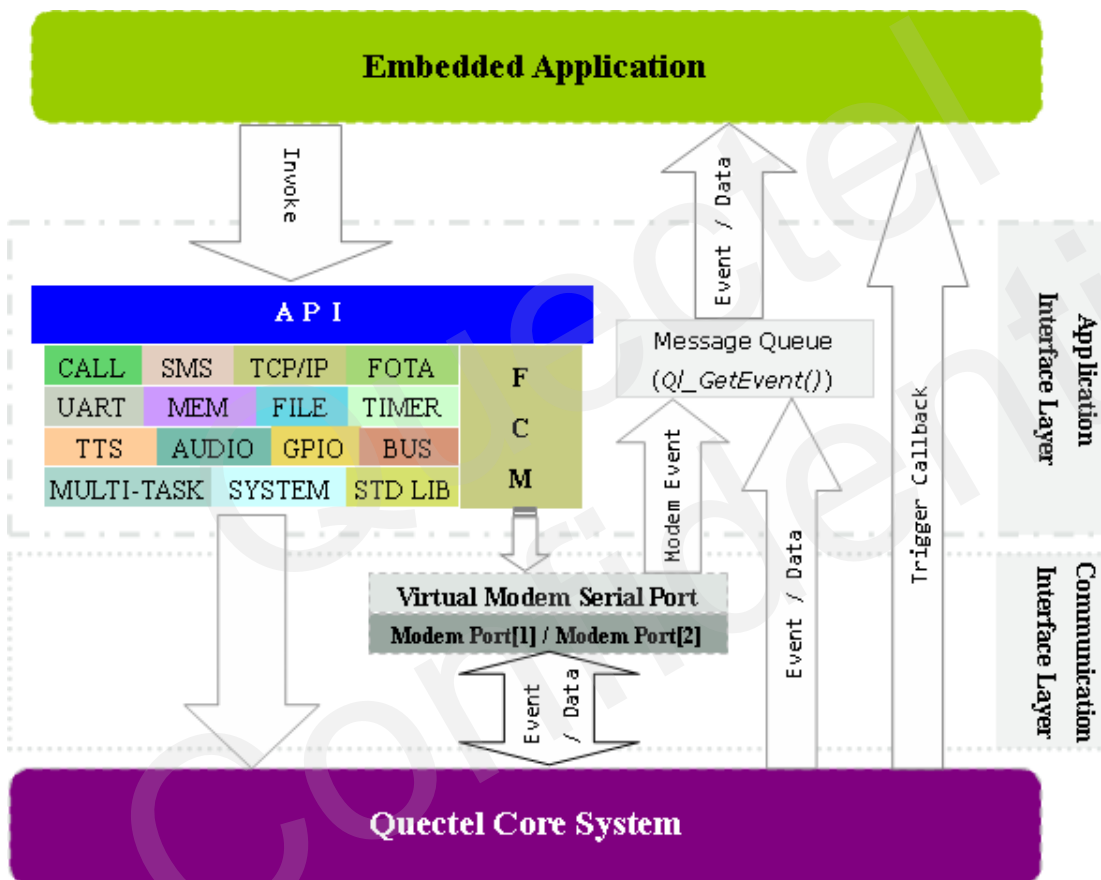
- Compile environment.
- A set of header files defines all interface functions and OpenCPU parameters.
- Development guide and other related documents.
- Source code for examples.
- Quectel Core System firmware, binary file.
- Download tool for image bin.

Quectel
Confidential

3 OpenCPU Software Architecture

3.1. Software architecture

The following figure shows the fundamental principle of OpenCPU software architecture.



When transmitting information from the Core System to Embedded Application, the Ql_GetEvent function catches EVENT then notifies Embedded Application of a parameter of event type.

When need to use the resources of Core System, developers may simply call API functions for the appropriate purposes. Some operations have to dispose the feedback data by Ql_GetEvent.

OpenCPU software mechanism appropriately prevents the developers from directly accessing core variables and stack spaces, which insures the developers can safely access to the resources of the Core System, and at the same time, reduces the workload of developing application.

3.2. Application code frame

3.2.1. Least Application Code

The following code is the least codes that an Embedded Application requires.

```
/* main function */
QIEventBuffer qIEventBuffer;
bool keepGoing = TRUE;
void ql_entry(void)
{
    while (keepGoing)
    {
        QI_GetEvent(&qIEventBuffer); //get event from Quectel Core system
        switch(qIEventBuffer.eventTyp)
        {
            //To do: add your code for parsing EVENTS.
            default:
                break;
        }
    }
}
```

ql_entry is the main entrance of Embedded Application.

QI_GetEvent is the important system interface that the Embedded Application receives events (or messages) from Core System.

3.2.2. Application entry

The *ql_entry()* is the entry of Embedded Application, just like the *main()* in C application.

3.2.3. Application termination

In the entry function of the Embedded Application, the **While** statement keeps the application running. If developers need to terminate the application (actually a task, the main task or a subtask) programmatically, a simple setting, as below, will make it.

```
keepGoing = FALSE.
```

3.3. Memory resources

OpenCPU runs within Real-Time Kernel Task. Developers have to pre-define the size of customer application call-stack. The stack size of the main task is defined using `QL_TASK_STACK_SIZE` in `ql_customer_config.c`. And please refer to the `QIMutitask` structure to know the definition of the stack size for subtask.

Quectel Core System and Embedded Application manage their own RAM regions respectively. Accessing from one of these programs to the other's RAM region is prohibited, because the operation may cause fatal error.

Quectel
Confidential

4 Basic data types

File *ql_typ.h* declares all the basic data types used in OpenCPU.

```
typedef unsigned char    bool;
typedef      char       ascii;
typedef unsigned char    u8;
typedef signed   char    s8;
typedef unsigned short   u16;
typedef      short      s16;
typedef unsigned int     u32;
typedef      int        s32;
typedef unsigned long long u64;
typedef unsigned int     ticks;
```

5 Event

OpenCPU provides an event-driven communication mechanism for the communication from Core System to Embedded Application. Embedded Application can accept the EVENTS from Core System by calling *QI_GetEvent(&qIEventBuffer)*.

EVENT data structure is wrapped in the structure *QIEventBuffer*, and EVENT data consists of two parts:

- EVENT type
The type of the received EVENT. It can also be used to work out the associated structure of the EVENT data part.
- EVENT data
Specific EVENT body.

```
typedef struct QIEventBufferTag
{
    QIEventType  eventType;
    QIEventData  eventData;
}QIEventBuffer;
```

5.1. Event type

All EVENTS are defined in “*QIEventType*”.

```
typedef enum QIEventTypeTag
{
    EVENT_NULL = 0,
    EVENT_INTR,
    EVENT_KEY,
    EVENT_UARTREADY,
    EVENT_UARTDATA,
    EVENT_MODEMDATA,
    EVENT_TIMER,
    EVENT_SERIALSTATUS,
    EVENT_MSG,
    EVENT_POWERKEY,
    EVENT_HEADSET,
    EVENT_UARTESCAPE,
    EVENT_UARTFE,          /* only support Generic uart, uart2 */
}
```

```
EVENT_MEDIA_FINISH,  
EVENT_MAX = 0xFF  
}QIEventType;
```

5.1.1. EVENT_INTR

The event is triggered when Embedded Application receives an interrupt signal from the Core System. Interrupt signals are generated by the interrupt pins which were subscribed by calling “*QI_pinSubscribe*” function. To get extended information about the interrupt pins, please refer to the PERIPHERY API chapter.

5.1.2. EVENT_KEY

The event is triggered when a key status has changed, and the key EVENT data contains the information of a key press or release. Under the reset condition of OpenCPU, there is a predefined 5*5 keypad matrix. When the status of any one of these keys (suppose that above mentioned pins have not been configured as other uses) is changed, Embedded Application will receive the EVENT_KEY event.

5.1.3. EVENT_UARTDATA

The event is triggered when Core System has received data from UART port. For example, when GPS data is fed in through UART port, embedded Application will receive the EVENT_KEY event.

5.1.4. EVENT_UARTREADY

This event is designed for the function “*QI_SendToUart*”. When the number of bytes sent actually is less than the number of bytes to send, Embedded Application should stop sending data, till receive the EVENT_UARTREADY event. The data which has not been sent out last time should be resent.

5.1.5. EVENT_MODMEDATA

The event is triggered when data from Core System is sent to Embedded Application. For instance when Embedded Application receives an AT command's response, or PPP/CSD data from protocol stack.

5.1.6. EVENT_TIMER

The event is triggered when a timer expires. Timer can be stopped before it expires, thus stopped timer will no longer trigger *EVENT_TIMER*. For more details about timer, please refer to TIMER API section.

5.1.7. EVENT_SERIALSTATUS

The event is triggered when the status of CTS or DCD of serial port changes.

5.1.8. EVENT_MSG

The event transports messages between tasks.

5.1.9. EVENT_POWERKEY

The event is triggered when power key is pressed.

NOTE

The EVENT_POWERKEY event can only be received by main task.

5.1.10. EVENT_HEADSET

When earphone is plugged in device or plugged out from device, Embedded Application will receive this event.

NOTE

The EVENT_HEADSET event can only be received by main task.

5.1.11. EVENT_UARTESCAPE

UART port has the Escape function. Embedded Application can configure this function by calling “*QI_UartConfigEscape()*”. Under the condition that the Escape function is enabled, if external device inputs three consecutive characters ‘+’ through UART port, Embedded Application will receive the EVENT_UARTESCAPE event.

The default control character for Escape function is ‘+’. However, developer can set it to other character by calling *QI_UartConfigEscape()*.

NOTE

*The EVENT_UARTESCAPE event can be received only by the owner task. Please see *QI_SetPortOwner()* to get extended information about owner task.*

5.1.12. EVENT_UARTFE

When error occurs during transferring data through UART port, such as frame error, parity error, Embedded Application will receive the EVENT_UARTFE event.

NOTES

1. The EVENT_UARTFE event just works for UART2.
2. Embedded Application should call "QI_UartGenericClearFEFlag()" to clear some flags after receiving the EVENT_UARTFE event. Or, no EVENT_UARTFE event will be broadcast when next error occurs.
3. The EVENT_UARTFE event can only be received by the owner task. Please see "QI_SetPortOwner()" to get extended information about owner task.

5.1.13. EVENT_MEDIA_FINISH

After finishing playing audio files, Embedded Application will receive this event.

5.2. Event Data Structure

5.2.1. Event Data Union

Each of the EVENT types is one to one mapping with the corresponding EVENT data.

```
typedef union QIEventDataTag
{
    Timer_Event      timer_evt;
    Key_Event        key_evt;
    PortData_Event   uartdata_evt;
    PortData_Event   modemdata_evt;
    Intr_Event       intr_evt;
    PortStatus_Event portstatus_evt;
    Msg_Event        msg_evt;
    Powerkey_Event   powerkey_evt;
    Headset_Event    headset_evt;
    UartEscape_Event uartescape_evt;
    UartFE_Event     uartfe_evt; /*only support Generic uart, uart2*/
}QIEventData;
```

"QIEventData" is a union type, and each of the data types has its own structure. Below sections will describe these structures in detail respectively.

5.2.2. Timer_Event

```
typedef struct Timer_EventTag
{
    u32  timer_id;  // read-only field
    u32  interval;  // it is measured in tick.
}Timer_Event;
```

timer_id:

Timer ID. It's read-only.

interval: The time, in milliseconds, between raisings of the elapsed event.

5.2.3. Key_Event

```
typedef struct Key_EventTag
{
    u16  key_val;
    bool  isPressed;
}Key_Event;
```

key_val:

The key value.

isPressed:

Whether the key is pressed.

5.2.4. PortData_Event

OpenCPU provides two virtual modem serial ports and three physical UART ports. Embedded Application can use virtual modem serial ports to send AT commands to Core System or receive AT response from Core System. Embedded Application can communicate with external device through physical UART ports. All virtual modem serial ports and physical UART ports are enumerated in the structure of "QIPort".

```
typedef enum QIPortTag
{
    ql_md_port1,
    ql_md_port2,
    ql_uart_port1,
    ql_uart_port2,
    ql_uart_port3
}QIPort;

typedef struct PortData_EventTag
{
    s16  len;
```

```
s8    data[EVENT_MAX_DATA];  
QIPort port;  
QIDataType type;  
}PortData_Event;
```

len:

The length of the data being transported.

data:

The data buffer, which maximum size is 1024 bytes.

port:

Serial port that data comes from.

type:

The type of the data, one of the *QIDataType* types. The *QIDataType* types are defined as below:

```
typedef enum QIDataTypeTAG  
{  
    DATA_AT = 0,  
    DATA_PPP,  
    DATA_CSD,  
    DATA_MAX  
}QIDataType;
```

DATA_AT:

AT command data type.

DATA_PPP:

Point-to-Point Protocol data type.

DATA_CSD:

Circuit-Switched data type.

5.2.5. Intr_Event

```
typedef struct Intr_EventTag  
{  
    QIPinName    pinName;  
    bool         pinState;  
}Intr_Event;
```

pinName:

Name of the pin.

pinState:

State of the pin, high level or low level:

5.2.6. PortStatus_Event

```
typedef enum QILineTypeTag
{
    DCD,
    CTS
}QILineType;
typedef struct PortStatus_EventTag{
    u8 val;
    QILineType type;
    QIPort port;
}PortStatus_Event;
```

val:

Serial port data

type:

Line type of serial port.

port:

Serial port.

5.2.7. Msg_Event

```
typedef struct Msg_EventTag
{
    s32      src_taskid;
    u32      data1;
    u32      data2;
}Msg_Event;
```

src_taskid:

Id of the task, which sends message event.

data1:

User data.

data2:

User data.

5.2.8. Powerkey_Event

This event takes effect only when the function of Controlling Switch (i.e. “Customer_user_qlconfig.powerautoon” and “Customer_user_qlconfig.powerautooff” are set to FALSE) is turned on.

```
typedef enum QIPowerKeyTypeTag
{
    POWERKEY_ON,
```

```

POWERKEY_OFF
}QIPowerKeyType;
typedef struct Powerkey_EventTag
{
    QIPowerKeyType powerkey;
    bool            isPressed;
}Powerkey_Event;

```

powerkey:

If POWERKEY_ON, this event indicates switching on device;

If POWERKEY_OFF, this event indicates switching off device.

isPressed:

When powerkey is POWERKEY_OFF, TRUE indicates that the PWRKEY key is pressed, and FALSE indicates that the PWRKEY key is released.

5.2.9. Headset_Event

This event takes effect only after the earphone detecting function is configured to ON.

```

typedef enum QIHeadsetTypeTag
{
    HEADSET_PLUGOUT,    // Plug out earphone
    HEADSET_PLUGIN,     // Plug in earphone
    HEADSET_KEYPRESS,   // Press down the button on earphone
    HEADSET_KEYRELEASE, // Release the button on earphone
    HEADSET_ADC         // ADC
}QIHeadsetType;

```

```

typedef struct Headset_EventTag
{
    QIHeadsetType    headsettype;
    u32             data1;
}Headset_Event;

```

data1:

ADC value is available only when headsettype is set to HEADSET_ADC.

5.2.10. UartEscape_Event

UART port has the Escape function. Embedded Application can configure this function by calling *QI_UartConfigEscape()*. Under the condition that the Escape function is enabled, if external device consecutively inputs three characters of '+' through UART port, Embedded Application will receive the EVENT_UARTESCAPE event.

```
typedef struct UartEscape_EventTag
{
    QIPort    port;
}UartEscape_Event;
```

port:

UART port, in which Escape event occurs.

5.2.11. UartFE_Event

```
typedef struct UartFE_EventTag
{
    QIPort    port;
    u32       data1;
    u32       data2;
}UartFE_Event;
```

port:

UART port, in which error occurs.

data1:

Reserved.

data2:

Reserved.

5.3. Example for Event Handling

The following code skeleton demonstrates how events are captured and dealt with in Embedded Applications.

```
QIEventBuffer  qlEventBuffer;    // Please keep this variable a global variable
                                   // because of its big size.

void ql_entry() //task entrance
{
    // ...
    while(TRUE)
    {
        QI_GetEvent(&qlEventBuffer);
        switch(qlEventBuffer.eventType)
        {
            case EVENT_UARTDATA:
                break;
            case EVENT_MODEMDATA:
                break;
            case EVENT_KEY:
```

```
        break;
    case EVENT_TIMER:
        if(qlEventBuffer.sig_p.timer_evt.timer_id == timerDemo.timerId)
        {
            //deal with the timerDemo's event
            Ql_SendToUart(PortNo, "the timer expired!", Len);
        }
        break;
    //...
    default:
        break;
    }
}
}
```

Here, if the expired timer's ID equals to "*timerDemo.timerId*", Embedded Application sends "the timer expired!" to the specified uart port.

6. API Functions

6.1. SYSTEM API

The “*ql_interface.h*” file declares system related to APIs. These functions are essential to any customer’s applications. Make sure to include the header file.

6.1.1. QI_GetEvent

This function gets system EVENTS from the Core System. When there is no event in customer task’s event queue, the task is in the waiting state.

- **Prototype**

```
void QI_GetEvent(QIEventBuffer *event_p);
```

- **Parameters**

event_p:

A pointer to a particular “*QIEventBuffer*”, refer to chapter EVENT for “*QIEventBuffer*” structure.

The following code is an example about how to create a signal buffer, and listen to incoming signals by using *QI_GetEvent* function.

```
QIEventBuffer qlEventBuffer;    // Please keep this variable a global variable
                                // because of its big size.

void QI_entry(void)
{
    while (TRUE)
    {
        QI_GetEvent(&qlEventBuffer); //get EVENT from Quectel Core system
        switch(qlEventBuffer.eventType)
        {
            .....
        }
    }
}
```

- **Return value**

None.

6.1.2. QI_Reset

This function resets the system. And this function can be the alias of different functions with different parameters.

- **Prototype**

```
void QI_Reset(u8 resetype);
```

- **Parameters**

resetype: must be 0.

- **Return value**

None.

6.1.3. QI_Sleep

This function suspends the execution of the current task until the time-out interval elapses.

- **Prototype**

```
void QI_Sleep(u32 msec);
```

- **Parameters**

msec:

The time interval for which execution is to be suspended is in milliseconds.

- **Return value**

None.

6.1.4. QI_PowerDown

This function powers off the module.

- **Prototype**

```
void QI_PowerDown(u8 powertype);
```

- **Parameters**

powertype:

Action types of this function.

0 = Urgently power off

1 = Normal power off

- **Return value**

None.

6.1.5. QI_PwrKey_Init

This function registers the callback for PWRKEY indication. The callback function will be triggered when the power KEY pressed down or released. The *Customer_user_qlconfig.powerautooff* should be set to FALSE before calling this function.

- **Prototype**

```
s32 QI_PwrKey_Init(OCPU_CB_PWRKEY_IND pwrKey_callback);
```

- **Parameters**

pwrKey_callback:

callback function for PWRKEY indication.

- **Return value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.1.6. QI_VoltageInd_Init

This function registers the callback for voltage indication. The callback function will be triggered when the power supply to module exceeds the limitation (3.4v-4.5v, and not include the boundary)

- **Prototype**

```
s32 QI_VoltageInd_Init(OCPU_CB_VOL_IND volInd_callback);
```

- **Parameters**

volInd_callback:

callback function for voltage indication.

- **Return value**

QL_RET_OK indicates this function is executed successfully.

QL_RET_ERR_INVALID_PARAMETER, indicates the wrong input parameter.

6.1.7. QI_GetPowerSupply

This function queries the battery balance, voltage and the charging status.

- **Prototype**

```
s32 QI_GetPowerSupply(u8* charge_stat, u32* capacity, u32* voltage );
```

- **Parameters**

charge_stat:

charging state, 0=uncharged status, 1=charging, 2=charging completes

capacity:

battery balance, a percent.

voltage:

battery voltage, unit in mV

- **Return value**

QL_RET_OK, indicates this function successes.

QL_RET_ERR_INVALID_PARAMETER, indicates the wrong input parameter.

QL_RET_ERR_GENERAL_FAILURE, fails to get the info of power supply.

6.1.8. QI_StartWatchdog

This function starts watch-dog service. If calling QI_FeedWatchdog exceeds $90 \times 200 \times 10s = 18000s = 3$ minutes, the module will automatically power down.

- **Prototype**

```
bool QI_StartWatchdog(u16 tick10ms, u32 overfeedcount, u16 resettype);
```

tick10ms:

A Counter which counts by step 1 every 10ms.

overfeedcount:

Counter threshold.

The module will restart when the counter's value is over this threshold.

resettype:

0 = WDT_RESET, 1 = powerdown, 2 = ASSERT.

- **Return Value**

TRUE indicates success in starting watch dog service.

FALSE indicates failure.

6.1.9. QI_FeedWatchdog

Calling this function will reset watch-dog. If calling QI_FeedWatchdog exceeds $90 \times (200 \times 10\text{ms}) = 180000\text{ms} = 3$ minutes, the module will automatically power down.

- **Prototype**

```
void QI_FeedWatchdog(void);
```

- **Parameters**

None.

- **Return value**

None.

6.1.10. QI_StopWatchdog

Stop the watch-dog, which was started previously.

- **Prototype**

```
void QI_StopWatchdog(void);
```

- **Parameters**

None.

- **Return value**

None.

6.1.11. QI_ReadModuleUID

This function gets the module UID.

- **Prototype**

```
s32 QI_ReadModuleUID(u16* uid_buf);
```

- **Parameters**

uid_buf:

Pointer to the buffer to store the UID. Need 20 bytes length of buffer.

- **Return value**

If the uid_buf is null, this function will return QL_RET_ERR_INVALID_PARAMETER.
If this function read the UID successfully, the length of UID will be returned.

6.1.12. QI_GetCoreVer

Get the version ID of the core.

- **Prototype**

```
s32 QI_GetCoreVer(u8* ptr_ver, u16 len);
```

- **Parameters**

ptr_ver:

A pointer to the buffer which receives the version ID of the core.

len:

The length of the buffer.

- **Return Value**

The return value is the length of version ID of the core if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.1.13. QI_GetSDKVer

Get the version ID of the SDK.

- **Prototype**

```
s32 QI_GetSDKVer(u8* ptr_ver, u16 len);
```

- **Parameters**

ptr_ver:

A pointer to the buffer that receives the version ID of the SDK.

Len:

The length of the buffer.

- **Return Value**

The return value is the length of version ID if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.1.14. QI_GetPowerOnReason

Get Power on Reason.

- **Prototype**

```
void QI_GetPowerOnReason(u8* cause);
```

- **Parameters**

cause:

A pointer to the buffer that receives the Power on Reason.

```
typedef enum{  
    PWRKEYPWRON = 0,  
    CHRPWRON= 1,  
    RTCPWRON = 2,  
    CHRPWROFF = 3,  
    WDTRESET = 4, /*NORMAL*/  
    ABNRESET = 5, /*ABNORMAL RESET*/  
    USBPWRON = 6,  
    USBPWRON_WDT = 7,  
    PRECHRPWRON = 8,  
    HWSYSRST = 9,  
    UNKNOWN_PWRON = 0xF9  
}power_on_enum;
```

- **Return Value**

None.

6.1.15. QI_GetDeviceCurrentRunState

This function retrieves the current run-state, including SIM card state, network registration state, GPRS network registration state, signal strength and bit error rate.

- **Prototype**

```
void QI_GetDeviceCurrentRunState(s32 *simcard, s32 *creg, s32 *cgreg, u8 *rssi, u8 *ber);
```

- **Parameters**

simcard:

[out] SIM card state, a value of “*QI_SIM_State*”.

creg:

[out] Network registration state, a value of “*QI_Reg_State*”.

cgreg:

[out] GPRS Network registration state, a value of “*QI_Reg_State*”.

rssi:

[out] Signal strength, unit in dBm.

ber:

[out] Bit error rate.

- **Return Value**

None.

6.1.16. QI_SetSleepMode

This function set enable or disable of sleep mode.

- **Prototype**

```
s32 QI_SetSleepMode(u8 mod);
```

- **Parameters**

mod:

one value of QL_SleepMode,. 0: disable, 1:enable.

```
typedef enum
```

```
{  
    SM_Disable = 0,  
    SM_Enable  = 1  
}QL_SleepMode;
```

- **Return value**

QL_RET_OK , indicates this function is executed succeefully.

QL_RET_ERR_INVALID_PARAMETER, indicates the wrong input parameter.

6.1.17. QI_SetCFUN

This function switchs CFUN state to 0/1/4.

- **Prototype**

```
s32 QI_SetCFUN(u8 cfun_type, OCPU_CB_SGL_REP cfun_callback);
```

- **Parameters**

cfun_type:

cfun_type: 0/1/4

cfun_callback:

callback function for cfun switch action.

- **Return value**

QL_RET_OK, indicates this function successes.

QL_RET_ERR_INVALID_PARAMETER, indicates the wrong input parameter.

QL_RET_ERR_GENERAL_FAILURE, fails to switch cfun state.

6.1.18. QI_GetCFUN

This function gets CFUN state.

- **Prototype**

```
s32 QI_GetCFUN(void);
```

- **Parameters**

None

- **Return Value**

This function returns the CFUN state value (0 /1 /4) if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.1.19. QI_GetCHINFO

This function gets Channel Information.

- **Prototype**

```
s32 QI_GetCHINFO(QICHINFO *CHINFO);
```

- **Parameters**

CHINFO:

A pointer to the QICHINFO object.

```
typedef struct QICHINFOTag
{
    u8  rr_state; //RR state:
                // 0   null state
                // 1   inactive state
                // 2   cell select state
                // 3   idle state
                // 4   access state
                // 5   packet transfer state
                // 6   dedicated state
                // 7   cell reselect state
}
```

```

u16 arfcn;      //Absolute Radio Frequency Channel Number
s8  dbm;        //Receiving level in dBm
u8  ch_type;    //Channel type
u8  ta;         //Timing Advance, the value 255 is unavailable
u8  rxq_sub;    //Receiving quality(sub), range is 0-7, the same as the parameter of QENG
u8  rxq_full;   //Receiving quality(full), range is 0-7, the same as the parameter of QENG
}QICHINFO;

```

- **Return Value**

QL_RET_OK indicates this function succeeds.

QL_RET_ERR_PARAM, indicates the input parameter is NULL.

6.1.20. QI_StartJammingDet

This function starts Jamming Detection.

- **Prototype**

```
s32 QI_StartJammingDet (QI_Callback_JDC callback_jdc);
```

- **Parameters**

callback_jdc:

A pointer to callback result.

- **Return value**

QL_RET_OK indicates this function succeeds.

QL_RET_ERR_PARAM indicates the input parameter is NULL.

6.1.21. QI_StopJammingDet

This function stops Jamming Detection.

- **Prototype**

```
s32 QI_StopJammingDet (void);
```

- **Parameters**

None.

- **Return value**

Always return QL_RET_OK.

6.1.22. QI_SetLocInfoLang

This function sets language and charset for the responded address information. The default language is English and UTF8 code. If needed, call this function to change language or charset for the responded address information.

- **Prototype**

```
s32 QI_SetLocInfoLang(QILanguage lang, QICharSet charset);
```

- **Parameters**

lang:

language, one value of QILanguage.

```
typedef enum{  
    LANG_ENG = 0,  
    LANG_CHZ = 1,  
    LANG_END  
}QILanguage;
```

charset:

character set, one value of QICharSet.

```
typedef enum{  
    CHARSET_UTF8 = 0,  
    CHARSET_GB2132,  
    CHARSET_GBK,  
    CHARSET_END  
}QICharSet;
```

- **Return Value**

QL_RET_OK, indicates this function successes.

QL_RET_ERR_INVALID_PARAMETER, indicates the wrong input parameter.

6.1.23. QI_GetLocation

This function gets location information of module. The location information will be provided via callback function.

- **Prototype**

```
s32 QI_GetLocation(QIGsmOpType op_type, OCPU_CB_LocInfo cb_loc);
```

- **Parameters**

op_type:

operation type, one value of QIGsmOpType. It decides the response content via callback.cb

```
typedef enum{
    REQ_GSM_NONE = 0,
    REQ_GSM_TIME_FMT1, // Date and UTC time, format ex.: 2012/08/07,02:10:17
    REQ_GSM_TIME_FMT2, // Week, date and UTC time, format ex.: Tue, 07 Aug 2012 02:04:24
    GMT
    REQ_GSM_LOC_TIME, // Location, date and UTC time, format ex.:
    121.392285,31.164563,2012/08/07,02:05:50
    REQ_GSM_LOC_ADDR, // Location, date, UTC time and address, format ex.:
    121.392285,31.164563,2012/08/07,02:08:37,{"country":"China","country_code":"CN","region":"Shanghai",
    "city":"Shanghai","street":"Tianlin Rd","street_number":""}
    REQ_GSM_MULTICELL_NOADDR, // Same to 'REQ_GSM_LOC_TIME', but it is generated by
    information of cells
    REQ_GSM_MULTICELL_ADDR, // Same to 'REQ_GSM_LOC_ADDR', but it is generated by
    information of cells
    REQ_GSM_LOC_BYCOORD, // Get address by coordinate
    REQ_GSM_END
}QIGsmOpType;
```

cb_loc:

A pointer to callback function "OCPU_CB_LocInfo".

- **Return value**

QL_RET_OK, indicates this function successes.

QL_RET_ERR_INVALID_PARAMETER, indicates the wrong input parameter.

QL_RET_ERR_INVALID_OP, indicates module is in calling.

QL_RET_ERR_SYS_NOT_READY, indicates the GPRS network is not ready/available.

6.1.24. QI_GetLocationByCell

This function gets location information of module by the input cell information. The location information will be provided via callback function.

- **Prototype**

```
s32 QI_GetLocationByCell(QILocParam param, OCPU_CB_LocInfo cb_loc);
```

- **Parameters**

param:

A structure is variable for QILocParam.

```
typedef struct
{
    QIGsmOpType op_type;
```

```
s32 wait_time; // in second, 60s by default
QIGsmCell cell;
} QILocParam;
```

cb_loc:

A pointer to callback function "OCPU_CB_LocInfo"

- **Return value**

QL_RET_OK, indicates this function successes.

QL_RET_ERR_INVALID_PARAMETER, indicates the wrong input parameter.

QL_RET_ERR_INVALID_OP, indicates module is in calling.

QL_RET_ERR_SYS_NOT_READY, indicates the GPRS network is not ready/available.

6.1.25. QI_GetAddrByCoordinate

This function gets address information of module by the input coordinate information. The location information will be provided via callback function.

- **Prototype**

```
s32 QI_GetAddrByCoordinate(QICoordinate xy, OCPU_CB_LocInfo cb_addr);
```

- **Parameters**

xy:

a structure variable of QICoordinate.

```
typedef struct{
    double longitude;
    double latitude;
}QICoordinate;
```

cb_addr:

A pointer to callback function "OCPU_CB_LocInfo"

- **Return Value**

QL_RET_OK, indicates this function successes.

QL_RET_ERR_INVALID_PARAMETER, indicates the wrong input parameter.

QL_RET_ERR_INVALID_OP, indicates module is in calling.

QL_RET_ERR_SYS_NOT_READY, indicates the GPRS network is not ready/available.

6.1.26. OCPU_CB_LocInfo

This is a callback function. The location information will be provided via this callback function.

- **Prototype**

```
void (*OCPU_CB_LocInfo)(s32 result_code, QIModuleLocInfo* loc_info);
```

- **Parameters**

result_code:

GSM location response error code, 0 Success, 1 Failure.

loc_info:

A buffer to get the module's location. QIModuleLocInfo is defined as below:

```
typedef struct
{
    s32 longitude;    // unit in (degree*1000000)
    s32 latitude;    // unit in (degree*1000000)
    s32 accuracy;    // unit in metre
    u8  dt[30];      // date, and UTC time, format: yyyy/mm/dd, hh/mm/ss
    u8  addr[200];   // address information
    s32 reserved;    // reserved field
}QIModuleLocInfo ;
```

- **Return value**

None.

6.1.27. QI_IsSIMInserted

This function detects SIM Card is inserted or not.

- **Prototype**

```
bool QI_IsSIMInserted(void);
```

- **Parameters**

None.

- **Return value**

1 indicates SIM Card is inserted and 0 is not.

6.1.28. QI_GSM_GetIMEI

This function gets the IMEI (International Mobile Equipment Identity).

- **Prototype**

```
s32 QI_GSM_GetIMEI(u8* ptr_imei, u16 len);
```

- **Parameters**

ptr_imei:

A pointer to the buffer that receives the IMEI.

len:

The length of the buffer.

- **Return value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.1.29. QI_SIM_GetIMSI

This function gets the IMSI (International Mobile Subscriber Identity) of SIM Card.

- **Prototype**

```
s32 QI_SIM_GetIMSI(u8* ptr_imsi, u16 len);
```

- **Parameters**

ptr_imsi:

A pointer to the buffer that receives the IMSI.

len:

The length of the buffer.

- **Return value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.1.30. QI_GetOperator

This function gets the current operator name.

- **Prototype**

```
s32 QI_GetOperator(u8* opr_str, u8 len);
```

- **Parameters**

opr_str:

A pointer to the buffer that receives the operator names.

len:

The length of the buffer.

- **Return value**

The return value is `QL_RET_OK` if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.1.31. QI_RetrieveNodeBInfo

This function retrieves base station info.

- **Prototype**

```
s32 QI_RetrieveNodeBInfo(OCPU_CB_QENG callback_bsInfo);
```

- **Parameters**

callback_bsInfo:

A pointer to callback function.

- **Return value**

The return value is `QL_RET_OK` if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.1.32. QI_ChangeUsbMode

This function changes the usb working mode (as COM or MS). New working mode takes effect only after next rebooting module. To reboot module, you can call `QI_Reset(0)`.

- **Prototype**

```
s32 QI_ChangeUsbMode(QIUsbMode usb_mod);
```

- **Parameters**

usb_mod:

[in] one value of 'QIUsbMode'. if the value is `USB_NORMAL`, the setting for `QICustomerConfig.usb_mode` will be adopted.

- **Return value**

`QL_RET_OK`, success

`QL_RET_ERR_PARAM`, invalid parameter

6.1.33. QI_GenericURC_Init

Register the callback for some generic URC, such as 'Call Ready', 'SIM NOT INSERTED'. The callback function will be triggered when the generic URC related event happens.

- **Prototype**

```
s32 Ql_GenericURC_Init(OCPU_CB_GENERIC_URC urc_callback);
```

- **Parameters**

urc_callback:

Callback function for generic URC. Please do not put too much tasks in this callback.

- **Return value**

QL_RET_OK indicates this function succeeds.

QL_RET_ERR_INVALID_PARAMETER, indicates the wrong input parameter.

6.2. Memory API

Header file “*ql_memory.h*” declares memory related to APIs. These functions are essential to any custom applications. Make sure to include the header file.

The example in the “*example_multimemory.c*” of OpenCPU SDK shows the proper usages of these methods.

6.2.1. Define memory size

The memory size is defined by using *QL_MEMORY_HEAP_SIZE* in *ql_customer_config.c*.

For example

Defines the memory size to 10K bytes:

```
#define QL_MEMORY_HEAP_SIZE (10*1024)
```

6.2.2. Ql_GetMemory

This function allocates a block of memory large enough from memory pool.

- **Prototype**

```
void *Ql_GetMemory(u32 Size);
```

- **Parameter**

Size:

The size of memory will be allocated, unit is the byte.

- **Return value**

The address of the allocated memory. NULL will be returned if the allocation fails.

6.2.3. QI_FreeMemory

Deallocate or free a memory block.

- **Prototype**

```
s32 QI_FreeMemory (void *Ptr);
```

- **Parameters**

Ptr:

The address of allocated memory.

- **Return value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.2.4. QI_memMaxCanAllocSize

This function gets the maximum available free memory block that is allocable in SRAM.

- **Prototype**

```
u32 QI_memMaxCanAllocSize(void);
```

- **Parameters**

None.

- **Return Value**

The return value is Number of bytes if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.2.5. QI_memTotalLeftSize

This function gets the total number of bytes of the free space in SRAM.

- **Prototype**

```
u32 QI_memTotalLeftSize(void);
```

- **Parameters**

None.

- **Return Value**

The return value is Number of bytes if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.3. File system API

These interfaces are used to operate file system. In order to use these interfaces the header file “*ql_filesystem.h*” must be included.

The example in the “*example_file.c*” of OpenCPU SDK shows the proper usages of these methods.

NOTE

This stack size of the task, in which file operations will be executed, cannot be less than 4KB.

6.3.1. QI_FileGetFreeSize

This function obtains the total amount of free space of file system.

- **Prototype**

```
s32 QI_FileGetFreeSize(void);
```

- **Parameters**

None.

- **Return Value**

The return value is the total number of bytes of the free space in file system if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.3.2. QI_FileGetFreeSizeEx

This function obtains the amount of free space on Flash or SD card.

- **Prototype**

```
s32 Ql_FileGetFreeSizeEx(u8 storage);
```

- **Parameters**

storage:

The type of storage, which is one value of "QlFsStorageReg".

```
typedef enum QlFsStorageRegTag
{
    QL_FS_FLASH = 1,    //Flash region
    QL_FS_SD_CARD      //SD card region
}QlFsStorageReg;
```

- **Return Value**

The return value is the total number of bytes of the free space in the specified storage, if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.3.3. Ql_FileOpenEx

This function opens or creates a file with a specified name.

- **Prototype**

```
s32 Ql_FileOpenEx(u8* asciifilename, u32 Flag);
```

- **Parameters**

asciifilename:

The name of the file. The name is limited to 252 characters. You must use a relative path, such as "filename.ext" or "dirname\filename.ext".

Flag:

A u32 that defines the file's opening and access mode. The possible values are listed as follow:

QL_FS_READ_WRITE,	can read and write
QL_FS_READ_ONLY,	can only read
QL_FS_CREATE,	opens the file, if it exists. If the file does not exist, the function creates the file
QL_FS_CREATE_ALWAYS,	creates a new file. If the file exists, the function overwrites the file and clears the existing attributes.

- **Return Value**

If the function succeeds, the return value specifies a file handle. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.3.4. QI_FileRead

Read data from the specified file, starting at the position indicated by the file pointer. After the read operation has been completed, the file pointer is adjusted by the number of bytes actually read.

- **Prototype**

```
s32 QI_FileRead(s32 filehandle, u8 *readbuffer, u32 readlength, u32 *readedlen);
```

- **Parameters**

filehandle:

[in] A handle to the file to be read, which is the return value of the function.
"QI_FileOpen".

readbuffer:

[out] Pointer to the buffer that receives the data read from the file.

readlength:

[in] Number of bytes to be read.

readedlen:

[out] Pointer to the number of bytes has been read.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.3.5. QI_FileWrite

Write data to the specified file, starting at the position indicated by the file pointer. After the writing operation has been completed, the file pointer is adjusted by the number of bytes actually written.

- **Prototype**

```
s32 QI_FileWrite(s32 filehandle, u8 *writebuffer, u32 writelength, u32 *written);
```

- **Parameters**

filehandle:

[in] A handle to the file to be written, which is the return value of the function
"QI_FileOpen".

writebuffer:

[in] Pointer to the buffer containing the data to be written to the file.

writelength:

[in] Number of bytes to write to the file.

written:

[out] Pointer to the number of bytes written by the function call.

- **Return Value**

The return value is `QL_RET_OK` if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.3.6. **QI_FileSeek**

This function repositions the pointer in the previously opened file.

- **Prototype**

```
s32 QI_FileSeek(s32 filehandle, s32 Offset, s32 Whence);
```

- **Parameters**

filehandle:

[in] File handle, which is the return value of the function `QI_FileOpen`.

Offset:

[in] Number of bytes to move the file pointer.

Whence:

[in] Pointer movement mode. Must be one of the following values.

```
typedef enum QIFsSeekPosTag
{
    QL_FS_FILE_BEGIN,
    QL_FS_FILE_CURRENT,
    QL_FS_FILE_END
}QIFsSeekPos;
```

- **Return Value**

The return value is `QL_RET_OK` if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.3.7. **QI_FileGetFilePosition**

This function gets the current value of the file pointer.

- **Prototype**

```
s32 QI_FileGetFilePosition(s32 filehandle, u32 * Position);
```

- **Parameters**

filehandle:

[in] File handle, which is the return value of the function `QI_FileOpen`.

Position:

[out] Address of an u32 that will receive the current offset from the beginning of the file.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.3.8. **QI_FileTruncate**

This function truncates the specified file to zero length.

- **Prototype**

```
s32 QI_FileTruncate(s32 filehandle);
```

- **Parameters**

filehandle:

The file handle, it is the return value of the function "*QI_FileOpen*".

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.3.9. **QI_FileFlush**

Force any data remaining in the file buffer to be written to the file.

- **Prototype**

```
void QI_FileFlush(s32 filehandle);
```

- **Parameters**

filehandle:

The file handle, which is the return value of the function *QI_FileOpen*.

- **Return value**

None.

6.3.10. **QI_FileClose**

Closes the file associated with the file handle and makes the file unavailable for reading or writing.

- **Prototype**

```
void Ql_FileClose(s32 filehandle);
```

- **Parameters**

filehandle:

The file handle, which is the return value of the function *Ql_FileOpen*.

- **Return value**

None.

6.3.11. Ql_FileGetSize

This function retrieves the size, in bytes, of the specified file.

- **Prototype**

```
s32 Ql_FileGetSize(u8 *asciifilename, u32 *filesize);
```

- **Parameters**

asciifilename:

The name of the file. The name is limited to 252 characters. You must use a relative path, such as "filename.ext" or "dirname\filename.ext".

filesize:

A pointer to the variable where the file size, in bytes, is stored.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.3.12. Ql_FileDelete

This function deletes an existing file.

- **Prototype**

```
s32 Ql_FileDelete(u8 *asciifilename);
```

- **Parameters**

asciifilename:

The name of the file to be deleted. The name is limited to 252 characters. You must use a relative path, such as "filename.ext" or "dirname\filename.ext".

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.3.13. QI_FileCheck

This function checks whether the file exists or not.

- **Prototype**

```
s32 QI_FileCheck(u8 *asciifilename);
```

- **Parameters**

asciifilename:

The name of the file. The name is limited to 252 characters. You must use a relative path, such as "filename.ext" or "dirname\filename.ext".

- **Return Value**

QL_RET_OK indicates that the file exists.

If *QL_RET_ERR_FILENOTFOUND*, indicates the file does not exist. Otherwise, please see ERROR CODES.

6.3.14. QI_FileRename

This function renames an existing file.

- **Prototype**

```
s32 QI_FileRename(u8 *asciifilename, u8 *newasciifilename);
```

- **Parameters**

asciifilename:

The current name of the file. The name is limited to 252 characters. You must use a relative path, such as "filename.ext" or "dirname\filename.ext".

newasciifilename:

The new name of the file. The new name is different from the existing names. The name is limited to 252 characters. You must use a relative path, such as "filename.ext", "dirname\filename.ext".

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.3.15. QI_FileCreateDir

This function creates a directory.

- **Prototype**

```
s32 QI_FileCreateDir(u8 *asciidirname);
```

- **Parameters**

asciidirname:

The name of the directory to create. The name is limited to 252 characters. You must use a relative path, such as "dirname1" or "dirname1\dirname2".

- **Return value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.3.16. QI_FileRemoveDir

This function removes an existing directory.

- **Prototype**

```
s32 QI_FileRemoveDir(u8 *asciidirname);
```

- **Parameters**

asciidirname:

The name of the directory to be removed. The name is limited to 252 characters.

You must use a relative path, such as "dirname1" or "dirname1\dirname2".

- **Return value**

The return value is *QL_RET_OK* if this function succeeds. Other negative, the return value is an error code. To get extended error information, please see ERROR CODES.

6.3.17. QI_FileCheckDir

This function checks whether the directory exists or not.

- **Prototype**

```
s32 QI_FileCheckDir(u8 *asciidirname);
```

- **Parameters**

asciidirname:

The name of the directory. The name is limited to 252 characters. You must use a relative path, such as “dirname1” or “dirname1\dirname2”.

- **Return Value**

QL_RET_OK indicates the directory exists.

QL_RET_ERR_FILENOTFOUND indicates the directory does not exist. Other negative, please refer to ERROR CODES.

6.3.18. QI_FileFindFirst

Search a directory for a file or subdirectory which name matches the specified file name.

- **Prototype**

```
s32 QI_FileFindFirst(u8 *asciipath, u8 *asciifilename, u32 filenamelength, u32 *filesize, bool *isdir);
```

- **Parameters**

asciipath:

[in] The directory or path. The “*asciipath*” is limited to 252 characters, which can include wildcard characters. You must use a relative path, such as “dirname*” or “dirname*.txt”.

asciifilename:

[in] A pointer to the buffer that receives the name of the file.

filenamelength:

[in] The maximum number of bytes to be received of the name.

filesize:

[in] A pointer to the variable which represents the size specified by the file.

isdir:

[in] A pointer to the variable which represents the type specified by the file.

- **Return Value**

If the function succeeds, the return value is a search handle that can be used in a subsequent call to the “*QI_FindNextFile*” or “*QI_FindClose*” function.

QL_RET_ERR_FILENOMORE indicates that there’s no file and subdirectory found. Other negative indicates failure. To get extended error information, please see ERROR CODES.

6.3.19. QI_FileFindNext

This function continues a file to search from a previous call to the “*QI_FileFindFirst*” function.

- **Prototype**

```
s32 QI_FileFindNext(s32 handle, u8 *asciifilename, u32 filenamelength, u32 *filesize, bool *isdir);
```

- **Parameters**

handle:

The search handle returned by a previous call to the “*QI_FileFindFirst*” function.

asciifilename:

A pointer to the buffer that receives the name of the file.

filenamelength:

The maximum number of bytes to be received of the name.

filesize:

A pointer to the variable which represents the size specified by the file.

isdir:

A pointer to the variable whose type is specified by the file.

- **Return Value**

If positive or 0, it means the file handle has found successfully. If *QL_RET_ERR_FILENOMORE*, indicate no files and subdirectories. Other negative, please refer to chapter for ERROR CODES.

6.3.20. QI_FileFindClose

This function closes the specified search handle.

- **Prototype**

```
void QI_FileFindClose(s32 handle);
```

- **Parameters**

handle:

Find handle, returned by a previous call of the “*QI_FileFindFirst*” function.

- **Return Value**

None.

6.3.21. QI_FileXDelete

This function deletes a file or directory.

- **Prototype**

```
s32 QI_FileXDelete(u8* asciiopath, u32 flag);
```

- **Parameters**

asciipath:

File path to be deleted.

flag:

A u32 that defines the file's opening and access mode.

The possible values are shown as follow:

QL_FS_FILE_TYPE,
QL_FS_DIR_TYPE,
QL_FS_RECURSIVE_TYPE

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.3.22. QI_FileXMove

This function provides a facility to move or copy a file or folder

- **Prototype**

```
s32 QI_FileXMove(u8* asciisrcpath, u8* asciidestpath, u32 flag);
```

Parameters

asciisrcpath:

Source path to be moved or copied.

asciidestpath:

Destination path.

flag:

A u32 that defines the file's opening and access mode.

The possible values are shown as follow:

QL_FS_MOVE_COPY,
QL_FS_MOVE_KILL,
QL_FS_MOVE_OVERWRITE

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.3.23. QI_FileSys_GetSpaceInfo

This function gets the space information in file system area.

- **Prototype**

```
s32 Ql_FileSys_GetSpaceInfo(u8 storage, u32* freeSpace, u32* totalSpace);
```

- **Parameters**

storage:

One value of 'FileSys_Storage'

```
typedef enum tagFileSys_Storage
{
    Ql_FS_UFS = 1,
    Ql_FS_RAM = 2,
    Ql_FS_SD = 3
} FileSys_Storage;
```

freeSpace:

A pointer to the buffer that receives the free space of storage (Unit in bytes).

totalSpace:

A pointer to the buffer that receives the total space size of storage (Unit in bytes).

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.3.24. Ql_Fs_Format

This function format the SD card or UFS

- **Prototype**

```
s32 Ql_Fs_Format( QlFsFormatType type);
```

- **Parameters**

type:

The format storage ,which is one value of "QlFsFormat".

```
typedef enum QlFsFormatTypeTag
{
    QL_FS_FORMAT_UFS,
    QL_FS_FORMAT_SD,
}QlFsFormatType;
```

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.4. Periphery API

The interface functions in this section are declared in header file “ql_bus.h”, “ql_pin.h”.

6.4.1. Multifunctional Pins

OpenCPU provides abundant multifunctional pins for Embedded Application. Application can configure diverse function for each pin programmatically. Each multifunctional pin may work in one of four modes.

6.4.2. Pin Names

The definition of multifunctional pins is as below.

```
typedef enum QIPinNameTag
{
    /*****
    * <<<BEGIN: The fowllowing PINs for all modules
    * (TOTAL: 19)
    *****/
    QL_PINNAME_NETLIGHT = 0,
    QL_PINNAME_STATUS,
    QL_PINNAME_PCM_SYNC,
    QL_PINNAME_KBR0,
    QL_PINNAME_KBR1,
    QL_PINNAME_KBR2,
    QL_PINNAME_KBR3,
    QL_PINNAME_KBR4,
    QL_PINNAME_KBC0,
    QL_PINNAME_KBC1,
    QL_PINNAME_KBC2,
    QL_PINNAME_KBC3,
    QL_PINNAME_KBC4,
    QL_PINNAME_DCD,
    QL_PINNAME_RI,
    QL_PINNAME_DTR,
    QL_PINNAME_CTS,
    QL_PINNAME_RTS,
    QL_PINNAME_SIM_PRESENCE,

    /*****
    * <<<BEGIN: The fowllowing PINs for M12(R2.0)
    *****/
}
```

```

QL_PINNAME_PCM_RST,
QL_PINNAME_GPIO0,
QL_PINNAME_SD_DATA,
QL_PINNAME_SD_CLK,
QL_PINNAME_SD_CMD,
QL_PINNAME_GPIO2 = QL_PINNAME_PCM_SYNC,
QL_PINNAME_GPIO3 = QL_PINNAME_PCM_RST,
QL_PINNAME_GPIO1_KBC5 = QL_PINNAME_SD_CMD + 1,
QL_PINNAME_GPIO4,
QL_PINNAME_MAX
}QIPinName;

```

6.4.3. Working Modes

Each multifunctional pin may work in one of four Pin Modes once. The four modes are listed in the following enumeration.

```

typedef enum QIPinModeTag{
    QL_PINMODE_1 = 0,    // The default working mode
    QL_PINMODE_2 = 1,
    QL_PINMODE_3 = 2,
    QL_PINMODE_4 = 3,
    QL_PINMODE_UNSET = 255
} QIPinMode;

```

Table 1: Multifunctional PINs and working modes for M12 (R2.0)

PIN No	PIN NAME	RESET	MODE1	MODE2	MODE3	MODE4
1	QL_PINNAME_SD_DATA	O,L,PD	SD_DATA	GPIO		
2	QL_PINNAME_SD_CLK	O,L,PD	SD_CLK	GPIO		
3	QL_PINNAME_SD_CMD	O,H,PU	SD_CMD	GPIO		
4	QL_PINNAME_GPIO2	① O,L,PU		GPIO		
5	QL_PINNAME_GPIO3	② O,H,PU		GPIO		
6	QL_PINNAME_NETLIGHT	③ I,PD	NETLIGHT	GPIO	PWM	
11	QL_PINNAME_SIM_PRESENCE	I,PU	SIM_PRES	GPIO	EINT2	CLOCK
28	QL_PINNAME_KBR0	O	KBR0	GPIO		

PIN No	PIN NAME	RESET	MODE1	MODE2	MODE3	MODE4
29	QL_PINNAME_KBR1		KBR1	GPIO		
30	QL_PINNAME_KBR2		KBR2	GPIO		
31	QL_PINNAME_KBR3		KBR3	GPIO		
32	QL_PINNAME_KBR4		KBR4	GPIO		
33	QL_PINNAME_KBC0		KBC0	GPIO		
34	QL_PINNAME_KBC1		KBC1	GPIO		
35	QL_PINNAME_KBC2	I,PU	KBC2	GPIO		
36	QL_PINNAME_KBC3		KBC3	GPIO		
37	QL_PINNAME_KBC4		KBC4	GPIO		
38	QL_PINNAME_GPIO1_KBC5	I,PU	GPIO	KBC5	CLOCK	
39	QL_PINNAME_GPIO4	I,PD		GPIO		
54	QL_PINNAME_STATUS	I,PD	STATUS	GPIO		
55	QL_PINNAME_RI	I,PU	RI	GPIO		
56	QL_PINNAME_DCD	I,PU	DCD	GPIO	CLOCK	
57	QL_PINNAME_CTS	I,PU	CTS	GPIO		
58	QL_PINNAME_RTS	I,PU	RTS	GPIO		
59	QL_PINNAME_DTR	I,PU	DTR	GPIO	EINT0	CLOCK
64	QL_PINNAME_GPIO0	I,PU	GPIO	EINT1		

IMPORTANT NOTES FOR M12 (R2.0) GPIO:

Pin No.	Pin Name	Notes
4	QL_PINNAME_GPIO2 ①	Pull-up with 100K resistor within module.
5	QL_PINNAME_GPIO3 ②	Pull-up with 100K resistor within module.
6	QL_PINNAME_NETLIGHT ③	Pull-down with 100K resistor within module.

The 'MODE1' defines the original status of pin in standard module.

“RESET” column defines the default status of every pin after system powers on.

“I” means input.

“O” means output.

“PU” means internal pull-up circuit.

“PD” means internal pull-down circuit.

Customer may refer to the file "ql_cusxtomer_gpio.c" to initialize these PINs. When the working mode for PIN is set to QL_PINMODE_UNSET, the PIN will be configured with the default values which customer can referred to "RESET" mode in the above table; If the working mode is set to "QL_PINMODE1", the PIN will be configured with the values which customer can referred to "MODE1" mode in the above table. Configuration of the other modes is similar.

6.4.4. Configure multifunctional pins

Without configuration, these multifunctional pins have the same function status with the corresponding pins in standard module. Developer may configure these multifunctional pins to some initial working status.

```
const QIPinConfigTable Customer_QIPinConfigTable =
{
    QL_OPENCPU_FLAG,
    {
        // Pin Name          Pin Sub State          Pin Mode          pullen
        dir                  lvi                  pullsel
        {QL_PINNAME_NETLIGHT,QL_PINSUBSCRIBE_UNSUB,QL_PINMODE_1,QL_PINPULLENABLE_EN
        ABLE,QL_PINDIRECTION_OUT,QL_PINLEVEL_LOW, 0 },
        ...
        {QL_PINNAME_MAX, QL_PINSUBSCRIBE_UNSUB, QL_PINMODE_UNSET,          0,          0,          0,
        0} /*must end item is QL_PINNAME_MAX*/
    }
}
```

The "QIPinConfigTable" structure is defined as below:

```
typedef struct QIPinConfigTableTag
{
    s8 quectelflg[QL_OPENCPU_FLAG_MAXLEN];
    QIPinConfigItem pinconfigitem[QL_PINNAME_MAX+1]; /* must end item is QL_PINNAME_MAX
    */
}QIPinConfigTable;
```

The "QIPinConfigItem" structure is defined as below:

```
typedef struct QIPinConfigItemTag
{
    QIPinName      pinname;
    QIPinSubscribe subscribe;
    QIPinMode      pinmode;
    u32             parameter1;
    u32             parameter2;
    u32             parameter3;
    u32             parameter4;
}QIPinConfigItem;
```

subscribe:

Indicates whether the working mode of a pin takes effect. If this parameter is set to *QL_PINSUBSCRIBE_SUB*, the status of pin will be what the configuration is. And developers can directly do some operations later, such as *Ql_pinWrite()*, *Ql_pinRead()*. Or developers have to call *Ql_pinSubscribe()* to register the pin before calling *Ql_pinWrite()* or *Ql_pinRead()*.

pinmode:

Pin mode, a value of “*pinmode*”.

parameter:

Parameter of the pin. If “*pinmode*” is *QL_PINMODE_1* which is the default mode, this parameter has no effect and should be set as NULL. These parameters only take effect when GPIO mode is selected.

Parameter	Parameter1	Parameter2	Parameter3	Parameter4
Mode				
GPIO	Pull Enable	Direction	Level	Pull Selection

Please see 6.4.2.1 section for more details.

6.4.5. Pin functions

This section includes pin API functions that are applicable to request and control the functions of the pins.

The examples in the “*example_gpio.c*”, “*example_clk.c*” and “*example_pwm.c*” of OpenCPU SDK show the proper usages of these methods

6.4.6. Ql_pinSubscribe

This function subscribes a working mode for pin.

- **Prototype**

```
s32 Ql_pinSubscribe(QIPinName pinname, QIPinMode pinmode, QIPinParameter *pinparameter);
```

- **Parameter**

pinName:

The name of the pin.

pinmode:

Mode of the pin.

pinparameter:

Parameter of the pin. If *pinmode* is *QL_PINMODE_1* which is the default mode, this parameter has no effect and should be set as NULL. When other *pinmode* is chosen, the definition of this parameter will be related to the *pinmode*.

Below is the definition about data structure of this parameter.

```
typedef struct QIPinParameterTag
{
    QIPinParameterUnion    pinparameterunion;
}QIPinParameter;
```

```
typedef union QIPinParameterUnionTag
{
    QIGpioParameter        gpioparameter;
    QIEintParameter        eintparameter;
    QIClockParameter        clockparameter;
    QIPwmParameter        pwmparameter;
}QIPinParameterUnion;
```

- ◆ For GPIO function:

QIGpioParameter is the configuration parameter of GPIO.

```
typedef struct QIGpioParameterTag
{
    QIPinPullEnable    pinpullenable;
    QIPinDirection    pindirection;
    QIPinLevel        pinlevel;
    QIPinPullSel        pinpullsel;
}QIGpioParameter;
```

pinpullenable:

Enable pin pull-up or pull-down.

QL_PINPULLENABLE_DISABLE

QL_PINPULLENABLE_ENABLE

pindirection:

Set pin direction.

QL_PINDIRECTION_IN
QL_PINDIRECTION_OUT

pinlevel:

Set pin voltage level.

QL_PINLEVEL_LOW
QL_PINLEVEL_HIGH

pinpullsel:

Select pin pull up or pull down.

QL_PINPULLSEL_PULLDOWN
QL_PINPULLSEL_PULLUP

◆ For EINT function:

QIEintParameter is the configuration parameter of interrupt.

```
typedef struct QIEintParameterTag
{
    QIEintSensitiveType eintsensitivetyp;
    s32 hardware_de_bounce; /* ms */
    s32 software_de_bounce; /* ms */
}QIEintParameter;
```

eintsensitivetyp:

Set EINT sensitive type.

QL_EINTSENSITIVETYPE_EDGE
QL_EINTSENSITIVETYPE_LEVEL

hardware_de_bounce: set EINT hardware debounce time. Unit is ms; the maximum value is 63 ms.

software_de_bounce: set EINT software debounce time. Unit is ms. the maximum value is 2559 ms.

◆ For clock-out function:

QIClockParameter is the configuration parameter of clock-out.

```
typedef struct QIClockParameterTag
{
    QIClockSource clocksource;
}QIClockParameter;
```

clocksource:

Set clock source.

QL_CLOCKSOURCE_26M
QL_CLOCKSOURCE_13M

`QL_CLOCKSOURCE_6DOT5M`

`QL_CLOCKSOURCE_32K`

- ◆ For PWM function:

`QIPwmParameter` is the configuration parameter of PWM.

The output PWM frequency is determined by: $(\text{pwmsource}/\text{pwmclkdiv}) / (\text{lowpulesnumber} + \text{highpulesnumber})$.

```
typedef struct QIPwmParameterTag
{
    QIPwmSource      pwmsource;
    QIPwmSourceDiv   pwmclkdiv;
    u32               lowpulesnumber;
    u32               highpulesnumber;
}QIPwmParameter;
```

pwmsource:

Set PWM clock source.

`QL_PWMSOURCE_13M`

`QL_PWMSOURCE_32K` (the exact value is 32768Hz)

pwmclkdiv:

Set clock divide.

`QL_PWMSOURCE_DIV1`

`QL_PWMSOURCE_DIV2`

`QL_PWMSOURCE_DIV4`

`QL_PWMSOURCE_DIV8`

lowpulesnumber: set the number of clock cycles to stay at low level. The result of *lowpulesnumber* plus *highpulesnumber* is less than 8193.

highpulesnumber: set the number of clock cycles to stay at high level. The result of *lowpulesnumber* plus *highpulesnumber* is less than 8193.

6.4.7. QI_pinUnSubscribe

This function unsubscribes pin.

- **Prototype**

```
s32 QI_pinUnSubscribe(QIPinName pinname);
```

- **Parameter**

pinName:

The name of the pin.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.4.8. QI_pinQueryMode

This function queries the named pin's mode.

- **Prototype**

```
s32 QI_pinQueryMode(QIPinName pinname, QIPinSubscribe *subscribe, QIPinMode *pinmode,
QIPinParameter *pinparameter);
```

- **Parameters**

pinName:

Pin name.

subscribe:

The pointer to the pin's subscribe state.

pinmode:

The pointer to the pin's mode.

pinparameter:

Please refer to the function "*QI_pinSubscribe*".

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.4.9. QI_pinRead

This function inquires the specified pin's level.

- **Prototype**

```
s32 QI_pinRead(QIPinName pinname, QIPinLevel_e *pinlevel);
```

- **Parameters**

pinName:

The name of the pin.

pinlevel:

The pointer to the pin's level.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.4.10. QI_pinWrite

This function sets the specified pin's level. And this function is available only when the pin is configured as GPIO.

- **Prototype**

```
s32 QI_pinWrite(QIPinName pinname, QIPinLevel_e pinlevel);
```

- **Parameters**

pinName:

The name of the pin.

pinlevel:

The pointer to the pin's level

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.4.11. QI_pinControl

This function controls the specified pin. This function takes effect only when the pin is set to clock-out, PWM, or ALERT function mode.

- **Prototype**

```
s32 QI_pinControl(QIPinName pinname, QIPinControl_e pincontrol);
```

- **Parameters**

pinName:

The name of the pin.

pincontrol:

The operation of pin. For example, when the pin is set to clock-out function mode, *QL_PINCONTROL_START* indicates starting clock output, and *QL_PINCONTROL_STOP* indicates stopping clock output.

- **Return Value**

The return value is `QL_RET_OK` if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.4.12. QI_eintRead

This function reads level state of a specified interruption pin.

- **Prototype**

```
void QI_eintRead(u8 eintno, QIPinLevel *pinlevel);
```

Parameters

eintno:

[in] External interruption number

pinlevel:

[out] A pointer to 'QIPinLevel'

- **Return Value**

None.

6.4.13. QI_eintMask

The function masks or unmasks a specified external interruption.

- **Prototype**

```
void QI_eintMask(u8 eintno, bool mask);
```

- **Parameters**

eintno:

External interruption number

mask:

TRUE or FALSE, mask or unmask

- **Return Value**

None.

6.4.14. QI_eintSetPolarity

This function sets the polarity of the level of external interruption source.

- **Prototype**

```
void QI_eintSetPolarity(u8 eintno, QIPinLevel Polaritylevel);
```

- **Parameters**

eintno:

Pin name

Polaritylevel:

Level polarity; see the definition of 'QIPinLevel'.

- **Return Value**

None.

6.4.15. QI_ReadADC

This function reads the level value of ADC pin.

- **Prototype**

```
s32 QI_ReadADC(QIADCPin adc_pin, OCPU_CB_READ_ADC cb_adc);
```

- **Parameters**

adc_pin:

[in] One value of 'QIADCPin'.

```
typedef enum QIADCPintag  
{  
    QL_PIN_ADC0,  
    QL_PIN_ADC1,  
    QL_PIN_ADC_MAX  
}QIADCPin;
```

cb_adc:

[in] Callback, which will report the results.

- **Return Value**

The return value is QL_RET_OK if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.4.16. QI_GetGpioByName

This function gets GPIO number by pin name.

- **Prototype**

```
s32 Ql_GetGpioByName(QIPinName pin);
```

- **Parameters**

pin:

One value of "QIPinName".

- **Return value**

If this function successes, the gpio number is returned. Negative indicates failure.

6.4.17. BUS functions

This section describes API functions that are applicable to request and control the function of the bus. The example in the "example_i2c.c" of OpenCPU SDK shows the proper usages of these methods.

6.4.18. Ql_busSubscribe

This function subscribes BUS function.

- **Prototype**

```
QL_BUS_HANDLE Ql_busSubscribe(QIBusType bustype, QIBusParameter *busparameter);
```

- **Parameter**

bustype:

QL_BUSTYPE_LCD or QL_BUSTYPE_I2C.

busparameter:

A union structure. It has different definition of data structure according to the "bustype". Below is the definition of data structure.

```
typedef struct QIBusParameterTag
{
    s16 busconfigversion;
    QIBusParameterUnion busparameterunion;
}QIBusParameter;
```

```
typedef union QIBusParameterUnionTag
{
    QILcdParameter lcdparameter;
    QII2cParameter i2cparameter;
}QIBusParameterUnion;
```

Busconfigversion:

Must be set to QL_BUS_VERSION.

For I²C BUS, *QI2cParameter* is the configuration parameter of I²C.

```
typedef struct QI2cParameterTag
{
    QIPinName          pin_i2cdata;
    QIPinName          pin_i2cclk;
}QI2cParameter;
```

pin_i2cdata:

The pin that used as I2C data line.

pin_i2cclk:

The pin that used as I2C colock line.

- **Return value**

The return value is a positive number, which indicates the handle of bus, if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

NOTES

1. If *bustype* is QL_BUSTYPE_LCD, all LCD related to pins, QL_PINNAME_DISP_DATA, QL_PINNAME_DISP_CLK, QL_PINNAME_DISP_CS, QL_PINNAME_DISP_DC, and QL_PINNAME_DISP_RST will be used. These pins must be in unsubscribed state before the BUS is subscribed.
2. If *bustype* is QL_BUSTYPE_I2C, two I2C BUS related pins must also be in unsubscribed state.

6.4.19. QI_busUnSubscribe

This function cancels the subscribing for BUS.

- **Prototype**

```
s32 QI_busUnSubscribe(QL_BUS_HANDLE bushandle);
```

- **Parameter**

bushandle:

BUS handle is returned by "*QI_busSubscribe*".

- **Return Value**

The return value is QL_RET_OK if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.4.20. Ql_busWrite

This function writes data to the BUS.

• Prototype

```
s32 Ql_busWrite(QL_BUS_HANDLE bushandle, QlBusAccess *busAccess, u8 *data_p, u32 datalen);
```

• Parameters

bushandle:

BUS handle is returned by “*Ql_busSubscribe*”.

busAccess:

For different BUS type, this parameter has different meanings.

```
typedef struct QlBusAccessTag
{
    u32 address;
    u32 opcode;
}QlBusAccess;
```

Regarding to the function of parameter *address*, it indicates a 7-bit address of I²C slave device when the BUS is an I²C BUS. It has no effect when the BUS is an LCD BUS.

◆ For I²C bus:

The parameter “*opcod*” is set to *QL_BUSACCESSOPCODE_I2C_NOTSTOPBIT*, no stop bit will be sent. When *opcode* is set as *QL_BUSACCESSOPCODE_I2C_NOTHING*, the stop bit will be sent.

◆ For LCD bus:

When the parameter *opcode* is set to *QL_BUSACCESSOPCODE_LCD_CLEAR_RESET*, the pin *QL_PINNAME_DISP_RST*, which should be configured to LCD function in advance, will pull down to reset LCD. In this case, parameter *data_p* and *datalen* have no effect and should be set to NULL.

When the parameter *opcode* is set to *QL_BUSACCESSOPCODE_LCD_SET_RESET*, the pin *QL_PINNAME_DISP_RST*, which should be configured to LCD function in advance, will pull up to resume reset pin. In this case, parameter *data_p* and *datalen* have no effect and should be set to NULL.

When the parameter *opcode* is set to *QL_BUSACCESSOPCODE_LCD_DATAWRITE*, the parameter *data_p* pointing to data buffer will be sent to LCD.

When the parameter *opcode* is set to *QL_BUSACCESSOPCODE_LCD_CMDWRITE*, the parameter *data_p* point to the data buffer of LCD command will be sent to LCD.

data_p:

Address of the data that will be written to bus.

Datalen:

Length of the written data.

- **Return Value**

The return value is `QL_RET_OK` if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.4.21. QI_busRead

This function reads data from BUS. And this function is available only when the BUS is I2C bus.

- **Prototype**

```
s32 QI_busRead(QL_BUS_HANDLE bushandle, QIBusAccess *busAccess, u8 *data_p, u32 datalen);
```

- **Parameters**

bushandle:

BUS handle returned by “*QI_busSubscribe*”.

busAccess:

For different BUS type, this parameter has different meanings.

```
typedef struct QIBusAccessTag
{
    u32 Address;
    u32 opcode;
}QIBusAccess;
```

Regarding to the function of parameter *address*, it indicates a 7-bit address of I2C slave device when the bus is an I2C bus.

When the parameter *opcode* is set to `QL_BUSACCESSOPCODE_I2C_NOTSTOPBIT` for I2C BUS, no stop bit will be sent. When *opcode* is set to other value, the stop bit will be sent.

data_p:

Address of the buffer to save data which have been read.

Datalen:

Length of the data buffer to read.

- **Return Value**

The return value is `QL_RET_OK` if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.4.22. QI_busQuery

This function inquires the configuration of the BUS.

- **Prototype**

```
s32 Ql_busQuery(QL_BUS_HANDLE bushandle, QlBusType *bustype, QlBusParameter *busparameter);
```

- **Parameter**

bushandle:

[in] Bus handle returned by “Ql_busSubscribe”.

bustype:

[out] Pointer to the buffer that stores the current BUS.

busparameter:

[out] Pointer to the buffer that stores the configuration parameter of the bus.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.4.23. I2C functions

The section introduces the hardware I2C related API functions. Please refer to the example in the “*example_i2c.c*” of OpenCPU SDK for the proper usages of hardware I2C function.

6.4.24. Ql_I2C_Init

This function initializes the hardware I²C device. It should be invoked before using this I2C function.

- **Prototype**

```
void Ql_I2C_Init(void);
```

- **Parameter**

None

- **Return value**

None

6.4.25. Ql_I2C_Config

This function configures some parameters for one I2C owner.

- **Prototype**

```
s32 Ql_I2C_Config(I2C_OWNER owner, I2c_config_struct* config);
```

- **Parameter**

owner:

The slave device. Every slave device has its own enum

```
typedef enum
{
    HW_I2C_OWNER_1=6,
    HW_I2C_OWNER_2,
    HW_I2C_OWNER_3,
    HW_I2C_OWNER_4,
    HW_I2C_OWNER_5,
    HW_I2C_OWNER_6,
    HW_I2C_NUM_OF_OWNER
}I2C_OWNER;
```

config:

A pointer to "I2C_config_struct".

```
typedef struct
{
    I2C_MODE    I2C_mode;        // Transaction mode for existing I2c APIs
    bool        get_handle_wait; //When get handle wait until the I2c is available
    u8          slave_address;    //The address of the slave device
    u8          delay_len;        //Number of half pulse between transfers in a trasaction
    I2C_TRANSACTION_MODE  ransaction_mode; //I2C_TRANSACTION MODE
    u32         Fast_Mode_Speed;  //The speed of I2C fast mode(Kb)
    u32         HS_Mode_Speed;    //The speed of I2C high speed mode(Kb)
}I2c_config_struct;
```

```
typedef enum
{
    I2C_SW_8BIT=1,    //not support
    I2C_SW_16BIT,     //not support
    I2C_HW_8BIT,
    I2C_HW_16BIT
} I2C_MODE;
```

```
typedef enum
{
    I2C_TRANSACTION_FAST_MODE,
    I2C_TRANSACTION_HIGH_SPEED_MODE
}I2C_TRANSACTION_MODE;
```

If you use "I2C_TRANSACTION_FAST_MODE" mode, the range of "Fast_Mode_Speed" is 100~400(Kb).
If you use "I2C_TRANSACTION_HIGH_SPEED_MODE" mode, the range of "HS_Mode_Speed" is 400~3400(Kb).

The parameter of “delay_len” is not used; you can initialize it with ‘0’.

- **Return value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.4.26. QI_I2C_Read

This function read data form the specific slave device.

- **Prototype**

```
s32 QI_HW_I2cRead(I2C_OWNER owner,u8* para,u32 datalen);
```

- **Parameters**

owner:

the slave device. Every slave device has its own enum

para:

A pointer to the read data buffer.

datalen:

The number of bytes write to the specific slave device, the range of datalen is 1<datalen<8.

- **Return value**

If the function succeeds returns the number bytes of Read or returns an Error CodeThe return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.4.27. QI_I2C_Write

This function writes data to the specified slave device.

- **Prototype**

```
s32 QI_HW_I2cWrite(I2C_OWNER owner,u8* para,u32 datalen);
```

- **Parameters**

owner:

the slave device. Every slave device has its own enum

para:

A pointer to the Write data buffer.

datalen:

The number of bytes to write, the range of “datalen” is from 1 to 8.

- **Return value**

If the function succeeds returns the number bytes of Write or returns an Error Code. The return value is `QL_RET_OK` if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5. AUDIO API

File "`ql_audio.h`" needs to be included before calling audio functions. The example in the "`example_audio.c`" of OpenCPU SDK shows the proper usages of these methods

6.5.1. QI_PlayAudio

This function plays the predefined music in system. When Audio Play stops, an `EVENT_MEDIA_FINISH` event will be received by embedded application.

- **Prototype**

```
s32 QI_PlayAudio(QIAudioName name, bool repeat);
```

- **Parameters**

name:

The audio name. A value of "`QIAudioName`".

```
typedef enum QIAudioNameTag
{
    QL_AUDIO_EMS_CHIMES_HI = 1,
    QL_AUDIO_EMS_CHIMES_LO,
    QL_AUDIO_EMS_DING,
    QL_AUDIO_EMS_TADA,
    QL_AUDIO_EMS_NOTIFY,
    QL_AUDIO_EMS_DRUM,
    QL_AUDIO_EMS_CLAPS,
    QL_AUDIO_EMS_FANFARE,
    QL_AUDIO_EMS_CHORD_HI,
    QL_AUDIO_EMS_CHORD_LO,
    QL_AUDIO_1,
    QL_AUDIO_2,
    QL_AUDIO_3,
    QL_AUDIO_4,
    QL_AUDIO_5,
    QL_AUDIO_6,
    QL_AUDIO_7,
    QL_AUDIO_8,
    QL_AUDIO_9,
```

```
QL_AUDIO_10,  
QL_AUDIO_11,  
QL_AUDIO_12,  
QL_AUDIO_13,  
QL_AUDIO_14,  
QL_AUDIO_15,  
QL_AUDIO_16,  
QL_AUDIO_17,  
QL_AUDIO_18,  
QL_AUDIO_19,  
QL_AUDIO_END  
}QIAudioName;
```

repeat:

If TRUE, the audio will be played repeatedly until *QI_StopAudio* is called to stop playing audio. Otherwise, the audio will be played one time.

- **Return value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.2. QI_StopAudio

This function stops playing music

- **Prototype**

```
s32 QI_StopAudio(QIAudioName name);
```

- **Parameter**

name:

The audio name. A value of "*QIAudioName*". Please refer to "*QI_PlayAudio*".

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.3. QI_StartPlayAudioFile

This function plays the audio file in the system. It supports wav, mp3, amr audio format. When audio play stops, an *EVENT_MEDIA_FINISH* event will be received by embedded application.

- **Prototype**

```
s32 QI_StartPlayAudioFile(u8 *asciifilename, bool repeat, u8 volumelevel, u8 audiopath);
```

- **Parameters**

asciifilename:

The name of the audio file. The name is limited to 252 characters. You must use a relative path, such as "filename.wav" or "dirname\filename.mp3"

repeat:

If TRUE, the audio will be played repeatedly until *QI_StopPlayAudioFile* is called to stop playing audio. Otherwise, the audio will be played one time.

volumelevel:

Set audio volume, reference for "QIAudioVolumeLevel".

```
typedef enum QIAudioVolumeLeveltag
{
    QL_AUDIO_VOLUME_LEVEL1 = 0,
    QL_AUDIO_VOLUME_LEVEL2,
    QL_AUDIO_VOLUME_LEVEL3,
    QL_AUDIO_VOLUME_LEVEL4,
    QL_AUDIO_VOLUME_LEVEL5,
    QL_AUDIO_VOLUME_LEVEL6,
    QL_AUDIO_VOLUME_LEVEL7,
    QL_AUDIO_VOLUME_LEVEL_END
}QIAudioVolumeLevel;
```

audiopath:

Set audio play path.

```
typedef enum QIAudioPlayPathTag
{
    QL_AUDIO_PATH_HEADSET = 1,      /* earphone */
    QL_AUDIO_PATH_LOUDSPEAKER = 2, /* loudspeaker for free sound */
    QL_AUDIO_PATH_END
}QIAudioPlayPath;
```

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.4. QI_PlayAudioFile_8k

This function plays an audio file with 8 kHz sample, and wav format. When audio play stops, an EVENT_MEDIA_FINISH event will be received by embedded application.

- **Prototype**

```
s32 QI_PlayAudioFile_8k(u8* file_name, bool repeat, u8 volume, u8 path, u8 dlvolume);
```

- **Parameters**

file_name:

The name of the audio file. The name is limited to 252 characters. You must use a relative path, such as "filename.wav" or "dirname\filename.mp3".

repeat:

If TRUE, the audio will be played repeatedly until *QI_StopPlayAudioFile* is called to stop playing audio. Otherwise, the audio will be played one time.

volumelevel:

Set upstream audio volume. One value of "QIAudioVolumeLevel". Please refer to "QI_StartPlayAudioFile".

audiopath:

Set audio play path. One value of "QIAudioPlayPath". Please refer to "QI_StartPlayAudioFile".

dlvolume:

Set downstream audio volume. One value of "QIAudioVolumeLevel". Please refer to "QI_StartPlayAudioFile".

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.5. QI_StopAudioFile_8k

This function stops playing 8k wav file.

- **Prototype**

```
s32 QI_StopAudioFile_8k(void);
```

- **Parameters**

None.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.6. QI_StopPlayAudioFile

This function stops playing audio file.

- **Prototype**

```
s32 Ql_StopPlayAudioFile(void);
```

- **Parameters**

None.

- **Return value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.7. Ql_StartPlayAudioStream

This function plays the audio stream in the system. It supports wav, mp3, amr audio format. Customer uses *QIAudioResGen.exe* to build the audio stream data.

When audio play stops, an *EVENT_MEDIA_FINISH* event will be received by embedded application.

- **Prototype**

```
s32 Ql_StartPlayAudioStream(u8 *stream, u32 streamsize, s32 streamformat, bool repeat, u8 volume, u8 audiopath);
```

- **Parameters**

stream:

The audio stream data.

streamsize:

Size of the audio stream data.

streamformat:

Format of the audio stream data.

```
typedef enum QIAudioStreamFormattag
{
    QL_AUDIO_STREAMFORMAT_MP3 = 1,
    QL_AUDIO_STREAMFORMAT_AMR = 2,
    QL_AUDIO_STREAMFORMAT_WAV = 3,
    QL_AUDIO_STREAMFORMAT_END
}QIAudioStreamFormat;
```

repeat:

If TRUE, The audio will be played repeatedly until "*Ql_StopPlayAudioStream*" is called to stop playing audio. Otherwise, the audio will be played one time.

volumelevel:

Set audio volume, please refer to "*Ql_StartPlayAudioFile*".

audiopath:

Set audio paly path, please refer to “*QI_StartPlayAudioFile*”.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.8. QI_StopPlayAudioStream

This function stops playing audio stream.

- **Prototype**

```
s32 QI_StopPlayAudioStream(void);
```

- **Parameters**

None.

- **Return value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.9. QI_VoiceCallChangePath

This function switches voice output source.

- **Prototype**

```
bool QI_VoiceCallChangePath(QIAudioPlayPath path);
```

- **Parameters**

path:

Voice output source. One value of “*QIAudioPlayPath*”. Please refer to “*QI_StartPlayAudioFile*”.

- **Return value**

TRUE if the function succeeds.

FALSE, indicates failure.

6.5.10. QI_VoiceCallGetCurrentPath

This function retrieves the current voice output source.

- **Prototype**

```
s32 QIAudioPlayPath QI_VoiceCallGetCurrentPath(void);
```

- **Parameters**

None.

- **Return value**

The return value is the current voice output source if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.11. QI_SetVolume

This function sets the volume of audio.

- **Prototype**

```
s32 QI_SetVolume(u8 vol_type, u8 vol_level);
```

- **Parameters**

vol_type:

The volume type.

typedef enum

```
{  
    VOL_TYPE_CTN   = 0 ,    /* Call tone attribute */  
    VOL_TYPE_KEY   = 1 ,    /* Keypad tone attribute */  
    VOL_TYPE_MIC    = 2 ,    /* Microphone attribute */  
    VOL_TYPE_GMI    = 3 ,    /* Melody, imelody, midi attribute */  
    VOL_TYPE_SPH    = 4 ,    /* Speech sound attribute */  
    VOL_TYPE_SID    = 5 ,    /* Side tone attribute */  
    VOL_TYPE_MEDIA  = 6 ,    /* As MP3, Wave,... attribute */  
    MAX_VOL_TYPE    = 7 .  
}volume_type_enum;
```

vol_level:

The volume level. Range is 0-100.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.12. QI_SetVolume_Ex

This function sets the volume of audio on output source. Calling this function can change the volume of

TTS or the ring of coming call.

- **Prototype**

```
s32 QI_SetVolume_Ex(u8 vol_type, u8 vol_path, u8 vol_level);
```

- **Parameters**

vol_type:

The volume type. Please refer to “*QI_SetVolume*”.

vol_path:

The audio output source.

```
typedef enum QIAudioPlayPathTag
```

```
{  
    QL_AUDIO_PATH_NORMAL = 0,      /* speak1 */  
    QL_AUDIO_PATH_HEADSET = 1,     /* earphone */  
    QL_AUDIO_PATH_LOUDSPEAKER = 2, /* speaker2, loudspeaker for free sound */  
    QL_AUDIO_PATH_END  
}QIAudioPlayPath;
```

vol_level:

The volume level. Range is 0-100.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.13. QI_GetVolume

This function gets the volume level of audio.

- **Prototype**

```
s32 QI_GetVolume(u8 vol_type);
```

- **Parameters**

vol_type:

The volume type. Please refer to “*QI_SetVolume*”.

- **Return value**

The return value is the volume level if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.14. QI_SetMicGain

This function sets the microphone gain level.

- **Prototype**

```
s32 QI_SetMicGain(u8 channel, u8 gain_level);
```

- **Parameters**

channel:

The audio output source.

```
typedef enum QIAudioPlayPathTag
{
    QL_AUDIO_PATH_NORMAL = 0,      /* speak1 */
    QL_AUDIO_PATH_HEADSET = 1,     /* earphone */
    QL_AUDIO_PATH_LOUDSPEAKER = 2, /* speaker2, loudspeaker for free sound */
    QL_AUDIO_PATH_END
}QIAudioPlayPath;
```

gain_level:

The gain level. Range is 0-15.

- **Return value**

The return value is QL_RET_OK if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.15. QI_GetMicGain

This function gets the microphone gain level.

- **Prototype**

```
s32 QI_GetMicGain(u8 channel);
```

- **Parameters**

channel:

The audio output source. Please refer to “QI_SetMicGain”

- **Return value**

The return value is the microphone gain level if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.16. QI_SetSideToneGain

This function sets the sidetone gain level.

- **Prototype**

```
s32 QI_SetSideToneGain(u8 channel, u8 gain_level);
```

- **Parameters**

channel:

The audio output source. Please refer “QI_SetMicGain”.

gain_level:

The gain level. Range is 0-15.

- **Return Value**

The return value is QL_RET_OK if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.17. QI_GetSideToneGain

This function gets the sidetone gain level.

- **Prototype**

```
s32 QI_GetSideToneGain(u8 channel);
```

- **Parameters**

channel:

The audio output source. Please refer to “QI_SetMicGain”.

- **Return Value**

The return value is the sidetone gain level if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.18. QI_CtrlEchoCancel

This function extends echo cancellation control.

- **Prototype**

```
s32 QI_CtrlEchoCancel(u16 ctrlWord, u16 nlp, u16 suppression,  
u16 nr, u8 channel);
```

- **Parameters**

ctrlWord:

- 221 suitable for handset and handset applications
- 224 suitable for handfree
- 0 means disabling all echo algorithm

nlp:

- Range is 0-2048. The greater the value, the more reduction of echo.
- 0 means disabling the NLP algorithm.

suppression:

- Range is 0-32767. The smaller the value, the more reduction of echo.
- 0 means disabling the echo suppression algorithm.

Nr:

- Noise reduction controller. Should NOT be set to 0.
- 849 suitable for handset and handset for applications.
- 374 suitable for handfree.

channel:

- The audio output source. Please refer *Ql_SetMicGain*.

- **Return Value**

The return value is `QL_RET_OK` if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.19. QI_VTS

The function allows the transmission of DTMF tones and arbitrary tones in voice mode.

- **Prototype**

```
s32 QI_VTS(u8* dtmf_str, u8 len);
```

- **Parameters**

dtmf_str:

- The pointer to DTMF string.

len:

- The length of DTMF string, Max value is 20.

- **Return Value**

The return value is `QL_RET_OK` if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.20. QI_StartRecord

This function start recording.

- **Prototype**

```
s32 QI_StartRecord(char *szFileName, QIAudioRecordFormat format);
```

- **Parameters**

szFileName:

The record file name. must be NULL terminated string.prefix with "SD:" to access the file in card.

format:

recording format. Refer to QIAudioRecordFormat.

```
typedef enum
{
    QL_AUD_REC_FMT_BY_EXTENSION = -1,
    QL_AUD_REC_FMT_AMR = 0,
    QL_AUD_REC_FMT_WAV,
    QL_AUD_REC_FMT_WAV_ALAW,
    QL_AUD_REC_FMT_WAV_ULAW,
    QL_AUD_REC_FMT_WAV_DVI_ADPCM
}QIAudioRecordFormat;
```

- **Return Value**

The return value is QL_RET_OK if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.21. QI_StartRecord_FullFunc

This function starts recording.

- **Prototype**

```
s32 QI_StartRecord_FullFunc(char *szFileName, QIAudioRecordFormat format, u8 quality, bool
fUseDefInput, u8 inputDev,u32 size_limit,u32 time_limit);
```

- **Parameters**

szFileName:

The record file name must be NULL terminated string.prefix with "SD:" to access the file in card.

format:

recording format. Refer to QIAudioRecordFormat.

quality:

record quality. 1 = high, 0 = low.

fUseDeflInput:

whether to use the default input device.

inputDev:

input source.

size_limit:

Max record size in byte. 0: unlimited.

time_limit:

Max record time in second. 0: unlimited

- **Return Value**

The return value is QL_RET_OK if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.22. QL_StopRecord

This function stops the recording.

- **Prototype**

```
s32 QL_StopRecord(void);
```

- **Parameters**

None

- **Return Value**

The return value is QL_RET_OK if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.23. QL_PlayTone

This function starts playing tone.

- **Prototype**

```
s32 QL_PlayTone(u32 freq, u32 period_on, u32 period_off, u32 duration);
```

- **Parameters**

freq:

frequency of the tone.

Period_on:

duration(ms)of tone is ON in one cycle.

duration:

total duration(ms).

- **Return Value**

The return value is QL_RET_OK if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.24. Ql_StopTone

This function stops tone.

- **Prototype**

```
s32 Ql_StopTone(void);
```

- **Parameters**

None:

- **Return value**

The return value is QL_RET_OK if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.25. Ql_SelectRingTone

This function selects the ring tone.

- **Prototype**

```
s32 Ql_SelectRingTone(u8 tone_id);
```

- **Parameters**

tone_id:

tone id : range from 1 to 10.

- **Return value**

The return value is QL_RET_OK if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.26. Ql_WDTMF

This function allows the transmission of DTMF tones and arbitrary tones in voice mode.

- **Prototype**

```
s32 Ql_WDTMF(u8 ul_volume, u8 dl_volume, ascii *str ,u16 len);
```

- **Parameters**

ul_volume:

Uplink channel of the volume. range 0~7.

dl_volume:

downlink channel of the volume. range 0~7

str:

"<dtmfcode>,<continuancetime>,<mutetime>"

len:

the length of the str.

- **Return Value**

The return value is QL_RET_OK if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.5.27. QI_EnabledDTMF

The function enables dtmf detection.

- **Prototype**

```
s32 QI_EnabledDTMF(OCPU_CB_DTMF_INFO cb_dtmf_info);
```

- **Parameters**

cb_dtmf_info:

[out] point to callback function.

- **Return value**

QL_RET_OK, suceess

QL_RET_ERR_GENERAL_FAILURE, indicates general failture.

6.5.28. QI_DisableDTMF

The function is used to disable DTMF detecting.

- **Prototype**

```
s32 QI_DisableDTMF(void);
```

- **Parameters**

None.

- **Return value**

QL_RET_OK, success

6.5.29. QI_GetDTMFConfig

The function gets the current DTMF configurations.

- **Prototype**

```
s32 QI_GetDTMFConfig(QIDTMFCONFIG *dtmfconfig);
```

- **Parameters**

dtmfconfig :

a structure of QIDTMFCONFIG.

```
typedef struct{  
    QIDTMFMODE mode;        //Mmode config  
    u32 prefixpause;        // Prefix pause number  
    u32 lowthreshold;        // Low threshold value  
    u32 highthreshold;      // High threshold value  
}QIDTMFCONFIG;
```

[in]dtmfconfig.mode, mode config (range 0~2)
[out]dtmfconfig.prefixpause, prefix pause number(range 0~u32)
[out]dtmfconfig.lowthreshold, low threshold tone (range 0~u32)
[out]dtmfconfig.highthreshold, high threshold tone(range 0~u32)

- **Return Value**

QL_RET_OK, success

QL_RET_ERR_PARAM, indicates error.

6.5.30. QI_SetDTMFConfig

The function is used to set DTMF config.

- **Prototype**

```
s32 QI_SetDTMFConfig(QIDTMFCONFIG dtmfconfig);
```

- **Parameters**

dtmfconfig :

a structure of QIDTMFCONFIG.

```
typedef struct{
    QIDTMFMODE mode;      // Mode config
    u32 prefixpause;      // Prefix pause number
    u32 lowthreshold;      // Low threshold value
    u32 highthreshold;     // High threshold value
}QIDTMFCONFIG;

[in]dtmfconfig.mode,      mode config (range 0~2)
[in]dtmfconfig.prefixpause, prefix pause number(range 0~u32)
[in]dtmfconfig.lowthreshold, low threshold tone (range 0~u32)
[in]dtmfconfig.highthreshold, high threshold tone(range 0~u32)
```

- **Return value**

QL_RET_OK, success

QL_RET_ERR_PARAM, indicates error.

6.5.31. QI_SetRingToneEnable

The function is used to control incoming ring.

- **Prototype**

```
s32 QI_SetRingToneEnable(QIRingToneEnable ring_enable);
```

- **Parameters**

mode:

0:normal mode, 1:mute mode

- **Return value**

QL_RET_OK, success

QL_RET_ERR_PARAM, indicates error.

6.6. TIMER API

File “ql_timer.h” needs to be included before calling the following functions.

6.6.1. TIMER STRUCTURE

```
typedef struct QITimerTag
{
    ticks      timeoutPeriod; //the time elapse before the timer times out
    u32        timerId;       // the ID of the timer
```

```
}QITimer;
```

6.6.2. QI_StartTimer

This function starts a timer.

- **Prototype**

```
u32 QI_StartTimer(QITimer * timer_p);
```

- **Parameter**

timer_p:

Timer to be started

- **Return value**

Timer Id (positive number) indicates success in starting the timer.

QL_RET_ERR_INVALID_TIMER indicates invalid timer.

QL_RET_ERR_TIMER_FULL indicates all timers are used up.

- **Example**

```
QLtimer timerDemo;  
// set timeout period to 2 seconds  
timerDemo.timeoutPeriod = QI_SecondToTicks(2);  
//start the timer  
QI_StartTimer(&timerDemo);
```

6.6.3. QI_StopTimer

This function stops the previously raised timer.

- **Prototype**

```
s16 QI_StopTimer(QITimer *timer_p);
```

- **Parameter**

timer_p:

The timer to be stopped

- **Return value**

QL_RET_OK indicates succeed in stopping the timer

QL_RET_ERR_INVALID_TIMER indicates invalid timer

6.6.4. QI_SecondToTicks

This function converts time from seconds to ticks.

- **Prototype**

```
ticks QI_SecondToTicks(u32 seconds);
```

- **Return Value**

Ticks are equivalent to the seconds.

6.6.5. QI_MillisecondToTicks

This function converts time from milliseconds to ticks.

- **Prototype**

```
ticks QI_MillisecondToTicks(u32 milliseconds);
```

- **Return value**

Ticks are equivalent to the milliseconds.

6.6.6. QI_GetRelativeTime

This function returns the number of milliseconds since the device booted.

- **Prototype**

```
u32 QI_GetRelativeTime(void);
```

- **Parameters**

None.

- **Return Value**

Number of milliseconds.

6.6.7. QI_GetRelativeTime_Counter

This function returns the number of MCU counters since the device is booted.

- **Prototype**

```
u32 QI_GetRelativeTime_Counter(void);
```

- **Parameters**

None.

- **Return value**

The return value is the number of MCU counters.

6.6.8. QI_GetLocalTime

This function gets the local time.

- **Prototype**

```
bool QI_GetLocalTime (QISysTimer * datetime);
```

- **Parameter**

datetime:

A “*QISysTimer*” struct for storing current local time.

“*QISysTimer*” are defined as below:

```
typedef struct QISysTimerTag
{
    unsigned short year;
    unsigned char month;
    unsigned char day;
    unsigned char hour;
    unsigned char minute;
    unsigned char second;
}QISysTimer;
```

- **Return Value**

TRUE, if the function succeeds. Otherwise, returns *FALSE*.

6.6.9. QI_SetLocalTime

This function sets the current local date and time.

- **Prototype**

```
bool QI_SetLocalTime(QISysTimer * datetime);
```

- **Parameter**

datetime:

A pointer to *QISysTimer*, which carries the information of date and time.

- **Return value**

TRUE if this function succeeds in retrieving the local date and time, otherwise FALSE.

6.6.10. QI_Mktime

This function get total seconds elapsed since 1970.01.01 00:00:00.

- **Prototype**

```
u32 QI_Mktime(QISysTimer *psysime);
```

- **Parameter**

psysime:

A pointer to “*QISysTimer*”, which will store the information of date and time.

- **Return value**

The total seconds.

6.6.11. QI_LocalTime2CalendarTime

This function convert local broken-down time to the seconds, elapsed since 1970.01.01 00:00:00

- **Prototype**

```
u32 QI_LocalTime2CalendarTime(QISysTimer *pSysTime, u32 baseyear);
```

- **Parameter**

psysime:

[out]A pointer to “*QISysTimer*”, which will store the information of date and time.

baseyear:

[in]the base year, eg. 1900, 2000

- **Return value**

Seconds elapsed since 1970.01.01 00:00:00. Return invalid value(-1) if failed.

Remark: the real year would be baseyear+pSysTime->year;

6.6.12. QI_CalendarTime2LocalTime

This function converts the seconds, elapsed since 1970.01.01 00:00:00, to local broken-down time.

- **Prototype**

```
bool QI_CalendarTime2LocalTime(u32 seconds, QISysTimer *pSysTime, u32 baseyear);
```

- **Parameter**

seconds:

[in]seconds elapsed since 1970.01.01 00:00:00

psysTime:

[out]A pointer to “QISysTimer”, which will store the information of date and time.

baseyear:

[in]the base year, eg. 1900, 2000

- **Return value**

TRUE if successful, FALSE if failed.

Remark: the real year would be baseyear+pSysTime->year;

Usage:

```
QISysTimer t;  
QI_GetLocalTime(&t);  
secs = QI_LocalTime2CalendarTime(&t, 2000);  
bool ret = QI_CalendarTime2LocalTime(secs, &t, 2000);
```

6.6.13. QI_StartGPTimer

This function starts a hardware timer, only support one lowlevel timer.

- **Prototype**

```
bool QI_StartGPTimer(u32 tick10ms, void (*Callback_timerout)(void *), void *parameter);
```

- **Parameter**

tick10ms:

the value of the timer , unit is 10 ms.

callback_timerout:

A pointer to the callback_timerout function. When time out, this function will be invoked.

parameter:

A pointer to the parameter of the callback_timerout function.

- **Return value**

TRUE if successful, FALSE if failed.

NOTE

The old name `QI_StarLowLevelTimer` is still permitted for compatibility, but not recommended.

6.6.14. QI_StopGPTimer

This function stops the hardware timer.

- **Prototype**

```
void QI_StopGPTimer(void);
```

- **Parameter**

None.

- **Return value**

None.

NOTE

The old name `QI_StopLowLevelTimer` is still permitted for compatibility, but not recommended.

6.6.15. QI_Alarm_StartUp

This function sets an alarm clock that will resume the module later.

- **Prototype**

```
s32 QI_Alarm_StartUp(QISysTimer * datetime,u8 recurr);
```

- **Parameter**

datetime:

[out] Pointer to the "QISysTimer" object.

```
typedef struct QISysTimerTag
```

```
{  
    unsigned short year;      /* A short format of year, 0-127 */  
    unsigned char month;  
    unsigned char day;  
    unsigned char hour;  
    unsigned char minute;
```

```
    unsigned char second;  
}  
QISysTimer;
```

recurr:

0: start alarm only one time.1: repeat alarm every day.2: repeat alarm every week.3: repeat alarm every month.

- **Return Value**

QL_RET_OK indicates this function succeeds.

QL_RET_ERR_PARAM indicates parameter error.

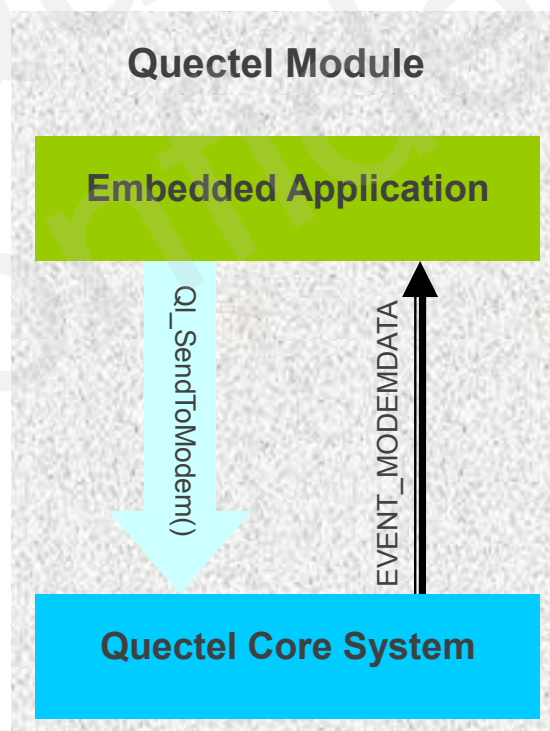
QL_RET_ERR_INVALID_TIMER indicates invalid timer.

6.7. FCM API

The required head file is “*ql_fcm.h*” for these APIs in this section. The examples in OpenCPU SDK show the proper usages of these methods.

6.7.1. Virtual modem port

The following diagram illustrates how Embedded Application communicates with Core System via the virtual modem port.



6.7.2. QI_OpenModemPort

This function will open virtual modem serial port. Before Embedded Application sends data to Core System or receive data from Core System by the virtual modem serial port, the port must be opened successfully in advance.

- **Prototype**

```
bool QI_OpenModemPort(QIPort port);
```

- **Parameters**

port:

The virtual serial port that data will be sent to.

- **Return Value**

TRUE, if this function opens the virtual serial port successfully.

FALSE, fails to open modem port.

6.7.3. QI_SendToModem

This function sends data to Core System, and the data may be AT commands, CSD data or GPRS data. For AT commands, result codes Ok or ERROR is retrieved by *QI_GetEvent* function. Refer to the EVENT section for more details. A special character “\x0d” or “\r” (carriage return) should be appended after a string of AT command to indicate the ending. For example: *QI_SendToModem("ati\x0d",4)* is the same as you type "ATI" command and press ENTER key in Hyper Terminal.

- **Prototype**

```
s32 QI_SendToModem(QIPort port, u8 *data,u16 data_len);
```

- **Parameters**

port:

The virtual serial port that data will send to.

data:

The data sent to Core System

len:

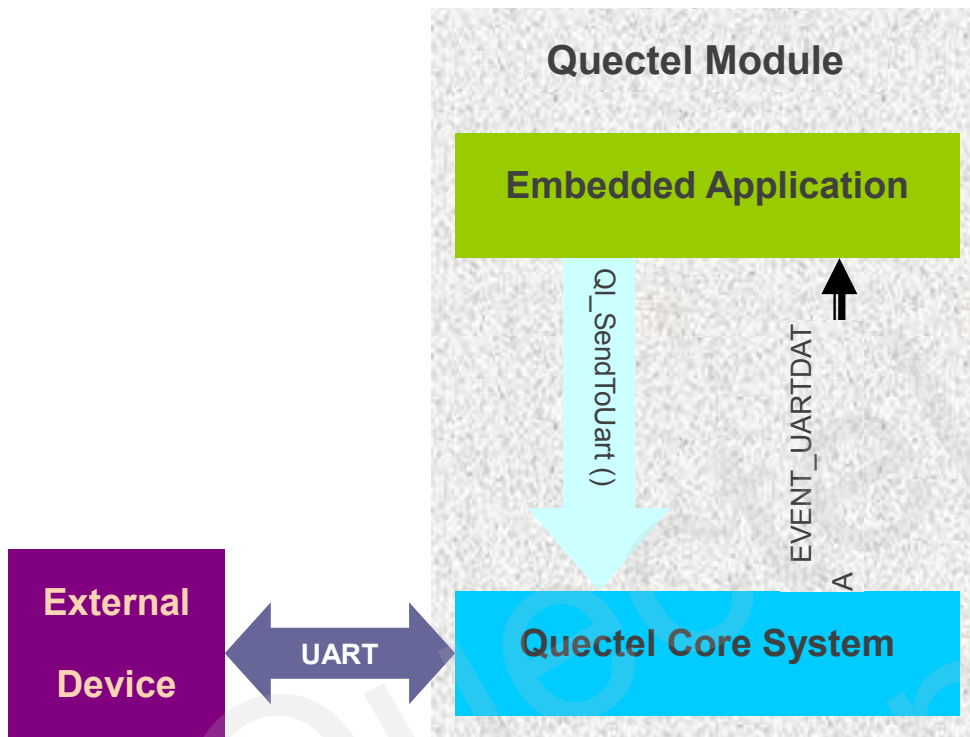
The length of the data cannot exceed 1024.

- **Return Value**

Number of bytes actually sent. If this function fails to send data, a negative number will be returned. To get extended information, please see 'Error Code Definition'.

6.7.4. UART Port

The following diagram illustrates how Embedded Application communicates with external device.



6.7.5. QI_SendToUart

This function is used to send data to the specified UART port. When the number of bytes actually sent is less than that to send, Application should stop sending data, and application will receive an event -- EVENT_UARTREADY later. After receiving this Event, Application can continue to send data, and previously unsent data should be resend.

For UART2, only after debug mode is set to basic mode, this function is available.

- **Prototype**

```
s32 QI_SendToUart(QIPort port, u8* data, u16 data_len);
```

- **Parameters**

port:

UART port.

data:

Pointer to data to be send.

len:

The length of the data cannot exceed 1024.

- **Return Value**

Number of bytes actually sent. If this function fails to send data, a negative number will be returned. To get extended information, please see 'ERROR CODES'.

6.7.6. QI_SetPortRts

This function sets the RTS of the specified UART port. If RTS is set to 0, the virtual modem port or UART port will stop sending data to Embedded .Application until RTS is set to 1 again.

- **Prototype**

```
void QI_SetPortRts(QIPort port, bool rts);
```

- **Parameter**

port:

Port name.

rts:

RTS enable.

- **Return Value**

None.

6.7.7. QI_SetUartBaudRate

This function sets baud rate of the specified UART port.

- **Prototype**

```
s32 QI_SetUartBaudRate(QIPort port,s32 rate);
```

- **Parameter**

port:

Port name.

rate:

Baud rate.

Supported baud rates:

1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600, 115200.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.7.8. QI_SetPortOwner

This function sets the id of the task which uses the specified virtual modem port or UART port.

- **Prototype**

```
s32 QI_SetPortOwner(QIPort port, QITaskId id);
```

- **Parameter**

port:

Port name.

id:

ID of the task which uses the port.

```
typedef enum QITaskIDtag{
    ql_main_task,
    ql_sub_task1,
    ql_sub_task2,
    ql_sub_task3,
    ql_sub_task4,
    ql_sub_task5,
    ql_sub_task6,
    ql_sub_task7,
    ql_sub_task8,
    ql_sub_task9,
    ql_sub_task10,
    ql_max_task
} QITaskId;
```

- **Return Value**

The return value is `QL_RET_OK` if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

NOTES

1. This interface function is designed for multitask programming.
2. The default owner of the ports is main task.
3. Only the owner task can receive the `EVENT_MODEMDATA` event from Core System.
4. Any tasks can directly call "`QI_SendToModem`" after calling "`QI_OpenModemPort`", and do not have to be `QI_SetPortOwner`.

6.7.9. QI_SetUartDCBConfig

This function sets the configuration parameters of the specified UART port.

- **Prototype**

```
s32 QI_SetUartDCBConfig(QIPort port, s32 rate, s32 dataBits, s32 stopBits, s32 parity);
```

- **Parameter**

port:

Port name.

rate:

Baud rate.

dataBits:

Data bit. Possible value: 5, 6, 7, 8.

stopBits:

Value	Meaning
1	1 stop bit
2	2 stop bits
3	1.5 stop bits

parity:

Value	Meaning
0	No parity
1	Odd parity
2	Even parity
3	Space
4	Mark

Supported baud rates:

300, 600, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600, 115200

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.7.10. QI_SetUartFlowCtrl

This function sets the flow-ctrl mode of the specified UART port. And this setting just takes effect during run-time.

- **Prototype**

```
void QI_SetUartFlowCtrl(QIPort port, QI_FlowCtrlMode wrtFlowCtrl, QI_FlowCtrlMode rdFlowCtrl);
```

- **Parameter**

port:

Port name.

wrtFlowCtrl:

Write flow control.

rdFlowCtrl:

Read flow control.

- **Return Value**

None.

6.7.11. QI_UartGetBytesAvail

This function gets the number of bytes of data in the receive buffer.

- **Prototype**

```
u16 QI_UartGetBytesAvail(QIPort port);
```

- **Parameter**

port:

Port name.

- **Return Value**

The number of bytes of data in the receive buffer.

6.7.12. QI_UartGetTxRoomLeft

This function gets the number of bytes of free space in the send buffer.

- **Prototype**

```
u16 QI_UartGetTxRoomLeft(QIPort port);
```

- **Parameter**

port:

UART port

- **Return Value**

The number of bytes of the free space in the send buffer.

6.7.13. QI_UartGetTxRestBytes

This function gets the number of bytes not sent out in the send buffer.

- **Prototype**

```
u32 QI_UartGetTxRestBytes(QIPort port, u32* totalbuffer_size);
```

- **Parameter**

port:

UART port

totalbuffer_size:

Pointer to the total number of bytes of the send buffer. This parameter may be set to NULL if do not cared.

- **Return Value**

The number of bytes not sent out in the send buffer.

6.7.14. QI_UartConfigEscape

This function configures Escape function of specified UART port.

- **Prototype**

```
bool QI_UartConfigEscape(QIPort port, u8 escapechar, u16 escguardtime);
```

- **Parameter**

port:

UART port.

escapechar:

Escape character.

escguardtime:

Interval between Escape characters (the unit is ms).

- **Return value**

TRUE means configuration successful.

FALSE means configuration failed.

6.7.15. QI_UartClrTxBuffer

This function clears send-buffer of specified UART port.

- **Prototype**

```
void QI_UartClrTxBuffer(QIPort port);
```

- **Parameter**

port:

port name.

- **Return Value**

None.

6.7.16. QI_UartClrRxBuffer

This function clears receive-buffer of specified UART port.

- **Prototype**

```
void QI_UartClrRxBuffer(QIPort port);
```

- **Parameter**

port:

port name.

- **Return Value**

None.

6.7.17. QI_UartSetGenericThreshold

This function sets the threshold of receive buffer of specified UART port. Now, this function just works for UART2 (Generic uart, not vfifo uart).

- **Prototype**

```
bool QI_UartSetGenericThreshold(QIPort port, bool benable, u32 buffer_threshold, u32 waittimeout);
```

- **Parameter**

port:

UART port, only support UART2.

benable:

Enable flag

buffer_threshold:

The number of bytes in the internal input buffer before a Data Received event occurs; The maximum value is 2048.

waittimeout:

If the number of bytes of arrived data has been less than the value of 'buffer_threshold' within the time of 'waittimeout', a Data Received event will occur as a result of expiring 'waittimeout'.

- **Return value**

TRUE indicates success, otherwise failure.

6.7.18. QI_UartGenericClearFEFlag

This function clears FE flag of specified UART port. Now, this function just works for UART2.

- **Prototype**

```
void QI_UartGenericClearFEFlag(QIPort port);
```

- **Parameter**

port:

UART port, only supports UART2.

- **Return value**

None.

6.7.19. QI_UartSetVfifoThreshold

This function sets the buffer size and the threshold in input buffer and output buffer of specified VFIFO UART port.

Now, this function just works for UART1 and UART3.

- **Prototype**

```
bool QI_UartSetVfifoThreshold(  
    QIPort port,  
    u32 rx_len,  
    u32 tx_len,  
    u32 rx_alert_length,  
    u32 tx_alert_length  
);
```

- **Parameter**

port:

UART port, only support UART1 and UART3.

rx_len:

The size (number of bytes) of the input buffer of specified UART port

tx_len:

The size (number of bytes) of the output buffer of specified UART port

rx_alert_length:

The size (number of bytes) of the internal input buffer before a Data Received event occurs; The maximum value is 2048.

tx_alert_length:

The size (number of bytes) of the internal output buffer before a Data Received event occurs; The maximum value is 3584.

- **Return value**

TRUE indicates success; otherwise failure.

6.7.20. QI_UartGetVfifoThresholdInfo

This function gets the buffer size and the threshold in input buffer and output buffer of specified VFIFO UART port. Now, this function just works for UART1 and UART3.

- **Prototype**

```
bool QI_UartGetVfifoThresholdInfo(QIPort port,
    u32 *arg_rx_len,
    u32 *arg_tx_len,
    u32 *arg_rx_alert_length,
    u32 *arg_tx_alert_length
);
```

- **Parameter**

port:

[in] UART port, only support UART1 and UART3.

arg_rx_len:

[out] Pointer to the size (number of bytes) of the input buffer of specified UART port.

arg_tx_len:

[out] Pointer to the size (number of bytes) of the output buffer of specified UART port.

arg_rx_alert_length:

[out] Pointer to the size (number of bytes) of the internal input buffer before a Data Received event occurs.

arg_tx_alert_length:

[out] Pointer to the size (number of bytes) of the internal output buffer before a Data Received event occurs.

- **Return value**

TRUE indicates success; otherwise failure.

6.7.21. QI_UartMaxGetVfifoThresholdInfo

This function gets the possible maximum value of the buffer size and the threshold in input buffer and output buffer of specified VFIFO UART port.

Now, this function just works for UART1 and UART3.

- **Prototype**

```
bool QI_UartMaxGetVfifoThresholdInfo(QIPort port,
    u32 *arg_max_rx_len,
    u32 *arg_max_tx_len,
    u32 *arg_max_rx_alert_length,
    u32 *arg_max_tx_alert_length
);
```

- **Parameter**

port:

[in] UART port, only support UART1 and UART3.

Arg_max_rx_len:

[out] Pointer to the possible maximum size (number of bytes) of the input buffer of specified UART port.

arg_max_tx_len:

[out] Pointer to the possible maximum size (number of bytes) of the output buffer of specified UART port.

arg_max_rx_alert_length:

[out] Pointer to the possible maximum size (number of bytes) of the internal input buffer before a Data Received event occurs.

arg_max_tx_alert_length:

[out] Pointer to the possible maximum size (number of bytes) of the internal output buffer before a Data Received event occurs.

- **Return value**

TRUE indicates success; otherwise failure.

6.7.22. QI_UartRcvData

This function reads data from specified physical serial port.

If the calling origin is in the EVENT_UARTDATA event, caller should not call this function before handling the data about the EVENT_UARTDATA event.

- **Prototype**

```
s32 QI_UartRcvData(QIPort port, u8* buffer_read);
```

- **Parameter**

port:

Port number.

buffer_read:

[out] Pointer to the read data, and the buffer size must great than or equal to 1024 bytes.

- **Return value**

The length of read data will be returned if the function succeeds. Otherwise, an Error Code will be returned.

NOTE

The old name QI_UartDirectnessReadData is still permitted for compatibility, but not recommended.

6.7.23. QI_UartQueryDataMode

This function queries the working mode (Command Mode or Data Mode) of specified virtual serial port.

- **Prototype**

```
int QI_UartQueryDataMode(QIPort port);
```

- **Parameter**

port:

Port number.

- **Return value**

1 indicates data mode, 0 indicates command mode and -1 indicates invalid port.

6.7.24. QI_UartForceSendEscape

This function notifies the virtual serial port to quit from Data Mode, and return back to Command Mode.

- **Prototype**

```
bool QI_UartForceSendEscape(QIPort port);
```

- **Parameter**

port:

Port number.

- **Return Value**

TRUE indicates success in calling this function.

FALSE is returned in the case of invalid port or the input port is debug port.

6.8. TCP/IP API

The interface functions in this section are declared in “*ql_tcpio.h*”.

Before transferring any data packet between the mobile and the network, a PDP context (OpenCPU uses only IP context) must be defined (refer to the “*Ql_GprsAPNSet*” function) and activated (refer to the “*Ql_GprsNetworkActive*” function).

Developers can define up to **10 PDP-context-profiles** (refer to the *Ql_GprsAPNSet* function), but only **two** can be activated at the same time.

The examples in the “*example_tcpip.c*” and “*example_tcplong.c*” of OpenCPU SDK show the proper usages of these methods.

NOTE

The maximum number of socket is 6.

6.8.1. Possible error codes

The error codes are enumerated in the “*ql_soc_error_enum*” as below.

```
typedef enum
{
    QL_SOC_SUCCESS           = 0,
    QL_SOC_ERROR             = -1,
    QL_SOC_WOULDBLOCK        = -2,
    QL_SOC_LIMIT_RESOURCE    = -3, /* limited resource */
    QL_SOC_INVALID_SOCKET    = -4, /* invalid socket */
    QL_SOC_INVALID_ACCOUNT   = -5, /* invalid account id */
    QL_SOC_NAMETOOLONG       = -6, /* address too long */
    QL_SOC_ALREADY           = -7, /* operation already in progress */
    QL_SOC_OPNOTSUPP         = -8, /* operation not support */
}
```

```

QL_SOC_CONNABORTED      = -9, /* Software caused connection abort */
QL_SOC_INVALID          = -10, /* invalid argument */
QL_SOC_PIPE             = -11, /* broken pipe */
QL_SOC_NOTCONN          = -12, /* socket is not connected */
QL_SOC_MSGSIZE          = -13, /* msg is too long */
QL_SOC_BEARER_FAIL      = -14, /* bearer is broken */
QL_SOC_CONNRESET        = -15, /* TCP half-write close, i.e., FINED */
QL_SOC_DHCP_ERROR       = -16,
QL_SOC_IP_CHANGED       = -17,
QL_SOC_ADDRINUSE        = -18,
QL_SOC_CANCEL_ACT_BEARER = -19 /* cancel the activation of bearer */
} ql_soc_error_enum;

```

6.8.2. QI_GprsAPNSet

This function sets the authentication parameters apn/login/password to use with a profile ID during PDP activation.

• **Prototype**

```
s32 QI_GprsAPNSet(u8 profileid, u8 *apn, u8 *userId, u8 *password, QI_Callback_GprsAPNSet gprsapnset);
```

• **Parameters**

Profileid:

PDP context profile, which ranges from 0 to 1.

apn:

NULL-terminated APN characters.

userId:

User Id, NULL-terminated characters.

password:

Password, NULL-terminated characters.

gprsapnset:

Callback function. The Core System will invoke this callback function to notify Embedded Application whether this function succeeds or not.

• **Return Value**

The possible return values are as follows:

QL_SOC_WOULDBLOCK

The application should wait, till the callback function is called. And the application can get the information of success or failure in callback function.

QL_SOC_INVALID	Invalid argument
----------------	------------------

QL_SOC_ALREADY	The function is running.
----------------	--------------------------

6.8.3. QI_GprsAPNGet

This function gets the authentication parameters apn/login/password with a profile id.

- Prototype**

```
s32 QI_GprsAPNGet(u8 profileid, QI_Callback_GprsAPNGet gprsapnget);
```

- Parameters**

Profileid:

PDP context profile, which ranges from 0 to 1.

gprsapnget:

Callback function. The Core System will invoke this callback function to notify Embedded Application whether this function succeeds or not.

- Return Value**

The possible return values are as follows:

QL_SOC_WOULDBLOCK	The application should wait, till the callback function is called. The application can get the information of success or failure in callback function.
-------------------	--

QL_SOC_INVALID	Invalid argument
----------------	------------------

QL_SOC_ALREADY	The function is running.
----------------	--------------------------

6.8.4. QI_GprsNetworkInitialize

This function initializes the PDP context.

- Prototype**

```
s32 QI_GprsNetworkInitialize(u8 contextid, u8 profileid, OpenCpuTcplp_Callback_t *callback_func);
```

- Parameters**

contextid:

OpenCPU supports two **PDP-contexts** to the destination host at a time. This parameter can be 0 or 1.

profileid:

PDP context profile, which ranges from 0 to 1.

callback_func:

This callback is called by OpenCPU to inform Embedded Application whether this function succeeds or not. And this callback function should be implemented by Embedded Application. This callback function is defined as below:

```
typedef struct
{
    void(*callback_network_activated)(u8 contexid);
    void(*callback_network_deactivated)(u8 contexid, s32 error_cause, s32 error);
    void(*callback_socket_connect)(u8 contexid, s8 sock, bool result, s32 error_code);
    void(*callback_socket_close)(u8 contexid, s8 sock, bool result, s32 error_code);
    void(*callback_socket_accept)(u8 contexid, s8 sock, bool result, s32 error_code);
    void(*callback_socket_read)(u8 contexid, s8 sock, bool result, s32 error_code);
    void(*callback_socket_write)(u8 contexid, s8 sock, bool result, s32 error_code);
}OpenCpuTcpIp_Callback_t;
```

callback_network_activated t:

Refer to “*Ql_GprsNetworkActive*”.

callback_network_deactivated:

Refer to “*Ql_GprsNetworkDeactive*”.

callback_socket_connect:

Refer to “*Ql_SocketConnect*”.

callback_socket_close:

This callback is called by OpenCPU when peer closes the socket TCP connection.

callback_socket_accept:

Refer to “*Ql_SocketAccept*”.

callback_socket_read:

Refer to “*Ql_SocketRecv*”.

callback_socket_write:

Refer to “*Ql_SocketSend*”.

• **Return value**

The possible return values are as follows:

<i>QL_SOC_SUCCESS</i>	This function succeeds.
<i>QL_SOC_INVALID</i>	Invalid argument.
<i>QL_SOC_ALREADY</i>	The GPRS network is already initialized.

6.8.5. Ql_GprsNetworkUnInitialize

This function restores the PDP context.

• **Prototype**

```
s32 Ql_GprsNetworkUnInitialize(u8 contextid);
```

- **Parameters**

contextid:

OpenCPU supports two PDP-contexts to the destination host at a time. This parameter can be 0 or 1.

- **Return value**

QL_SOC_SUCCESS if this function succeeds. Otherwise, other Error Code will be returned. To get extended information, please see Possible Error Codes.

6.8.6. Ql_GprsNetworkActive

This function activates the PDP context.

- **Prototype**

```
s32 Ql_GprsNetworkActive(u8 contextid);
```

- **Parameters**

contextid:

OpenCPU supports two PDP-contexts to the destination host at a time. This parameter can be 0 or 1.

- **Return value**

The return value is 0 if successes. Otherwise, a value of "ql_soc_error_enum" is returned.

The possible return values are as follows:

QL_SOC_SUCCESS	This function succeeds
QL_SOC_WOULDBLOCK	The application should wait, till the callback function is called. The application can get the information of success or failure in callback function.
QL_SOC_INVALID	Invalid argument
QL_SOC_ALREADY	The function is running.
QL_SOC_BEARER_FAIL	Bearer is broken

6.8.7. Ql_GprsNetworkDeactive

This function deactivates the PDP context.

- **Prototype**

```
s32 Ql_GprsNetworkDeactive(u8 contextid);
```

- **Parameters**

contextid:

OpenCPU supports two **PDP-contexts** to the destination host at a time. This parameter can be 0 or 1.

- **Return Value**

The return value is 0 if this function succeeds. Otherwise, a value of “*ql_soc_error_enum*” is returned, please see Possible Error Codes.

The possible values are as follow:

<i>QL_SOC_SUCCESS</i>	This function succeeds
<i>QL_SOC_WOULDBLOCK</i>	The application should wait, till the <i>callback_network_deactivated</i> function is called. The application can get the information of success or failure in callback function.
<i>QL_SOC_INVALID</i>	Invalid argument
<i>QL_SOC_ERROR</i>	Socket error

6.8.8. Ql_GprsNetworkGetState

This function gets the state of GPRS network and PDP context.

- **Prototype**

```
s32 Ql_GprsNetworkGetState(u8 contextid, OpenCpuNetWorkState_e *networkstate, u8 *ps_status);
```

- **Parameters**

contextid:

[in] OpenCPU supports two PDP-contexts to the destination host at a time. This parameter can be 0 or 1.

networkstate:

[out] Pointer to “*OpenCpuNetWorkState_e*”, in which the state of PDP context is stored.

ps_status:

[out] The GPRS network state. If the GPRS network is registered, this parameter will be output with the number 1. Any other value indicates that the GPRS network is not registered.

- **Return Value**

This return value will be *QL_SOC_SUCCESS* (0) if the function succeeds. Otherwise, a value of “*ql_soc_error_enum*” is returned.

6.8.9. QI_SocketCreate

This function creates a socket. The maximum number of socket is 6.

- Prototype**

```
s8 QI_SocketCreate(u8 contextid, u8 socket_type);
```

- Parameters**

contextid:

[in] OpenCPU supports two **PDP-contexts** to the destination host at a time. This parameter can be 0 or 1.

socket_type:

[in] This parameter is one of:

```
typedef enum
{
    SOC_SOCKET_STREAM = 0, /* stream socket, TCP */
    SOC_SOCKET_DGRAM,      /* datagram socket, UDP */
    SOC_SOCKET_SMS,        /* SMS bearer */
    SOC_SOCKET_RAW         /* raw socket */
} socket_type_enum;
```

- Return Value**

The return value is the socket ID, Otherwise, a value of “*ql_soc_error_enum*” is returned. The possible returned values are as follow:

<i>QL_SOC_INVALID</i>	Invalid argument
<i>QL_SOC_BEARER_FAIL</i>	Bearer is broken
<i>QL_SOC_LIMIT_RESOURCE</i>	Exceed the maximum socket number

6.8.10. QI_SocketClose

This function closes a socket.

- Prototype**

```
s32 QI_SocketClose(s8 socketId);
```

- Parameters**

socketId:

The socket id.

- **Return value**

This return value will be `QL_SOC_SUCCESS` (0) if the function succeeds. Otherwise, a value of “`ql_soc_error_enum`” is returned.

6.8.11. QI_SocketConnect

This function establishes a connection to the socket. The host is specified by an IP address and a port number.

- **Prototype**

```
s32 QI_SocketConnect(s8 socketId, u8 address[4], u16 port);
```

- **Parameters**

socketId:

The socket id.

address:

Peer IPv4 address.

port:

Peer IPv4 port.

- **Return value**

This return value will be `QL_SOC_SUCCESS` (0) if the function succeeds. Otherwise, a value of “`ql_soc_error_enum`” is returned.

The possible returned values are as follow:

<code>QL_SOC_SUCCESS</code>	This function succeeds
<code>QL_SOC_WOULDBLOCK</code>	The application should wait, till the <code>callback_socket_connect</code> function is called. The application can get the information of success or failure in callback function.
<code>QL_SOC_INVALID_SOCKET</code>	Invalid socket

6.8.12. QI_SocketSend

The function sends data to a connected socket.

- **Prototype**

```
s32 QI_SocketSend(s8 socket, u8 *data_p, s32 dataLen);
```

- **Parameters**

socket:

The socket id.

data_p:

Pointer to the data to send.

dataLen:

Number of bytes to send.

- **Return value**

If no error occurs, “*QI_SocketSend*” returns the total number of bytes sent, which can be less than the number requested to be sent in the *dataLen* parameter. Otherwise, a value of “*ql_soc_error_enum*” is returned.

NOTES

1. *The application should call “QI_SocketSend()” circularly to send data till all the data in data_p are sent out. If the number of bytes actually sent is less than the number requested to be sent in the dataLen parameter, the application should keep sending out the left data.*
2. *If the “QI_SocketSend” returns a negative number, but not SOC_WOULDBLOCK, which indicates some error happened to the socket, the application has to close the socket by calling “QI_SocketClose()” and reestablish a connection to the socket. If the return value is SOC_WOULDBLOCK, embedded application should stop sending data, and wait for the QI_Callback_socket_write() to be invoked to continue to send data.*

6.8.13. QI_SocketRecv

This function receives data from a bound socket.

- **Prototype**

```
s32 QI_SocketRecv(s8 socket, u8 *data_p, s32 dataLen);
```

- **Parameters**

socket:

The socket id.

data_p:

Pointer to a buffer that is the storage space for the received data.

dataLen:

Length of *data_p*, in bytes.

- **Return value**

If no error occurs, “*Ql_SocketRecv*” returns the total number of bytes received. Otherwise, a value of “*ql_soc_error_enum*” is returned.

NOTES

1. The application should call “*Ql_SocketRecv()*” circularly in the *callback_socket_read* function to receive data and do data processing work till the *SOC_WOULDBLOCK* is returned.
2. If this function returns 0, which indicates the server closed the socket, the application has to close the socket by calling “*Ql_SocketClose()*” and reestablish a connection to the socket.
3. If the “*Ql_SocketRecv()*” returns a negative number, but not *SOC_WOULDBLOCK*, which indicates some error happened to the socket, the application has to close the socket by calling “*Ql_SocketClose()*” and reestablish a connection to the socket.

6.8.14. *Ql_SocketTcpAckNumber*

This function gets the TCP socket ACK number.

- **Prototype**

```
s32 Ql_SocketTcpAckNumber(s8 socket, u64 *ackedNum);
```

- **Parameters**

socket:

[in] The socket id.

ackedNum:

[out] Pointer to an u64 type that is the storage space for the TCP ACK number.

- **Return value**

If no error occurs, this return value will be *QL_SOC_SUCCESS* (0). Otherwise, a value of “*ql_soc_error_enum*” is returned.

6.8.15. *Ql_SocketSendTo*

This function sends data to a specific destination through UDP socket.

- **Prototype**

```
s32 Ql_SocketSendTo(s8 socket, u8 *data_p, s32 dataLen, u8 address[4], u16 port);
```

- **Parameters**

socket:

The socket id.

data_p:

Buffer containing the data to be transmitted.

dataLen:

Length of the data in *data_p*, in bytes.

address:

Pointer to the address of the target socket.

port:

The target port number.

- **Return Value**

If no error occurs, this function returns the number of bytes actually sent. Otherwise, a value of “*ql_soc_error_enum*” is returned.

6.8.16. QI_SocketRecvFrom

This function receives a datagram data through TCP socket.

- **Prototype**

```
s32 QI_SocketRecvFrom(s8 socket, u8 *data_p, s32 dataLen, u8 address[4], u16 *port);
```

- **Parameters**

socket:

[in] The socket id.

data_p:

[out] Buffer to store the received data.

dataLen:

[in] Length of the data in *data_p*.

address:

[out] An optional pointer to a buffer that receives the address of the connecting entity.

port:

[out] An optional pointer to an integer that contains the port number of the connecting entity.

- **Return Value**

If no error occurs, this function returns the number of bytes received. Otherwise, a value of “*ql_soc_error_enum*” is returned.

6.8.17. QI_SocketBind

This function associates a local address with a socket.

- **Prototype**

```
s32 Ql_SocketBind(s8 socketId, u8 socktype, u16 port);
```

- **Parameters**

socketId:

Descriptor identifying an unbound socket.

sockettype:

Socket type, which can be 0 (socket stream), 1 (socket datagram).

port:

Socket port number.

- **Return Value**

If no error occurs, this function returns `QL_SOC_SUCCESS (0)`. Otherwise, a value of `"ql_soc_error_enum"` is returned.

6.8.18. Ql_SocketListen

This function places a socket in a state in which it is listening for an incoming connection.

- **Prototype**

```
s32 Ql_SocketListen(s8 listensocket, u8 address[4], u16 port, u8 maxconnections);
```

- **Parameters**

listensocket:

The listened socket id.

address:

Local IPv4 address.

port:

Local IPv4 listen port.

maxconnections:

Maximum connection number.

- **Return Value**

If no error occurs, this function returns `QL_SOC_SUCCESS (0)`. Otherwise, a value of `"ql_soc_error_enum"` is returned.

6.8.19. Ql_SocketAccept

This function permits a connection attempt on a socket.

- **Prototype**

```
s8 Ql_SocketAccept(s8 listensocket, u8 address[4], u16 *port);
```

- **Parameters**

listensocket:

[in] The listen socket id.

address:

[out] An optional pointer to a buffer that receives the address of the connecting entity.

port:

[out] An optional pointer to an integer that contains the port number of the connecting entity.

- **Return Value**

If no error occurs, this function returns a socket Id, which is greater than or equal to zero. Otherwise, a value of “*ql_soc_error_enum*” is returned.

6.8.20. Ql_GetHostIpbyName

This function retrieves host IP corresponding to a host name.

- **Prototype**

```
s32 Ql_GetHostIpbyName(u8 contextid,  
    u8 *hostname,  
    u8 *addr,  
    u8 *addr_len,  
    u8 in_entry_num,  
    u8 *out_entry_num,  
    Ql_Callback_GetIpByName callback_getipbyname  
);
```

- **Parameters**

contextid:

[in] OpenCPU supports two **PDP-contexts** to the destination host at a time. This parameter can be 0 or 1.

hostname:

[in] The host name.

addr:

[out] The buffer of the host IPv4 address.

addr_len:

[out] The length of *addr*.

in_entry_num:

[in] address number

out_entry_num:

[out] get address number

callback_getipbyname:

[in] This callback is called by Core System to notify whether this function retrieves host IP successfully or not.

Syntax of Callback Function:

```
typedef void (*QI_Callback_GetIpByName)(u8 contextid,  
    bool result,  
    s32 error_code,  
    u8 num_entry,  
    u8 *entry_address  
);
```

contextid:

OpenCPU supports two PDP-contexts to the destination host at a time. This parameter can be 0 or 1.

result:

TRUE on success, or FALSE on fail

error_code:

Error code if fail

num_entry:

Get address number.

entry_address:

The host IPv4 address.

- **Return Value**

If no error occurs, this return value will be `QL_SOC_SUCCESS` (0). Otherwise, a value of “`ql_soc_error_enum`” is returned.

However, if the `QL_SOC_WOULDBLOCK` is returned, the application will have to wait till the “`callback_getipbyname`” is called to know whether this function retrieves host IP successfully or not.

6.8.21. QI_GetLocalIpAddress

This function retrieves local IP corresponding to the specified PDP context.

- **Prototype**

```
s32 QI_GetLocalIpAddress(u8 contextid, u8 ip_addr[4]);
```

- **Parameters**

contextid:

[in] OpenCPU supports two PDP-contexts to the destination host at a time. This parameter can be 0 or 1.

ip_addr:

[out] Pointer to the buffer that is the storage space for the local IPv4 address.

- **Return Value**

If no error occurs, this return value will be `QL_SOC_SUCCESS` (0). Otherwise, a value of “`ql_soc_error_enum`” is returned.

6.8.22. QI_GetDnsServerAddress

This function retrieves the DNS server's IP address.

- **Prototype**

```
s32 QI_GetDnsServerAddress(u8 contextid, u8 primary_address[4], u8 secondary_address[4]);
```

- **Parameters**

contextid:

[in] OpenCPU supports two **PDP-contexts** to the destination host at a time. This parameter can be 0 or 1.

primary_addr:

[out] Pointer to the buffer that is the storage space for the primary DNS server's IP address.

secondary_addr:

[out] Pointer to the buffer that is the storage space for the secondary DNS server's IP address.

- **Return Value**

If no error occurs, this return value will be `QL_SOC_SUCCESS` (0). Otherwise, a value of “`ql_soc_error_enum`” is returned.

6.8.23. QI_SetDnsServerAddress

This function sets the DNS server's IP address.

- **Prototype**

```
s32 QI_SetDnsServerAddress(u8 contextid,  
    bool primary_set,  
    u8 primary_address[4],  
    bool secondary_set,  
    u8 secondary_address[4]  
);
```

- **Parameters**

contextid:

[in] OpenCPU supports two **PDP-contexts** to the destination host at a time. This parameter can be 0 or 1.

primary_set:

[in] TRUE or FALSE. Enable to set the primary DNS server's IPv4 address.

primary_addr:

[in] Pointer to the buffer that is the storage space for the primary DNS server's IP address

secondary_set:

[in] TRUE or FALSE. Enable to set the secondary DNS server's IPv4 address.

secondary_addr:

[in] Pointer to the buffer that is the storage space for the secondary DNS server's IPv4 address.

- **Return Value**

If no error occurs, this return value will be `QL_SOC_SUCCESS` (0). Otherwise, a value of "`ql_soc_error_enum`" is returned.

6.8.24. `QL_SocketCheckIp`

This function checks whether an IP address is valid IP address or not. If yes, each segment of the IP address string will be converted into integer to store in "`ipaddr`" parameter.

- **Prototype**

```
s32 QL_SocketCheckIp(u8 *addressstring, u32* ipaddr);
```

- **Parameters**

Addressstring:

[in] IP address string.

ipaddr:

[out] Pointer to u32, each byte stores the IP digit converted from the corresponding IP string.

- **Return value**

The possible return values are as follow:

<code>QL_SOC_SUCCESS</code>	The IP address string is a valid IP address.
<code>QL_SOC_ERROR</code>	The IP address string is invalid.
<code>QL_SOC_INVAL</code>	Invalid argument

6.8.25. `QL_SocketSetSendBufferSize`

This function sets the length of send buffer.

- **Prototype**

```
s32 Ql_SocketSetSendBufferSize(s8 socket, u32 bufferSize);
```

- **Parameters**

socket:

Socket Id.

bufferSize:

Length of send buffer, in bytes.

NOTE

*The send buffer length would better be less than (8 * 1360) bytes.*

- **Return Value**

If no error occurs, this return value will be `QL_SOC_SUCCESS` (0). Otherwise, a value of “`ql_soc_error_enum`” is returned.

6.8.26. Ql_SocketSetRecvBufferSize

This function sets the length of receive buffer.

- **Prototype**

```
s32 Ql_SocketSetRecvBufferSize(s8 socket, u32 bufferSize);
```

- **Parameters**

socket:

Socket Id.

bufferSize:

Length of receive buffer, in bytes.

NOTE

*The send buffer length would better be less than (8 * 1360) bytes.*

- **Return Value**

If no error occurs, this return value will be `QL_SOC_SUCCESS` (0). Otherwise, a value of “`ql_soc_error_enum`” is returned.

6.8.27. QI_SocketHtonl

This function reverses the byte order in u32 integer.

- **Prototype**

```
u32 QI_SocketHtonl(u32 a);
```

- **Parameters**

a:

An u32 integer.

- **Return Value**

An u32 integer reversed.

6.8.28. QI_SocketHtons

This function reverses the byte order in u16 integer.

- **Prototype**

```
u16 QI_SocketHtons(u16 a);
```

- **Parameters**

a:

An u16 integer.

- **Return value**

An u16 integer reversed.

6.8.29. QI_SwitchHeartBeat

This function switches on/off the keep-alive function of TCP/IP, which can keep the TCP connection always online.

- **Prototype**

```
s32 QI_SwitchHeartBeat(u8 enable, s16 cycle_time, s16 repeat_time);
```

- **Parameters**

enable:

enable control, 1 switches on this function, 0 switches off this function. if this parameter is set to 0,

the time related parameters will be ignored.

cycle_time:

cycle_time (unit in second):the cycle time for sending heartbeat data packet,in second. If the cycle timer is timeout, the module sends a QUERY PACKET to server to keep the connection alive. The default value is 60, and the allowed value ranges from 60 to 1800. -1 indicates the default value will be adopted.

repeat_time:

repeat_time (unit in second):the interval for re-sending heartbeat data packet after the failure of of sending heartbeat packet. The default value is 10, and the allowed value ranges from 10 to 20. -1 indicates the default value will be adopted.

- **Return value**

QL_SOC_SUCCESS, indicates this function successes.

QL_RET_ERR_INVALID_PARAMETER, indicates the wrong input parameter.

6.8.30. Ql_Callback_GprsAPNSet

This callback function is invoked by “Ql_GprsAPNSet()”.

- **Prototype**

```
typedef void (*Ql_Callback_GprsAPNSet)(bool result, s32 error_code);
```

- **Parameters**

result:

TRUE if the “Ql_GprsAPNSet()” succeeds; Or FALSE

error_code:

Error code. If “*result*” is FALSE, an error code will be returned in “*error_code*”.

- **Return Value**

None.

6.8.31. Ql_Callback_GprsAPNGet

This callback function is invoked by “Ql_GprsAPNGet()”.

- **Prototype**

```
typedef void (*Ql_Callback_GprsAPNGet)(u8 profileid, bool result, s32 error_code, u8 *apn, u8 *userId, u8 *password);
```

- **Parameters**

Profileid:

PDP context profile, which ranges from 0 to 9.

result:

TRUE if the “*Ql_GprsAPNGet()*” succeeds; Or FALSE.

error_code:

Error code. If “*result*” is FALSE, an error code will be returned in “*error_code*”.

apn:

NULL-terminated APN characters.

userId:

User Id, NULL-terminated characters.

password:

Password, NULL-terminated characters.

- **Return Value**

None.

6.8.32. Ql_Callback_network_activated

This callback function is invoked by “*Ql_GprsNetworkActive()*” when the return value of *Ql_GprsNetworkActive()* is *QL_SOC_WOULDBLOCK*.

- **Prototype**

```
typedef void (*Ql_Callback_network_activated)(u8 contextid);
```

- **Parameters**

contextid:

OpenCPU supports two PDP-contexts to the destination host at a time. This parameter may be 0 or 1.

- **Return Value**

None.

6.8.33. Ql_Callback_network_deactivated

This callback function is invoked by “*Ql_GprsNetworkDeactive()*” when the return value of *Ql_GprsNetworkDeactive()* is *QL_SOC_WOULDBLOCK*.

- **Prototype**

```
typedef void (*CallBack_network_deactivated)(u8 contextid, s32 error_cause, s32 error);
```

- **Parameters**

contextid:

This parameter can be 0 or 1.

error_cause&:

Bearer error cause.

error:

Error reason.

- **Return Value**

None.

6.8.34. **QI_Callback_socket_connect**

This callback function is invoked by “*QI_SocketConnect()*” when the return value of *QI_SocketConnect()* is *QI_SOC_WOULDBLOCK*.

- **Prototype**

```
typedef void(*callback_socket_connect)(u8 contextid, s8 sock, bool result, s32 error_code);
```

- **Parameters**

contextid:

This parameter can be 0 or 1.

sock:

The socket id.

result:

TRUE if the “*QI_SocketConnect()*” succeeds, or FALSE

error_code:

Error code. If “*result*” is FALSE, an error code will be returned in “*error_code*”.

- **Return Value**

None.

6.8.35. **QI_Callback_socket_close**

This callback function will be invoked when the socket connection is closed.

- **Prototype**

```
typedef void(*callback_socket_close)(u8 contextid, s8 sock, bool result, s32 error_code);
```

- **Parameters**

contextid:

This parameter can be 0 or 1.

sock:

The socket ID.

result:

TRUE if succeeds, or FALSE.

error_code:

Error code. If “*result*” is FALSE, an error code will be returned in “*error_code*”.

- **Return Value**

None.

6.8.36. QI_Callback_socket_accept

Accept a connection on a socket when module is a server.

- **Prototype**

```
typedef void(*callback_socket_accept)(u8 contexid, s8 sock, bool result, s32 error_code);
```

- **Parameters**

contextid:

This parameter can be 0 or 1.

sock:

The socket id.

result:

TRUE if succeeds, or FALSE.

error_code:

Error code. If “*result*” is FALSE, an error code will be returned in “*error_code*”.

- **Return Value**

None.

6.8.37. QI_Callback_socket_read

Read data on a socket when received TCP or UDP data.

- **Prototype**

```
typedef void(*callback_socket_read)(u8 contexid, s8 sock, bool result, s32 error_code);
```

- **Parameters**

contextid:

This parameter can be 0 or 1.

sock:

The socket ID.

result:

TRUE if succeeds, or FALSE.

error_code:

Error code. If “*result*” is FALSE, an error code will be returned in “*error_code*”.

- **Return Value**

None.

6.8.38. QI_Callback_socket_write

When the return value of *QI_SocketSend()* is *QL_SOC_WOULDBLOCK*, this callback function will be invoked to enable application to continue to send TCP data.

- **Prototype**

```
typedef void(*callback_socket_write)(u8 contextid, s8 sock, bool result, s32 error_code);
```

- **Parameters**

contextid:

This parameter can be 0 or 1.

sock:

The socket id.

result:

TRUE if succeeds, or FALSE.

error_code:

Error code. If “*result*” is FALSE, an error code will be returned in “*error_code*”.

- **Return Value**

None.

6.9. CALL API

These interfaces are used for call functions. The required header file is “*QI_call.h*”. The example in the “*example_call.c*” of OpenCPU SDK shows the proper usages of these methods.

6.9.1. QI_Call_Initialize

This function initializes call functions.

- **Prototype**

```
s32 QI_Call_Initialize(QI_STCall_Callback* call_cbFunc);
```

- **Parameters**

call_cbFunc:

A pointer to the "QL_STCall_Callback".

```
typedef struct
{
    void (*OCPU_CB_DIAL)(s32 result);
    void (*OCPU_CB_RING)(u8* coming_num);
    void (*OCPU_CB_HANGUP)(void);
}QL_STCall_Callback;
```

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.9.2. QL_GetCallCntByType

This function gets the current call count by call type.

- **Prototype**

```
u8 QL_GetCallCntByType(QL_CallType call_type);
```

- **Parameters**

call_type:

Call Type, which is one value of '*QL_CallType*'.

```
typedef enum tagQL_CallType
{
    VOICE_CALL,                /* Voice Call */
    CSD_CALL = VOICE_CALL + 4, /* Circuit Switched Data */
    INVALID_CALL_TYPE = 255
} QL_CallType;
```

- **Return Value**

A nonnegative number acts as the current call number.

6.9.3. QL_Call_Dial

Dial a number.

- **Prototype**

```
s32 QL_Call_Dial(QL_CallType call_type, char* callNo);
```

- **Parameters**

call_type:

Call Type, which is one value of 'QI_CallType'. Please refer to “**QI_GetCallCntByType**”

callNo:

A pointer to call number buffer.

- **Return Value**

1. Indicates SIM card is not inserted.
2. Indicates SIM card pin is locked.
3. Indicates SIM card PUK is locked.

Or a nonnegative number indicates other error. Please see Error Code Definition.

6.9.4. QI_Call_Answer

This function answers a coming call.

- **Prototype**

```
s32 QI_Call_Answer(void);
```

- **Parameters**

None.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.9.5. QI_Call_Hangup

This function hangs up a call.

- **Prototype**

```
s32 QI_Call_Hangup(void);
```

- **Parameters**

None.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.10. SMS API

These interfaces are used for SMS functions. The required header file is “*Ql_sms.h*”. The example in the “*example_sms.c*” of OpenCPU SDK shows the proper usages of these methods.

6.10.1. Ql_SMSInitialize

This function does some initial work before sending or receiving a message. It registers the callback functions of SMS-related.

- **Prototype**

```
s16 Ql_SMSInitialize(Ql_SMS_Callback* sms_cbFunc);
```

- **Parameters**

sms_cbFunc:

A pointer to the “*Ql_SMS_Callback*” which contains pointers to “*CB_NewSMS*”, “*CB_SendSMS*” and so on. If you don't need part of them, please set them to NULL.

```
typedef struct
{
    void (*CB_NewSMS)(u16 index, QlSMSStorage storage);
    void (*CB_SendSMS)(bool result, s16 cause, u8 msg_ref);
    void (*CB_WriteSMS)(bool result, s16 cause, u16 index);
    void (*CB_DeleteSMS)(bool result, s16 cause, u16 index);
    void (*CB_ReadPDUSMS)(bool result, s16 cause, u16 index, u8 status, u8* data, u16 length);
    void (*CB_NewFlashPDUSMS)(u16 length, u8* pdu_string);
    void (*CB_PDUSMSStatusReport)(u16 length, u8* pdu_string);
    void (*CB_ReadTextSMS)(bool result, s16 cause, u16 index, u8 status, QlSMSTextMsg* sms);
    void (*CB_NewFlashTextSMS)(QlSMSTextMsg* sms);
    void (*CB_TextSMSStatusReport)(u8 fo, u8 msg_ref, u8* phone_num, QlSysTimer* scts, QlSysTimer* dt,
    u8 st);
}Ql_SMS_Callback;
```

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.10.2. Ql_SetSMSStorage

This function sets SMS storages.

- **Prototype**

```
s16 QI_SetSMSStorage(u8 storage);
```

- **Parameters**

storage:

SMS storage, which is one value of 'QISMSStorage'. *QL_SMS_STORAGE_SM* is default.

```
typedef enum QISMSStorageTag
{
    QL_SMS_STORAGE_SM = 0,
    QL_SMS_STORAGE_ME,
    QL_SMS_STORAGE_MT,
    QL_SMS_STORAGE_UNKNOWN = 255
}QISMSStorage;
```

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.10.3. QI_GetSMSStorageInfo

This function gets SMS storages info.

- **Prototype**

```
s16 QI_GetSMSStorageInfo(u8* current_mem, u8* used, u8* total);
```

- **Parameters**

current_mem:

Buffer to get current storage.

used:

Buffer to get used space of current storage.

total:

Buffer to get total space of current storage.

- **Return value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.10.4. QI_SetInfoCentreNum

This function sets SMS service centre number.

- **Prototype**

```
s16 QI_SetInfoCentreNum(u8* sca_number, u8 type);
```

- **Parameters**

sca_number:

A pointer to the SMS service centre number. If "*sca_number*" beginning with a plus, like "+8610086", type will be set to 145, else type is 129.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.10.5. QI_GetInfoCentreNum

This function gets SMS service centre number.

- **Prototype**

```
s16 QI_GetInfoCentreNum(u8* sca_number, u8 *type);
```

- **Parameters**

sca_number:

Pointer to the number buffer.

type:

Pointer to type.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.10.6. QI_SetNewSMSDirectToTE

This function makes new messages not to save in storage, like flash SMS.

- **Prototype**

```
s16 QI_SetNewSMSDirectToTE(bool op);
```

- **Parameters**

op:

1, enable this function.

0, disable this function.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.10.7. QI_SetSMSFormat

This function sets the format of SMS, PDU or text.

- **Prototype**

```
s16 QI_SetSMSFormat(u8 format);
```

- **Parameters**

format:

0 is PDU and 1 is text. 0 is default.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.10.8. QI_ReadSMS

This function reads an indexed message. If "*QI_SetSMSFormat*" is set to 0 (PDU mode), "*CB_ReadPDUSMS*" will be invoked. If "*QI_SetSMSFormat*" is set to 1 (text mode), "*CB_ReadTextSMS*" will be invoked. PDU mode is the default mode.

- **Prototype**

```
s16 QI_ReadSMS(u16 index, bool mode);
```

- **Parameters**

index:

Message index to read in message list.

mode:

0 changes status of the specified SMS record (unread to read) and 1 does not change.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.10.9. QI_DeleteSMS

This function deletes SMS messages in current storage.

- Prototype**

```
s16 QI_DeleteSMS(u16 index,u8 flag);
```

- Parameters**

index:

The index number of SMS message.

flag:

Delete flag, which is one value of 'QISMSDeleteFlag'.

```
typedef enum QISMSDeleteFlagTag
```

```
{
    QL_SMS_DEL_INDEXED_MSG = 0, /* Single message by index */
    QL_SMS_DEL_READ_MSG,        /* Already read messages */
    QL_SMS_DEL_READ_SENT_MSG,    /* Read and sent messages */
    QL_SMS_DEL_READ_SENT_UNSENT_MSG, /* Read ,sent and unsent messages */
    QL_SMS_DEL_ALL_MSG, /* All messages in current storage */
}QISMSDeleteFlag;
```

- Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.10.10. QI_ListSMS

This function gets message list in current storage by "QISMSStatus"

- Prototype**

```
s16 QI_ListSMS(QISMSListNode* sms_list, kal_uint16 node_count, kal_uint8 by_status, kal_uint16* sms_count);
```

- Parameters**

sms_list:

Buffer to get message list in current storage. 'QISMSListNode' define as below.

```
typedef struct QISMSListNodeTag
```

```
{
    u16 index;
    QISMSStatus status;
}QISMSListNode;
```

node_count:

The node count of sms list buffer.

by_status:

One value of 'QISMSStatus'.

```
typedef enum QISMSStatusTag
{
    QL_SMS_STAT_UNREAD = 0,
    QL_SMS_STAT_READ,
    QL_SMS_STAT_UNSENT,
    QL_SMS_STAT_SENT,
    QL_SMS_STAT_ALL
}QISMSStatus;
```

sms_count:

Actual SMS count found in current storage.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.10.11. QI_GetUnReadSMS

This function reads first unread message. If no unread message in current storage, the return value is -509(QL_RET_ERR_SMS_MSG_EMPTY) defined in ERROR CODES.

- **Prototype**

```
s16 QI_GetUnReadSMS(bool mode);
```

- **Parameters**

mode:

0 changes status of the specified SMS record(unread to read). 1 doesn't change.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.10.12. QI_SendPDUSMS

This function sends a message with PDU format. SMS format will be set to 0 automatic.

- **Prototype**

```
s16 QI_SendPDUSMS( u8 * pdu_string , u16 length,);
```

- **Parameters**

pdu_string:

A pointer to the PDU string.

length:

The length of PDU string, including sca length.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.10.13. QI_SetTextSMSPara

This function sets some parameters in text mode.

- **Prototype**

```
s16 QI_SetTextSMSPara(u8 chset,bool isflashsms, bool status_report);
```

- **Parameters**

chset:

TE character set, which is one value of '*QI_SMSChset*'. *QL_SMS_CHSET_GSM* is default.

typedef enum QISMSChsetTag

```
{  
    QL_SMS_CHSET_GSM = 0,  
    QL_SMS_CHSET_UCS2 = 1,  
    QL_SMS_CHSET_8BIT = 2,  
}QISMSChset;
```

isflashsms:

1 is flash SMS and 0 is common SMS. 0 is default.

status_report:

1 means needing SMS status report. After DA (destination address) receives SMS, SMS service centre will send SMS status report to SA (source address).

0 means do not need SMS status report. 0 is default.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.10.14. QI_SendTextSMS

This function sends a message with text format. SMS format will be set to 1 automatic.

- **Prototype**

```
s16 QI_SendTextSMS(u8 * phone_num, u8 * msg ,u16 length);
```

- **Parameters**

number:

A pointer to the phone number buffer.

msg:

A pointer to the message text buffer.

length:

The length of text content. The max length determined by character set. GSM is 160, UCS2 and 8BIT is 140.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.10.15. QI_SMS_DCS_ToCSCS

This function converts the characters of sms->data to go with the terminal if is necessary.

- **Prototype**

```
void QI_SMS_DCS_ToCSCS(QISMSTextMsg* sms, QISMSCSCS cscs);
```

- **Parameters**

sms:

A pointer to the message text buffer.

cscs:

The type of Character set, one value of "QISMSCSCS"

```
typedef enum QISMSCSCSTag
```

```
{
```

```
    CSCS_CHSET_IRA,  
    CSCS_CHSET_HEX,  
    CSCS_CHSET_GSM,  
    CSCS_CHSET_PCCP437,  
    CSCS_CHSET_PCDN,  
    CSCS_CHSET_88591,  
    CSCS_CHSET_UCS2,
```

```
CSCS_CHSET_UCS2_0X81,  
CSCS_CHSET_TOTAL_NUM  
}QISMSCSCS;
```

- **Return value**

None.

6.10.16. CB_NewSMS

This callback function is invoked when a new message comes. The new message stores in current SMS storage.

Prototype

```
typedef void (*CB_NewSMS)(u16 index,QISMSStorage storage);
```

- **Parameters**

index:

Message index.

Storage:

SMS storage

- **Return value**

None.

6.10.17. CB_SendSMS

This callback function is invoked by "QI_SendTextSMS" or "QI_SendPDUSMS"

- **Prototype**

```
typedef void (*CB_SendSMS)(bool result, s16 cause,u8 msg_ref);
```

- **Parameters**

result:

0 indicates failed and 1 indicates succeed.

cause:

0 means no error. If it is negative, please see ERROR CODES. If it is positive, please see CMS ERROR definition.

msg_ref:

Message reference. In a standard module, when a message is sent successfully, serial port will return "+CMGS: <MR>". In OpenCPU, the value of MR is corresponding with the second parameter of "CB_TextStatusReport". To get more info about MR, please refer the chapter 9.2.3.6 of "GSM 03.40" technical specification.

- **Return Value**

None.

6.10.18. CB_ReadPDUSMS

This callback function is invoked by “QI_ReadSMS” when “QI_SetSMSFormat” is set to 0.

- **Prototype**

```
typedef void CallBack_ReadPDUSMS(bool result, s16 cause,u16 index, u8 status, u8* data, u16 length);
```

- **Parameters**

result:

0 indicates failed and 1 indicates succeed.

cause:

0 means no error. If it is negative, please see ERROR CODES. If it is positive , please see CMS ERROR definition.

index:

Messegae index.

status:

One value of 'QISMSStatus'

data:

A pointer to the PDU string.

length:

The length of PDU string.

- **Return Value**

None.

6.10.19. CB_DeleteSMS

This callback function is invoked by “QI_DeleteSMS”.

- **Prototype**

```
typedef void CallBack_DeleteSMS(bool result, s16 cause,u16 index)
```

- **Parameters**

result:

0 indicates failed and 1 indicates succeed.

cause:

0 means no error. If it is negative, please see ERROR CODES. If it is positive, please see CMS

ERROR definition.

index:

Messeege index.

- **Return value**

None.

6.10.20. CB_NewFlashPDUSMS

This callback function is invoked when a flash messeege comes. Flash messeege doesn't store in stororage.

- **Prototype**

```
typedef void (*CB_NewFlashPDUSMS)(u16 length, u8* pdu_string);
```

- **Parameters**

length:

The length of PDU string.

pdu_string:

A pointer to the PDU string.

- **Return Value**

None.

6.10.21. CB_PDUSStatusReport

If the SMS status report function is enabled, SMS service centre will send SMS status report to SA (source address), after DA (destination address) receives SMS. If "QI_SetSMSFormat" is set to 0, this callback function will be invoked.

- **Prototype**

```
typedef void (*CB_PDUSStatusReport)(u16 length, u8* pdu_string);
```

- **Parameters**

length:

The length of PDU string.

pdu_string:

A pointer to the PDU string.

- **Return Value**

None.

6.10.22. CB_ReadTextSMS

This callback function is invoked by “QI_ReadTextSMS” when “QI_SetSMSFormat” is set to 1.

- **Prototype**

```
typedef void (*CB_ReadTextSMS)(bool result, s16 cause, u16 index, u8 status, QISMSTextMsg* sms);
```

- **Parameters**

result:

0 indicates failed and 1 indicates succeed.

cause:

0 means no error. If it is negative, please see ERROR CODES. If it is positive, please see CMS ERROR definition.

index:

Message index.

status:

One value of 'QISMSStatus'

sms:

A pointer to “QISMSTextMsg”.

```
#define QL_MAX_PHONE_NUM    41  
#define QL_MAX_TEXT_SMS_LENGTH 160
```

```
typedef struct QISMSTextMsgTag
```

```
{  
    u8 phone_num[QL_MAX_PHONE_NUM];  
    u8 num_type;  
    u8 udh[QL_MAX_TEXT_SMS_LENGTH - 1];    /*user data header*/  
    u8 udh_len;  
    u8 data[QL_MAX_TEXT_SMS_LENGTH];  
    u16 data_len;  
    QISMSChset chset;  
    QISysTimer scts;    /* SMS server center time stamp*/  
}QISMSTextMsg;
```

- **Return Value**

None.

6.10.23. CB_NewFlashTextSMS

This callback function is invoked when a flash message comes. Flash message does not store in storage.

- **Prototype**

```
typedef void (*CB_NewFlashTextSMS)(QISMSTextMsg* sms);
```

- **Parameters**

sms:

A pointer to “*QISMSTextMsg*”.

- **Return Value**

None.

6.10.24. CB_TextStatusReport

If the SMS status report function is enabled, SMS service centre will send SMS status report to SA (source address), after DA (destination address) receives SMS. If “*QI_SetSMSFormat*” is set to 1, this callback function will be invoked.

- **Prototype**

```
typedef void CallBack_TextStatusReport(u8 fo,u8 msg_ref, u8* phone_num, QISysTimer* scts,  
QISysTimer* dt, u8 st);
```

- **Parameters**

fo:

1 indicates that DA received a text message successfully and 0 means not.

msg_ref:

Message reference,

phone_num:

Phone number.corresponding with the third parameter of “*CB_SendSMS*”.

scts:

Service centre time stamp.

dt:

Discharge time, which indicates the time at which a previously submitted SMS-SUBMIT was successfully delivered to or attempted to deliver to the recipient SME or disposed by SC. To get more info about DT, please refer to the chapter 9.2.3.13 of “GSM 03.40” technical specification.

st:

The status indicates the status of previously submitted SMS-SUBMIT for which a Status-Report has been requested. To get more information about ST, please refer to the chapter 9.2.3.15 of “GSM 03.40” technical specification.

- **Return Value**

None.

6.11. TTS API

These interfaces are used for TTS (Text to speech) functions. The required header file is “*Ql_tts.h*”. The example in the “*example_tts.c*” of OpenCPU SDK shows the proper usages of these methods.

6.11.1. Ql_TTS_Initialize

This function initializes the TTS function.

- **Prototype**

```
s32 Ql_TTS_Initialize(OCPU_CB_TTS_PLAY cb_play);
```

- **Parameters**

cb_play:

A pointer to callback function.

```
typedef void (*OCPU_CB_TTS_PLAY)(s32 res);
```

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.11.2. Ql_TTS_Play

Text to speech.

- **Prototype**

```
s32 Ql_TTS_Play(u8* content, u8 len);
```

- **Parameters**

content:

A pointer to text.

len:

The length of text to speech. $len \leq 1024$

- **Return Value**

ivTTS_ERR_OK indicates success. Negative indicates failure. Please see Error Code Definition.

6.11.3. QI_TTS_PlayInCall

Play TTS Text as background audio during calling.

- **Prototype**

```
s32 QI_TTS_PlayInCall (u8* content, u8 len, u8 ul_volume, u8 dl_volume);
```

- **Parameters**

content:

A pointer to text.

len:

The length of text to speech. len <= 1024

ul_volume:

the volume at the remote end. The value ranges from 0 to 7. If set to 0, the side will not hear the TTS sound.

dl_volume:

The volume at the local. The value ranges from 0 to 7. If set to 0, the side will not hear the TTS sound.

- **Return Value**

QL_RET_OK, success

QL_RET_ERR_GENERAL_FAILURE, fail to play tts.

QI_RET_NOT_SUPPORT, not support this function.

6.11.4. QI_TTS_Stop

This function stops speech.

- **Prototype**

```
s32 QI_TTS_Stop(void);
```

- **Parameters**

None.

- **Return Value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise, the return value is an error code. To get extended error information, please see ERROR CODES.

6.11.5. QI_TTS_Query

This function checks status of TTS.

- **Prototype**

```
s32 QI_TTS_Query(void);
```

- **Parameters**

None.

- **Return value**

Nonnegative value indicates status of TTS, which is one value of 'ivTTS_status'.

```
typedef enum
{
    ivTTS_STATUS_IDLE,           /* invalid */
    ivTTS_STATUS_INITIALIZED,    /* initialized */
    ivTTS_STATUS_PAUSE,         /* pause */
    ivTTS_STATUS_SYNTHESIZING,   /* synthesizing */
    ivTTS_STATUS_PLAYING,       /* playing */
    ivTTS_STATUS_SYNTHESIZED,    /* synthesized */
    ivTTS_STATUS_PALYEND        /* played */
}ivTTS_status;
```

Negative indicates failure. Please see Error Code Definition.

6.12. MULTITASK API

These interfaces are used for multithread programming. The required header file is "QI_multitask.h". The example in the "example_multitask.c" of OpenCPU SDK shows the proper usages of these methods.

6.12.1. Main Task

The entrance of the main task is *ql_entry()*, please refer to chapter "Least Application Code". The stack of main task is defined by using *QL_TASK_STACK_SIZE* in "ql_customer_config.c".

For example, the following defines the stack of main task as 4K bytes.

```
/*-----
 * Stack size for main task:
 * range from 1K to 10K, and the default value is 4K.
 *-----*/
```

```
#define QL_TASK_STACK_SIZE (4*1024)
const u32 qlMainTaskStackSize = QL_TASK_STACK_SIZE;
const u32 qlMainTaskPriority = 200;    // 200-255, main task priority. the smaller, the greater priority
const u32 qlMainTaskExtqsize = 10;    // 10-30, the length of message queue of main task
```

NOTES

1. Stack Size

By default, the stack size is set to 4KB. If some operations with file will be done, the stack size must be set to at least 4KB.

2. Message Queue

If messages are sent to task incessantly, developer should avoid calling these functions: "QL_Sleep()", "QL_osTakeSemaphore()" and "QL_osTakeMutex()". These functions will block the task, so that the task cannot fetch message from the message queue. If the message queue is filled up, the system will automatically reboot unexpectedly.

6.12.2. Subtasks

The subtask is defined in the array of "QIMutitask SubMutitaskArray()" in "ql_customer_config.c". And the maximal number of subtasks is 10.

For example, the following defines the four subtasks.

```
QIMutitask SubMutitaskArray[] = /* max 10 subtasks */
{
    {emapple_subtask1_entry, 1024/*TaskStackSize*/, 201/*TaskPriority*/, 11/*TaskExtqsize*/},
    {emapple_subtask2_entry, 1024, 200, 12},
    {emapple_subtask3_entry, 1024, 200, 13},
    {emapple_subtask4_entry, 1024, 200, 14},
    {NULL, 0}, // Must be "NULL" at the end
};

typedef struct QIMutitaskTag
{
    void (*MultiTaskEntry)(s32 TaskId); // Subtask Entrance Procedure
    u32 TaskStackSize; // The stack size of subtask. Range from 1K to 10K.
    u32 TaskPriority; // The subtask priority. Range from 200 to 255, the smaller, the greater priority.
    u32 TaskExtqsize; // The length of message queue of subtask. Range from 10 to 30.
}QIMutitask;
```

NOTES

Strongly recommended, developer should set the priorities of subtasks to the same as that of the main task usually. An improper set of priorities of tasks probably brings some unexpected problems, such as subtasks' cannot be booted, jobs cannot be executed, and so on.

6.12.3. Possible Error Code

The frequent error-codes, which APIs in multitask programming could return, are enumerated in the “*ql_os_error_enum*” as below.

```
typedef enum {  
    OS_SUCCESS,  
    OS_ERROR,  
    OS_Q_FULL,  
    OS_Q_EMPTY,  
    OS_SEM_NOT_AVAILABLE,  
    OS_WOULD_BLOCK,  
    OS_MESSAGE_TOO_BIG,  
    OS_INVALID_ID,  
    OS_NOT_INITIALIZED,  
    OS_INVALID_LENGTH,  
    OS_NULL_ADDRESS,  
    OS_NOT_RECEIVE,  
    OS_NOT_SEND,  
    OS_MEMORY_NOT_VALID,  
    OS_NOT_PRESENT,  
    OS_MEMORY_NOT_RELEASE  
} ql_os_error_enum;
```

6.12.4. Ql_osSendEvent

This function transmits messages between tasks. The destination task will receive *EVENT_MSG* through “*Ql_GetEvent*”.

- **Prototype**

```
s32 Ql_osSendEvent(s32 desttaskid, u32 data1, u32 data2);
```

- **Parameters**

desttaskid:

The maximum value is 10. The destination task is main task if the value is 0. The destination

task is subtask if the value is between 1 and 10.

data1:

user data.

data2:

user data.

- **Return Value**

The return value is *OS_SUCCESS* if this function succeeds. Otherwise an error code is returned.

Possible values are:

OS_SUCCESS

OS_INVALID_ID

OS_MEMORY_NOT_VALID

OS_Q_FULL

6.12.5. QI_osCreateMutex

This function creates a MUTEX.

- **Prototype**

```
u32 QI_osCreateMutex(char *mutexname);
```

- **Parameters**

mutexname:

Name of the MUTEX to be created.

- **Return Value**

The created MUTEX ID.

6.12.6. QI_osTakeMutex

This function obtains an instance of the specified MUTEX.

- **Prototype**

```
void QI_osTakeMutex(u32 mutexid);
```

- **Parameters**

mutexid:

Destination MUTEX to be taken.

- **Return value**

None.

6.12.7. QI_osGiveMutex

This function releases an instance of the specified MUTEX.

- **Prototype**

```
void QI_osGiveMutex(u32 mutexid);
```

- **Parameters**

mutexid:

Destination MUTEX to be given.

- **Return Value**

None.

6.12.8. QI_osCreateSemaphore

This function creates a counting semaphore.

- **Prototype**

```
u32 QI_osCreateSemaphore(char *semname, u32 initial_count);
```

- **Parameters**

semname:

Name of the semaphore to be created.

initial_count:

The semaphore initial value.

- **Return Value**

The created semaphore.

6.12.9. QI_osTakeSemaphore

This function obtains an instance of the specified semaphore.

- **Prototype**

```
u32 QI_osTakeSemaphore(u32 semid, bool wait);
```

- **Parameters**

semid:

The destination semaphore to be taken.

wait:

The waiting style determines if a task waits infinitely or returns immediately.

- **Return Value**

The return value is `OS_SUCCESS` if this function succeeds. Otherwise an error code is returned.

Possible values are:

`OS_SUCCESS`

`OS_SEM_NOT_AVAILABLE`

6.12.10. QI_osGiveSemaphore

This function releases an instance of the specified semaphore.

```
void QI_osGiveSemaphore(u32 semid);
```

- **Parameters**

semid:

The destination semaphore to be given.

- **Return Value**

None.

6.12.11. QI_SetLastErrorCode

This function sets error code.

- **Prototype**

```
bool QI_SetLastErrorCode(s32 errorcode);
```

- **Parameters**

errorcode:

Error code.

- **Return Value**

True indicates success or False indicates failure.

6.12.12. QI_GetLastErrorCode

This function gets error code.

- **Prototype**

```
bool QI_GetLastErrorCode(s32 *errorcode);
```

- **Parameters**

errorcode:

A pointer to the buffer that receives the error code.

- **Return value**

True indicates success or False indicates failure.

6.12.13. QI_osGetCurrentTaskPriority

This function gets the priority of the current task.

- **Prototype**

```
u32 QI_osGetCurrentTaskPriority(void);
```

- **Parameters**

None.

- **Return Value**

The return value is task priority if this function succeeds. Otherwise an error code is returned.

6.12.14. QI_osGetCurrentTaskRemainStackSize

This function gets the left number of bytes in the current task stack.

- **Prototype**

```
u32 QI_osGetCurrentTaskRemainStackSize(void);
```

- **Parameters**

None.

- **Return Value**

The return value is number of bytes if this function succeeds. Otherwise an error code is returned.

6.12.15. QI_osChangeTaskPriority

This function changes the priority of the specified task. Developers can call this function in program run-time. But caller must be very careful to avoid deadlock.

- **Prototype**

```
u32 QI_osChangeTaskPriority(s32 desttaskid, u32 newpriority);
```

- **Parameters**

desttaskid:

Task id.

newpriority:

Task priority, ranges from 200 to 255.

- **Return value**

If success, returns previous priority. If fail, returns -1.

NOTE

When type is u32, 0xFFFFFFFF indicates -1 in computer.

6.13. OpenCPU FOTA API

The interface functions enumerated in the section are designed for OpenCPU FOTA (Firmware Over The Air) function. The required header file is “*ql_fota.h*”.

The example in the “*example_fota.c*” of OpenCPU SDK shows the proper usages of these methods.

NOTE

The FOTA function is available only in the module with 128 + 32 flash.

6.13.1. QI_Fota_App_Init

This function initialises OpenCPU FOTA_Application related functions. It is a simple API. Programmer only needs to pass the simple parameters to this API.

- **Prototype**

```
s32 Ql_Fota_App_Init(FEED_DOG *pDog);
```

- **Parameters**

pDog :

A pointer to watch dog (FEED_DOG).

```
typedef struct _TFeedDog    //Define Watchdog
{
    s16 Q_gpio_pin1;        //Watchdog GPIO pin 1, If only use one GPIO ,you can set other to -1,it
    means invalid.
    s16 Q_feed_interval1;   //GPIO1 time interval for feed dog.
    s16 Q_gpio_pin2;        //Watchdog GPIO pin 2, If only use one GPIO, you can set other to -1, it
    means invalid.
    s16 Q_feed_interval2;   //GPIO 2 time interval for feed dog.
    s16 reserved1;          //Reserve 1
    s16 reserved2;          //Reserve 2
}FEED_DOG;
```

- **Return Value**

QL_RET_OK indicates this function success.

QL_RET_ERR_PARAM indicates parameter error.

QL_RET_ERR_RAWFLASH_UNKNOW indicates unknown error.

6.13.2. Ql_Fota_App_Write_Data

This function writes the delta data of applications to the special space in the module.

- **Prototype**

```
s32 Ql_Fota_App_Write_Data(s32 length, s8* buffer);
```

- **Parameters**

length:

The length of writing (Unit: Bytes).

buffer:

A pointer to the data buffer.

- **Return value**

The return value is *QL_RET_OK* if this function succeeds. Otherwise an error code is returned.

6.13.3. Ql_Fota_App_Finish

This function tells Core System that the application completed writing the delta data of applications

- **Prototype**

```
s32 Ql_Fota_App_Finish(void);
```

- **Parameters**

None.

- **Return value**

If success, return *QL_RET_OK*.

If failure, return other error code.

6.13.4. Ql_Fota_Update

This function starts OpenCPU FOTA updating process.

- **Prototype**

```
s32 Ql_Fota_Update(void);
```

- **Parameters**

None.

- **Return Value**

If success, return *QL_RET_OK*.

If failure, return other error code.

6.14. DEBUG API

The head file *ql_trace.h* must be included so that the debug functions can be called. The examples in OpenCPU SDK show the proper usages of these methods.

6.14.1. Debug Mode

There are two working mode for UART2: BASE MODE and ADVANCE MODE.

```
typedef enum QlDebugModeTag  
{
```

```
BASIC_MODE,  
ADVANCE_MODE  
} QIDebugMode;
```

Under basic mode, application debug messages will be output as text through UART2 port. And the UART2 port can work like UART3.

Under advance mode, both application debug messages and system debug messages will be output through UART2 port with special format. The Catcher Tool provided by Quectel can be used to capture and analyze these messages.

Developers can set the working mode of UART2 by calling *QI_SetDebugMode()* with a parameter of *QIDebugMode*.

6.14.2. QI_SetDebugMode

This function sets the mode flash of debug port. The debug port will be configured as the new working mode when the module reboots next time.

- **Prototype**

```
void QI_SetDebugMode(QIDebugMode mode);
```

- **Parameter**

mode:

the debug mode type.

- **Return Value**

None.

6.14.3. QI_DebugTrace

This function formats and prints a series of characters and values through the debug serial port (UART2). Its function is same with 'sprintf'.

- **Prototype**

```
s32 QI_DebugTrace (char *fmt, ... );
```

- **Parameter**

format:

Pointer to a null-terminated multibyte string which specifies how to interpret the data. The maximum string length is 512 bytes.

Format-control string. A format specification has the following form:

%type

type, a character that determines whether the associated argument is interpreted as a character, a string, or a number.

Character	Type	Output Format
c	int	Specifies a single-byte character.
d	int	Signed decimal integer.
o	int	Unsigned octal integer.
x	int	Unsigned hexadecimal integer, using "abcdef."
f	double	Float point digit.
p	Pointer to void	Prints the address of the argument in hexadecimal digits.

• Return Value

Number of characters printed.

NOTES

1. *The string to be printed must not be larger than the maximum number of bytes allowed in buffer; otherwise, a buffer overrun can occur.*
2. *The maximum allowed number of characters to output is 512.*
3. *To print a 64-bit integer, please first convert it to characters using "QI_ QI_sprintf()", and then print the characters of the 64-bit integer.*

6.15. STANDARD LIBRARY API

Standard library API functions are defined in the file "ql_stdlib.h".

```
void    QI_itoa(char **buf, s32 i, s32 base);
void    QI_itof(char **buf, s32 i);
double  QI_atof(const char* s);
int      QI_atoi(const char* s);
char*    QI_strcpy(char* dest, const char* src);
char*    QI_strncpy(char* dest, const char* src, u16 size);
char*    QI_strcat(char* s1, const char* s2);
char*    QI_strncat(char* s1, const char* s2, u16 size);
s32      QI_strcmp(const char*s1, const char*s2);
void*    QI_memcpy(void* dest, const void* src, u16 size);
s32      QI_memcmp(const void* dest, const void*src, u16 size);
```

```
void*   QI_memmove(void* dest, const void* src, u16 size);
char*   QI_strchr(const char* s1, u16 ch);
u16     QI_strlen(const char* str);
s32     QI_strncmp(const char* s1, const char* s2, u16 size);
void*   QI_memset(void* dest, u8 value, u32 size);
char*   QI_strstr(const char* s1, const char* s2);
extern int (*QI_sprintf)(char *, const char *, ...);
```

NOTE

Above standard I/O functions are almost identical to the corresponding standard C functions, and the only difference is that these functions use Quectel defined types instead of standard C types.

7. System Configuration

In the file “*ql_customer_gpio.c*”, developer can set the initial status of the Embedded Application by configuring the ‘*Customer_user_qlconfig*’ structure with appropriate parameters.

7.1.1. Configure ‘Customer_user_qlconfig’

The “*Customer_user_qlconfig*” structure is defined as below:

```
typedef struct QICustomerConfigTag
{
    QIPinName handfreeamplifierpin;
    QIMicName handfreeinputmic;
    QIPinName headsetdetectpin;
    bool headsetadccapture;
    u8 headsetdetectdebounce;
    QIAdcName headsetdetectadc;
    u32 headsetadchigh;
    u32 headsetadclow;
    u32 headsetadcsendkey;
    bool powerautoon;
    bool powerautooff;
    bool uart3supportvfifo;
}QICustomerConfig;
```

handfreeamplifierpin:

Designate a GPIO pin to control PA.

If this field is set to *QL_PINNAME_MAX*, no pin will control PA. Or, you need to set this field to a GPIO pin. Beside, you need to configure the variable: “*Customer_QIPinConfigTable*”.

For example:

- Step 1:

Set ‘*handfreeamplifierpin*’ to *QL_PINNAME_GPIO0*.

- Sep 2:

Configure the variable: “*Customer_QIPinConfigTable*” as below.

```
{QL_PINNAME_GPIO0,          QL_PINSUBSCRIBE_UNSUB,          QL_PINMODE_1,
QL_PINPULLENABLE_ENABLE, QL_PINDIRECTION_OUT,QL_PINLEVEL_LOW,0}.
```

handfreeinputmic:

Specify the audio source, `QL_MIC_MIC1` or `QL_MIC_MIC2`.

headsetdetectpin:

Designate some GPIO pin to detect earphone's plug-in and plug-out.

If this field is set to `QL_PINNAME_MAX`, no pin will detect earphone's action. Or, you need to set this field to a GPIO pin. Besides, you need to configure the variable: *Customer_QIPinConfigTable*. For example:

- Step 1:

Set '*headsetdetectpin*' to `QL_PINNAME_GPIO0`.

- Sep 2:

Configure the variable: "*Customer_QIPinConfigTable*" as below.

```
{QL_PINNAME_GPIO0,    QL_PINSUBSCRIBE_UNSUB,    QL_PINMODE_1,    QL_PINPULL,
ENABLE_ENABLE, QL_PINDIRECTION_IN, 0, 0}.
```

- Sep 3:

Enable GPIO pin to bear the attributes of Interruption and Input Direction.

If the earphone detection function is enabled, Embedded Application will receive the `EVENT_HEADSET` event when a plug-in or plug-out action of earphone is taken.

headsetadccapture:

FALSE indicates: not capture headset ADC value. This is the default setting.

If you need to test MIC channel, you can set this field to TRUE. When the earphone is plug in or dialing, the `EVENT_HEADSET (HEADSET_ADC)` with the sampling value of ADC will be triggered.

headsetdetectdebounce:

Set the bounce time of plugging in or out earphone, unit in 10ms, the maximum possible value is 255.

headsetdetectadc:

ADC type, `QL_ADC_ADC0` or `QL_ADC_TEMP_BAT`.

If the field '*headsetdetectpin*' is set to `QL_PINNAME_MAX`, here setting will be useless.

headsetadchigh:

headsetadclow:

The upper two fields define the upper limit and lower limit of a valid ADC sampling value respectively. When a valid ADC value is sampled, the `EVENT_HEADSET (HEADSET_ADC)` with the sampling value of ADC will be triggered.

headsetadcsendkey:

This field defines the maximum level value during a coming call or the device being in calling. When the sampled ADC value is lower than the value that this field defines, the `EVENT_HEADSET (HEADSET_ADC)` with the sampling value of ADC will be triggered to indicate that the button on earphone is pressed down.

powerautoon:

When set to TRUE, the system will boot up automatically after a press is given to the power key, and Embedded Application does not need to do anything.

When set to FALSE, the EVENT_POWERKEY (POWERKEY_ON) event will be triggered. After receiving this event, Embedded Application must call *QI_PowerOnAck()* to enable the system to boot up before the earphone button is released. Or, the system will power down after the earphone button is released.

powerautooff:

When set to TRUE, the system will switch off automatically after a press is given to the PWRKEY key.

When set to FALSE, the EVENT_POWERKEY (POWERKEY_ON) event will be triggered. After receiving this event, Embedded Application may do post processing before the system switches off.

uart2supportvfifo:

Must be TRUE.

7.1.2. Example for Headset

Configure 'Customer_user_qlconfig'.

The following configuration is an example:

```
const QICustomerConfig Customer_user_qlconfig =
{
    QL_PINNAME_MAX,    /* handfreeamplifierpin */
    QL_MIC_MIC1,       /* handfreeinputmic */
    QL_PINNAME_MAX,    /* headsetdetectpin */
    FALSE,              /* headsetadccapture */
    100,               /* headsetdetectdebounce */
    QL_ADC_MAX,        /* headsetdetectadc */
    2800000,            /* headsetadchigh */
    500000,             /* headsetadclow */
    300000,             /* headsetadcsendkey */
    ...
    TRUE,              /* uart3supportvfifo */
};
```

When earphone is plug in or plug out:

```
...
Ql_GetEvent(&qlEventBuffer);
switch(qlEventBuffer.eventType)
{
    case EVENT_HEADSET:
        Headset_Event* pHeadsetEvt = & qlEventBuffer.eventType. headset_evt;
        if ( HEADSET_PLUGIN ==  pHeadsetEvt->headsettype)
        {
            // Set voice channel to headset
            Ql_VoiceCallChangePath(QL_AUDIO_PATH_HEADSET);
            // ...
        }
        else if ( HEADSET_PLUGOUT ==  pHeadsetEvt->headsettype)
        {
            // Set voice channel to loudspeaker
            Ql_VoiceCallChangePath(QL_AUDIO_PATH_LOUDSPEAKER);
            // ...
        }
        break;
default:
    break;
}
...
```

When press the button on earphone during a coming call or being in calling:

```
...
Ql_GetEvent(&qlEventBuffer);
switch(qlEventBuffer.eventType)
{
    case EVENT_HEADSET:
        Headset_Event* pHeadsetEvt = & qlEventBuffer.eventType. headset_evt;
        if ( HEADSET_KEYPRESS ==  pHeadsetEvt->headsettype)
        {
            // Answer the coming call
            // ...
        }
        else if ( HEADSET_KEYRELEASE ==  pHeadsetEvt->headsettype)
        {
            // End the active call
            // ...
        }
        break;
default:
    break;
}
```

```
}  
...
```

Quectel
Confidential

8. Appendix A - Error Codes Definition

The *ql_error.h* defines all the error codes that API functions probably return.

Error Code	Error Value	Description
QL_RET_OK	0	Normally return.
QL_RET_ERR_PARAM	-1	Parameter error
QL_RET_ERR_PORT_NOT_OPEN	-2	Port not opened
QL_RET_ERR_TIMER_FULL	-3	All timers are used up
QL_RET_ERR_INVALID_TIMER	-4	Invalid timer
QL_RET_ERR_FATAL	-5	Unknown fatal error
QL_RET_ERR_INVALID_OP	-6	Invalid operation
QL_RET_ERR_UART_BUSY	-7	UART is busy
QL_RET_ERR_INVALID_PORT	-8	Invalid serial port
QL_RET_ERR_NOMATCHVERSION	-9	No match pin version
QL_RET_ERR_NOSUPPORTPIN	-10	Not support pin operation
QL_RET_ERR_NOSUPPORTMODE	-11	Not support pin mode
QL_RET_ERR_NOSUPPORTEINT	-12	Not support EINT
QL_RET_ERR_NOSUPPORTSET	-13	Not support pin setting
QL_RET_ERR_NOSUPPORTCONTROL	-15	Not support pin control
QL_RET_ERR_PINALREADYSUBSCRIBE	-16	Pin already subscribed
QL_RET_ERR_BUSSUBBUSY	-17	BUS already subscribed
QL_RET_ERR_NOGPIOMODE	-18	Not GPIO mode
QL_RET_ERR_NORIGHTOPERATE	-19	Wrong PIN operation
QL_RET_ERR_ALREADYUNSUBSCRIBE	-20	Already unsubscribed

QL_RET_ERR_FULLI2CBUS	-21	I2C BUS is full
QL_RET_ERR_NOTSUPPORTBYHANDLE	-22	Not supported handle
QL_RET_ERR_INVALIDDBUSHANDLE	-23	Invalid BUS handle
QL_RET_ERR_NOEXISTOBJEXT	-24	Flash object not exist
QL_RET_ERR_OPERATEOBJEXTFAILED	-25	Fail to operate flash
QL_RET_ERR_OPENOBJEXTFAILED	-26	Fail to open flash object
QL_RET_ERR_WRITEOBJEXTFAILED	-27	Fail to write flash object
QL_RET_ERR_READOBJEXTFAILED	-28	Fail to read flash object
QL_RET_ERR_FLASHFULLOVER	-29	User space not enough
QL_RET_ERR_FLASHSPACE	-30	Flash space error
QL_RET_ERR_DRIVE	-31	Flash operation fatal
QL_RET_ERR_DRIVEFULLOVER	-32	Insufficient Flash space
QL_RET_ERR_INVALIDFLASHID	-33	Invalid flash ID
QL_RET_ERR_I2CHWFAILED	-34	Fail to operate I2C
QL_RET_ERR_FILEFAILED	-35	Fail to operate file
QL_RET_ERR_FILEOPENFAILED	-36	Fail to open file
QL_RET_ERR_FILENAMEETOOLENGTH	-37	Too long file name
QL_RET_ERR_FILEREADFAILED	-38	Fail to read file
QL_RET_ERR_FILEWRITEFAILED	-39	File write failed
QL_RET_ERR_FILESEEKFAILED	-40	File seek failed
QL_RET_ERR_FILENOTFOUND	-41	File not found
QL_RET_ERR_FILENOMORE	-42	File no more
QL_RET_ERR_FILEDISKFULL	-43	Disk is full
QL_RET_ERR_INVALID_BAUDRATE	-44	Invalid baud rate
QL_RET_ERR_API_NO_RESPONSE	-45	API no response
QL_RET_ERR_API_INVALID_RESPONSE	-46	API invalid response

QL_RET_ERR_SMS_EXCEED_LENGTH	-47	Sms exceed limited length
QL_RET_ERR_SMS_NOT_INIT	-48	Sms not initialed
QL_RET_ERR_INVALID_TASK_ID	-49	Invalid task id
QL_RET_ERR_NOT_IN_BASIC_MODE	-50	Not in basic mode
QL_RET_ERR_INVALID_PARAMETER	-51	Invalid paratemer
QL_RET_ERR_PATHNOTFOUND	-52	Path not found
QL_RET_ERR_GET_MEM	-53	Fail to get memory
QL_RET_ERR_GENERAL_FAILURE	-54	
QL_RET_ERR_FILE_EXISTS	-55	File existed
QL_RET_ERR_SMS_INVALID_FORMAT	-56	Invlid SMS format
QL_RET_ERR_SMS_GET_FORMAT	-57	
QL_RET_ERR_SMS_INVALID_STORAGE	-58	Invlid SMS storage
QL_RET_ERR_SMS_SET_STORAGE	-59	
QL_RET_ERR_SMS_SEND_AT_CMD	-60	
QL_RET_ERR_API_CMD_BUSY	-61	AT command is busy
QL_RET_ERR_MED_BAD_FORMAT	-70	Format is error
QL_RET_ERR_MED_BUSY	-71	Media is busy
QL_RET_ERR_MED_DISC_FULL	-72	Disc is full
QL_RET_ERR_MED_OPEN_FILE_FAIL	-73	File open fail
QL_RET_ERR_MED_BAD_FILE_EXTENSION	-74	
QL_RET_ERR_MED_WRITE_PROTECTION	-75	
QL_RET_ERR_MED_FILE_EXIST	-76	
QL_RET_ERR_MED_UNSUPPORT_FMT_IN_CALLING	-77	
QL_RET_ERR_AUD_REC_STOP_FAIL	-78	
QL_RET_ERR_MED_DRIVE_NOT_FOUND	-79	
QL_RET_ERR_MED_NO_CARD	-80	

QI_RET_ERR_MEM_FULL	-81	Memory is full
QL_ERR_DTMFSTRING_TOO_LONG	-82	DTMF string is too long
QL_ERR_WDTMF_PS_BUSY	-83	
QL_ERR_DTMF_BUSY	-84	
QL_ERR_DTMF_NO_CALLING	-85	
QL_RET_ERR_MED_UNKNOWN	-100	
QL_RET_ERR_FILE_NO_CARD	-101	
QL_RET_ERR_FILE_UNKNOWN	-200	
QI_RET_ERR_EINT_USED	-300	This eint vector is used
QL_RET_ERR_SMS_ERROR	-500	
QL_RET_ERR_SMS_NOT_INITIAL	-501	
QI_RET_ERR_SMS_NOT_READY	-502	Not ready
QL_RET_ERR_SMS_INVALID_PARAM	-503	Invalid parameter
QL_RET_ERR_SMS_OUT_OF_MEMORY	-504	Out of memory
QL_RET_ERR_SMS_INCORRECT_DATA_LENGTH	-505	Incorrect data length
QL_RET_ERR_SMS_PDU_SYNTAX_ERROR	-506	
QL_RET_ERR_SMS_INVALID_MEM_INDEX	-507	
QL_RET_ERR_SMS_CMD_CONFLICT	-508	
QL_RET_ERR_SMS_MSG_EMPTY	-509	
QL_RET_ERR_SMS_INVALID_NUMBER_STRING	-510	
QL_RET_ERR_SMS_INVALID_TEXT_CONTENT	-511	
QL_RET_ERR_SMS_NOT_SUPPORTED	-512	
QL_RET_ERR_SMS_INCORRECT_FORMAT	-513	
QL_RET_ERR_SMS_STORAGE_FULL	-514	The storage is full
QI_RET_ERR_RAWFLASH_OVERRANGE	-8001	
QI_RET_ERR_RAWFLASH_UNINITIALIZED	-8002	

QI_RET_ERR_RAWFLASH_UNKNOW	-8003	
QI_RET_ERR_RAWFLASH_INVLIDBLOCKID	-8004	
QI_RET_ERR_RAWFLASH_PARAMETER	-8005	
QI_RET_ERR_RAWFLASH_ERASEFIASH	-8006	
QI_RET_ERR_RAWFLASH_WRITEFLASH	-8007	
QI_RET_ERR_RAWFLASH_READFLASH	-8008	
QI_RET_ERR_RAWFLASH_MAXLENGATH	-8009	
QI_RET_ERR_UNKOWN	-9999	Unkown error
QI_RET_NOT_SUPPORT	-10000	API not supported

9. Appendix B Reference

Table 2: Related Documents

SN	DocumentName
[1]	Mxx AT Commands Set
[2]	Quectel_OpenCPU_Specification
[4]	Mxx OpenCPU HD

Table 3: Glossary

Glossary	Description
Application Task	OpenCPU provides an absolute application task to simulate the MCU environment. Like a normal MCU task, the Application Task has a main entrance and runs forever.
Embedded Application API	Software interfaces developed by Quectel is open to licensed Embedded Application developers. The APIs include audio API, FCM API, system API, periphery API, timer API, and debug API, standard C library API, and so on.
Embedded Application	User created application that utilizes Embedded API functions to interact with Quectel core software. The application can only run on a Quectel product.
Core System	The Core system released by Quectel, which includes the core binary file and Quectel library.
EVENT	Capitalized EVENT notion used in this document is to represent specified system EVENT in Embedded Application.

Table 4: Abbreviations

Abbreviation	Description
API	Application Programming Interface

CPU	Central Processing Unit
FCM	Flow Control Manager
FOTA	Firmware Over The Air
KB	Kilobyte
OS	Operating System
PDU	Protocol Data Unit
RAM	Random-Access Memory
ROM	Read-Only Memory
TCP/IP	Transfer Control Protocol / Internet Protocol
UART	Universal Asynchronous Receiver and Transmitter
TTS	Test To Speech